



TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN COMPUTACIÓN



Un robot cuadrúpedo que xoga ao fútbol: detectar e patear unha pelota

Estudante: Samuel Pérez Fente
Dirección: Carlos Vázquez Regueiro

A Coruña, setembro de 2025.

A meus pais, por ser sempre o meu exemplo, e aos meus amigos, por facer do camiño algo máis doado.

Agradecementos

En primeiro lugar, quero expresar o meu agradecemento máis profundo a miña nai e a meu pai, polo seu apoio incondicional ao longo de todos estes anos de estudo. Sen a súa confianza este camiño non tería sido posible.

Así mesmo, quero agradecer ao meu titor, Carlos Vázquez Regueiro, polo seu acompañamento constante, polas súas orientacións e polo tempo dedicado a resolver dúbidas e aportar claridade durante o desenvolvemento deste traballo.

O meu agradecemento tamén ao CITIC, pola cesión do robot Unitree Go2, recurso imprescindible para levar a cabo as probas prácticas que sustentan este proxecto.

Por último, non me quero esquecer dos meus amigos, que me deron ánimo nos momentos máis complicados e compartiron comigo espazos de desconexión tan necesarios nesta etapa. Grazas a todos eles por facer este proceso máis levadeiro e enriquecedor.

Resumo

Este traballo céntrase no desenvolvemento dun sistema de percepción e control para o robot cuadrúpedo Unitree Go2 EDU, orientado á detección de obxectos de interese e á execución dun comportamento autónomo de pateo. Para acadar este obxectivo intégranse técnicas de visión por computador mediante o modelo YOLO, datos procedentes do sensor LiDAR e a súa fusión no marco de referencia do robot. Así mesmo, descríbese a implementación en ROS 2, a definición de mensaxes e accións personalizadas, así como a planificación de fases que permiten a alineación co obxecto de pateo, a aproximación controlada e a execución final da acción. Os resultados mostran a viabilidade do enfoque e sentan as bases para futuras melloras en termos de precisión, robustez e aplicacións en contornas dinámicas.

Abstract

This project focuses on the development of a perception and control system for the Unitree Go2 EDU quadruped robot, aimed at detecting relevant objects and performing an autonomous kicking behavior. To achieve this goal, computer vision techniques with the YOLO model are integrated with data from the LiDAR sensor, combining them in the robot reference frame. Furthermore, the implementation in ROS 2, the definition of custom messages and actions, and the design of phases enabling object alignment, controlled approach, and final kick execution are presented. The results demonstrate the viability of the approach and establish the basis for future improvements in accuracy, robustness, and deployment in dynamic environments.

Palabras chave:

- Robot cuadrúpedo
- Visión por computador
- ROS 2
- Detección de obxectos
- Unitree Go2
- Robótica móvil
- Navegación autónoma
- LiDAR
- Cámara RGB

Keywords:

- Quadruped robot
- Computer vision
- ROS 2
- Object detection
- Unitree Go2
- Mobile robotics
- Autonomous navigation
- LiDAR
- RGB camera

Índice Xeral

1	Introdución	1
1.1	Contexto e motivación	1
1.2	Obxectivos do proxecto	2
1.3	Proposta	3
1.4	Traballo relacionado	3
1.5	Estrutura da memoria	4
2	Fundamentos tecnolóxicos	6
2.1	Hardware	6
2.1.1	Unitree Go2	6
2.1.2	Cámara RGB	7
2.1.3	LiDAR 4D	8
2.1.4	Cámara RGB-D (Intel RealSense D435i)	9
2.2	Software	10
2.2.1	Docker	10
2.2.2	Python	10
2.2.3	Visual Studio Code	10
2.2.4	Git e GitHub	11
2.2.5	LaTeX e Overleaf	11
2.3	ROS 2	11
2.3.1	Evolución respecto a ROS 1	12
2.3.2	Arquitectura de ROS 2	12
2.3.3	Nodos	12
2.3.4	Executor e rclcpp/rclpy	13
2.3.5	Tópicos	13
2.3.6	Mensaxes	13
2.3.7	Servizos	13

2.3.8	Accións	13
2.3.9	Launch files	14
2.4	CycloneDDS	14
2.5	WebRTC	14
2.6	Simuladores	15
2.6.1	Gazebo Classic	15
2.6.2	Isaac Gym/ Isaac Sim	16
2.7	YOLO	16
3	Metodoloxía e planificación do proxecto	18
3.1	Metodoloxía de desenvolvemento	18
3.2	Requisitos	19
3.2.1	Requisitos funcionais	19
3.2.2	Requisitos non funcionais	20
3.3	Actores	20
3.4	Fases do proxecto	20
3.4.1	Fase 0: Formación	20
3.4.2	Fase 1: Preparación do entorno	21
3.4.3	Fase 2: Desenvolvemento do módulo de percepción	21
3.4.4	Fase 3: Control e navegación	22
3.4.5	Fase 4: Comportamento do golpeo coa pata	22
3.4.6	Fase 5: Validación do comportamento completo	23
3.4.7	Fase 6: Documentación	23
3.5	Seguimento do proxecto	24
3.6	Recursos e custos	24
4	Deseño	28
4.1	Arquitectura do sistema	28
4.2	Modelo de datos	30
4.2.1	Detección visual	30
4.2.2	Estimación de distancias frontais	31
4.3	Módulo percepción	31
4.3.1	Nodo YOLO*	31
4.3.2	Nodo decaemento LiDAR	31
4.3.3	Nodo transformación frame	33
4.3.4	Nodo filtrado frontal	33
4.3.5	Nodo integración YOLO* con LiDAR	33
4.4	Nodo control	33

4.4.1	Fase aliñamento	34
4.4.2	Fase achegamento	38
4.4.3	Fase baixo nivel	45
5	Detalles de implementación	46
5.1	Nodo YOLO	46
5.1.1	Aplicación do modelo YOLO*	46
5.1.2	Extracción da cor HSV	47
5.2	Nodo de filtrado con decaemento	47
5.2.1	Definición da Área de Interese e Filtrado Espacial	48
5.2.2	Decaemento Temporal	48
5.3	Nodo de transformación frame	49
5.4	Nodo de filtrado frontal	49
5.4.1	Área de interese frontal	49
5.4.2	Altura dinámica segundo a distancia	50
5.4.3	Selección dos puntos máis representativos	50
5.5	Nodo integración cámara con LiDAR	51
5.5.1	Proxección dos puntos LiDAR á imaxe da cámara e asociación ás de- teccións	51
5.5.2	Estimación da posición 3D dos obxectos detectados	52
5.6	Módulo fase aliñamento	53
5.6.1	Aliñamento rotacional co obxecto de pateo	53
5.6.2	Control da distancia ao obxecto de pateo	54
5.6.3	Aliñamento lateral co obxecto destino	54
5.7	Módulo fase achegamento	55
5.7.1	Control temporal entre movementos	55
5.7.2	Retroceso por proximidade excesiva	56
5.7.3	Corrección lateral por desviación excesiva	57
5.7.4	Corrección de orientación	57
5.7.5	Avance a longa distancia	57
5.7.6	Avance a curta distancia	59
5.7.7	Aliñamento lateral co obxecto destino	59
5.8	Módulo fase baixo nivel	60
6	Probas	61
6.1	Limitacións do simulador	61
6.2	Conexión vía WiFi con CycloneDDS	62
6.3	Problemas cos robots	62

6.3.1	Problemas coa cámara do Go2A-B	62
6.3.2	Problemas co Go2A	63
6.3.3	Movimentos “suaves” e controlados	64
6.4	Probas específicas	64
6.4.1	Probas co módulo YOLO*	64
6.4.2	Probas da percepción LiDAR	66
6.4.3	Probas da integración LiDAR e cámara-YOLO*	69
6.4.4	Probas da fase aliñamento	70
6.4.5	Probas da fase achegamento	72
6.5	Probas finais	73
7	Conclusións e traballo futuro	77
7.1	Conclusións	77
7.2	Traballo futuro	80
A	Acceso ao repositorio do proxecto	82
B	Canal YouTube cos vídeos de probas	83
C	Creación y extracción de vídeos	84
C.1	Obtención das imaxes	84
C.2	Extracción de fotogramas	84
C.3	Inserción de texto	85
C.4	Resumo do proceso	86
	Relación de Acrónimos	87
	Glosario	88
	Bibliografía	89

Índice de Figuras

2.1	Robot cuadrúpede Unitree Go2 (versión EDU) con cámara montada.	7
3.1	Comparativa entre a planificación inicial e o seguimento do proxecto.	25
4.1	Diagrama xeral da arquitectura do sistema	29
4.2	Estructura interna do módulo de percepción e o seu fluxo de procesado	32
4.3	Diagrama de fluxo do aliñamento	35
4.4	Aliñamento rotacional co obxecto de pateo	36
4.5	Control da distancia do obxecto de pateo	37
4.6	Aliñamento lateral co obxecto destino	38
4.7	Diagrama de fluxo do acercamento	40
4.8	Retroceso por proximidade excesiva	41
4.9	Corrección lateral por desviación excesiva	42
4.10	Corrección de orientación cara ao obxectivo	43
4.11	Avance a longa distancia	43
4.12	Avance a curta distancia	44
4.13	Aliñamento lateral co obxecto de pateo	45
6.1	Problemas de percepción asociados á cámara integrada dos robots Go2 e primeira solución adoptada.	63
6.2	Influencia dun obxecto lateral na detección do obxecto frontal.	66
6.3	Un obxecto lateral non afecta á detección do obxecto frontal.	67
6.4	Detección correcta do obxecto frontal malia atoparse desprazado lateralmente.	67
6.5	Obxecto grande situado detrás do obxecto de interese inflúe na detección frontal.	68
6.6	Separación suficiente entre o obxecto grande e o obxecto de interese evita interferencias na detección.	68
6.7	Obxecto pequeno preto do obxecto de interese que non interfere na detección.	69

6.8	Resultados da integración LiDAR–cámara YOLO*, onde se observa que múltiples obxectos en distintas posicións son correctamente localizados en 3D. . . .	70
-----	--	----

Índice de Táboas

2.1	Especificacións principais do robot cuadrúpede Unitree Go2 [1]	7
2.2	Especificacións da cámara RGB integrada no Unitree Go2 [1]	8
2.3	Especificacións técnicas do LiDAR L1 4D incluído no Go2 [2]	9
2.4	Especificacións técnicas da cámara Intel RealSense D435i [3]	9
3.1	Coste estimado dos recursos humanos utilizados no proxecto.	26
3.2	Resumo dos custos estimados do proxecto.	27
6.1	Comparativa entre as distintas versións de YOLO empregadas no sistema de percepción. Resultados para 739 imaxes.	65
6.2	Valores das variables empregadas nas distintas configuracións das probas na fase de aliñamento, agrupadas por subfase.	71
6.3	Resultados comparativos das configuracións probadas na fase de aliñamento.	72
6.4	Valores das variables empregadas nas distintas configuracións das probas na fase de acercamento, agrupadas por subfase.	74
6.5	Resultados comparativos das diferentes configuracións probadas na fase de acercamento.	74
6.6	Resultados das probas realizadas a diferentes distancias iniciais.	75
6.7	Resultados das probas realizadas con diferentes obxectos de interese (a unha mesma distancia inicial de 1.20 metros).	76

Índice de códigos

5.1	Instruccións para cargar o modelo YOLO de <i>ultralytics</i>	46
5.2	Traducción de ROS2 a OpenCV	47
5.3	Execución da inferencia do modelo YOLO sobre unha imaxe de OpenCV	47
5.4	Extracción da rexión de interese e conversión a HSV mediante OpenCV	47
5.5	Cálculo da mediana dos canais HSV para estimar a cor do obxecto	48
5.6	Definición dos límites da rexión de interese	48
5.7	Asociación temporal aos datos da nube de puntos	48
5.8	Eliminación dos puntos antigos en función do tempo de validez	49
5.9	Transformación dunha nube de puntos ao marco de referencia desexado me- diante	49
5.10	Filtrado espacial frontal sobre a nube de puntos	49
5.11	Interpolación lineal para determinar a altura mínima aceptable segundo a dis- tancia ao robot	50
5.12	Aplicación do filtro de altura dinámica punto a punto na nube	50
5.13	Eliminación de puntos afastados mediante filtrado por mediana da distancia .	50
5.14	Cálculo da posición media dos puntos filtrados como representación do obxecto	51
5.15	Proxección dos puntos 3D sobre o plano da imaxe cun modelo de cámara <i>fisheye</i>	51
5.16	Filtrado dos puntos proxectados que caen dentro da imaxe	51
5.17	Identificación dos puntos que se atopan dentro dunha <i>bounding box</i> detectada	52
5.18	Cálculo da mediana dos puntos asociados a unha detección	52
5.19	Filtrado dos puntos máis afastados respecto á mediana para obter unha medida máis robusta	52
5.20	Cálculo do centróide dos puntos filtrados como estimación final da posición 3D	53
5.21	Cálculo do desprazamento horizontal do obxecto de pateo respecto ao centro da imaxe	53
5.22	Rotación proporcional ao desprazamento horizontal	53
5.23	Cálculo da proporción da bounding box do obxecto de pateo	54
5.24	Desprazamentos frontal segundo a distancia ao obxecto	54

5.25	Cálculo do desprazamento lateral entre obxectos de interese	54
5.26	Movemento lateral correctivo segundo desprazamento	55
5.27	Actualización do tempo do último movemento	55
5.28	Condicións para executar un novo tipo de movemento	55
5.29	Inicialización dos temporizadores de movemento lateral e frontal	56
5.30	Control do tempo de espera para movemento lateral	56
5.31	Control do tempo de espera para movemento frontal	56
5.32	Desprazamento cara atrás por proximidade excesiva	57
5.33	Corrección lateral por desviación excesiva (movemento lateral longo)	57
5.34	Cálculo do erro angular a partir da orientación do robot e as posicións dos obxectos de interese	58
5.35	Rotación proporcional ao erro angular detectado	58
5.36	Avance proporcional á distancia frontal ao obxecto	58
5.37	Lectura da distancia frontal ao obxecto	59
5.38	Avance controlado segundo distancia curta ao obxecto	59
5.39	Cálculo do erro lateral segundo a posición do obxecto	59
5.40	Desprazamento lateral proporcional ao erro detectado	60
5.41	Acción para realizar o pateo do obxecto de interés	60
C.1	Execución contedor Docker	85

Introdución

O PRESENTE proxecto insírese no ámbito da robótica móbil autónoma, unha das áreas de maior proxección dentro da enxeñaría moderna. En particular, inspírase nos retos propostos por competicións internacionais como a RoboCup, nas que se fomenta o desenvolvemento de sistemas robóticos capaces de percibir o seu entorno, tomar decisións e actuar de forma autónoma en contextos dinámicos e non estruturados. Estas competicións representan un banco de probas realista e estimulante para a aplicación de técnicas avanzadas de intelixencia artificial, visión por computador, planificación de movementos e control físico.

1.1 Contexto e motivación

A RoboCup, fundada en 1997, é unha competición internacional que ten como obxectivo promover a investigación en robótica e intelixencia artificial mediante desafíos atractivos e complexos [4, 5]. A idea de robots xogando ao fútbol foi proposta por primeira vez polo profesor Alan Mackworth en 1992, e posteriormente desenvolvida por un grupo de investigadores xaponeses que organizaron un taller sobre grandes desafíos en intelixencia artificial [6]. Estes esforzos culminaron na creación da RoboCup, cuxo obxectivo último é que, para mediados do século XXI, un equipo de robots humanoides totalmente autónomos sexa capaz de gañar un partido de fútbol contra os campións humanos da FIFA, cumprindo coas regras oficiais do xogo [7, 8].

Este obxectivo encapsula múltiples desafíos tecnolóxicos: visión artificial, percepción multimodal, localización e mapeado, planificación estratéxica, coordinación en tempo real, aprendizaxe continua e control motor de precisión [9]. Así, a RoboCup converteuse nun referente internacional que impulsa o desenvolvemento de técnicas aplicables tamén fóra do ámbito deportivo, como a robótica de servizos, a exploración autónoma ou a asistencia en tarefas complexas [8].

Inspirado por este contexto, o presente proxecto céntrase nun reto máis acoutado pero

representativo: dotar ao robot cuadrúpede Go2, desenvolvido pola empresa Unitree, da capacidade de percibir o seu entorno, detectar unha pelota, aproximarse a ela e, finalmente, executar unha acción motora de chute cara a un obxectivo concreto.

Este comportamento, aparentemente sinxelo, require a integración de múltiples compoñentes software e hardware: desde os módulos de percepción baseados en visión artificial e sensores [Light Detection and Ranging](#), ata a planificación de movementos e o control motor de baixo nivel. Ademais, implica abordar problemas reais como a fusión de datos sensoriais, a estimación de posición en diferentes marcos de referencia, a xestión do tempo en tarefas secuenciais e a robustez fronte a incertezas no entorno.

En definitiva, o proxecto motívase tanto polo desafío técnico que supón integrar todos estes elementos nun sistema funcional, como polo valor formativo que achega traballar nun escenario inspirado nos grandes retos da robótica moderna, seguindo a filosofía da RoboCup: avanzar cara a robots máis autónomos, cooperativos e intelixentes [4, 5].

1.2 Obxectivos do proxecto

O obxectivo xeral deste proxecto é deseñar, implementar e validar un sistema de comportamento autónomo para o robot Go2 que lle permita detectar, aproximarse e patear unha pelota cara a un obxectivo concreto. Para acadar este obxectivo xeral, defínense os seguintes obxectivos específicos:

1. Crear un entorno de simulación mediante Docker con Ubuntu, [Robot Operating System 2](#) e Gazebo.
2. Integrar un sistema de intelixencia artificial que permita detectar e localizar unha pelota, unha portería e persoas ou obxectivos similares no entorno próximo.
3. Utilizar as funcións de alto nivel dispoñibles no Go2 para definir traxectorias e desprazar o robot ata a pelota, de modo que esta quede entre o robot e o obxectivo ao que se desexa chutar.
4. Validar este comportamento no robot real, garantindo que é capaz de detectar o balón e posicionarse correctamente.
5. Usar ou adaptar un comportamento de alto nivel (por exemplo, avanzar) para empurrar a pelota mediante o propio desprazamento do robot.
6. Desenvolver un comportamento de chute mediante o control dunha das patas, empregando un movemento de retracción e extensión implementado a nivel baixo.

7. Validar o comportamento completo no robot real, de forma que sexa quen de detectar e chutar un balón cara a un obxectivo determinado nun entorno real.

1.3 Proposta

A proposta deste proxecto consiste no desenvolvemento dun sistema reactivo para o robot cuadrúpede Unitree Go2 que lle permita detectar unha pelota, aproximarse a ela e chutar cara a un obxectivo definido.

O enfoque adoptado é fundamentalmente **reactivo**: as decisións do robot baséanse en tempo real nas percepcións da cámara **Red, Green, Blue** e do sensor **LiDAR** integrados. Estes dispositivos permiten detectar e localizar os obxectos de interese e, en función da súa posición relativa, desencadear os movementos correspondentes.

Este enfoque contrasta con outros posibles, como o emprego de **aprendizaxe automática**, e en particular **aprendizaxe por reforzo**, onde o robot aprende estratexias a través de interaccións repetidas co entorno. Aínda que estas técnicas teñen demostrado gran potencial, presentan tres limitacións principais neste proxecto:

- A ausencia de coñecementos previos sobre as ferramentas empregadas e o hardware, o que supón unha curva de aprendizaxe inicial máis empinada.
- A falta de recursos computacionais axeitados para executar procesos de adestramento, xa que os algoritmos de aprendizaxe por reforzo requiren unha capacidade de cómputo elevada e prolongadas sesións de execución.
- A dificultade de xerar un comportamento tan complexo como o que require este proxecto mediante unicamente técnicas de aprendizaxe por reforzo.

En consecuencia, optouse por unha estratexia reactiva, máis sinxela de implementar, depurar e validar, e suficientemente eficaz para o obxectivo proposto. O sistema baséase, polo tanto, nun conxunto de regras de control que procesan a información sensorial e xeran movementos de forma directa e determinista, asegurando un funcionamento estable e previsible en tempo real.

1.4 Traballo relacionado

Existen múltiples traballos previos que abordan problemas semellantes, ben no ámbito da manipulación de balóns por robots cuadrúpedes, ben na integración de percepción e control para tarefas dinámicas.

En [10], preséntase un sistema no que un robot cuadrúpede utiliza visión por computador e planificación de movementos para interactuar cun balón en escenarios estruturados.

O obxectivo céntrase en demostrar a capacidade de coordinación entre percepción visual e locomoción.

Neste outro traballo [11], propónse un modelo de control avanzado para cuadrúpedes aplicado a tarefas de manipulación de balón. Este enfoque destaca polo uso de estratexias híbridas que combinan percepción sensorial e control dinámico, aproximándose ao espírito da RoboCup.

Outro exemplo relevante é [12], propón un sistema para robots de fútbol con patas, baseado nun planificador de movemento optimizado que predí a posición da pelota e axusta o ciclo de marcha en tempo real. Isto permite chutar sen deter o avance, asignando roles específicos ás patas. As probas con penaltis e drible dinámico amosan que os robots con patas poden acadar mobilidade e manipulación dinámicas.

Por outra banda, no traballo de van Bree [13], desenvólvese un sistema de manipulación de balón baseado en aprendizaxe por imitación e reforzo para unha plataforma cuadrúpede. Este enfoque contrasta co do presente proxecto: en lugar dunha arquitectura reactiva determinista, confíase nun proceso de aprendizaxe automático. Se ben ofrece potencial para mellorar a adaptación do robot, presenta os inconvenientes xa mencionados (complexidade, opacidade e tempo de adestramento).

En resumo, o presente proxecto sitúase na intersección destes traballos, apostando por unha aproximación práctica, reactiva e centrada na integración eficiente dos sensores e controis dispoñibles no Unitree Go2.

1.5 Estrutura da memoria

Esta memoria estrutúrase en varios capítulos nos que se recolle o traballo desenvolvido e os coñecementos necesarios para a súa comprensión. A continuación preséntanse estes capítulos cunha breve descrición:

1. **Introdución** preséntase o contexto do proxecto, a súa motivación, os obxectivos formulados, a proposta inicial, así como unha revisión do traballo relacionado.
2. **Fundamentos tecnolóxicos**: expónse a base teórica e práctica necesaria, incluíndo o hardware e software empregados, a arquitectura de ROS 2 e ferramentas complementarias.
3. **Metodoloxía e planificación**: descríbese a metodoloxía seguida, a división en fases de desenvolvemento, a planificación temporal e a análise de recursos e custos.
4. **Deseño**: detállase a arquitectura global do sistema, o modelo de datos e os módulos de percepción e control propostos.

5. **Detalles de implementación:** descríbese a implementación concreta dos distintos módulos, incluíndo fragmentos de código e explicacións técnicas.
6. **Probas:** Recóllense os experimentos realizados, as limitacións atopadas, a validación dos módulos por separado e as probas finais do sistema completo.
7. **Conclusións:** Preséntanse as conclusións máis relevantes e posibles liñas de mellora ou extensión do proxecto.

Fundamentos tecnolóxicos

ESTE capítulo describe os recursos materiais e tecnolóxicos necesarios para o desenvolvemento do proxecto, divididos en dúas categorías principais: hardware e software.

2.1 Hardware

Nesta sección descríbense o hardware empregado no proxecto: o robot Unitree Go2, a cámara RGB e o LiDAR 4D.

2.1.1 Unitree Go2

O Unitree Go2 é un robot cuadrúpede de última xeración desenvolvido pola empresa chinesa Unitree Robotics. Trátase dun sistema robótico avanzado deseñado para tarefas de locomoción autónoma, percepción e interacción con contornas dinámicas, tanto en interiores como exteriores. A súa arquitectura equilibrada entre hardware de altas prestacións e software accesible ao desenvolvemento fan del unha plataforma ideal para investigacións aplicadas no eido da robótica móbil e a intelixencia artificial.

Este robot constitúe o pilar fundamental sobre o que se sustenta todo o proxecto, xa que todas as funcionalidades desenvolvidas —percepción, navegación e golpeo do balón— están deseñadas para executarse directamente sobre el, ben sexa no entorno simulado, ben no robot físico. O seu uso permite experimentar con comportamentos complexos nun sistema realista, reproducindo retos similares aos que se presentan en competicións como a RoboCup.

A versión empregada neste proxecto é a Unitree Go2 EDU, que incorpora unha [Central Processing Unit](#) de 8 núcleos con 16 GB de RAM, ofrecendo un alto rendemento para o procesamento de datos e execución de algoritmos en tempo real. Conta con sensores integrados como cámara RGB, LiDAR 4D, IMU e sensores de forza nas extremidades, que permiten ao robot percibir o entorno con precisión [1]. Ademais, é compatible con Ubuntu e ROS 2, facilitando a integración cos paquetes desenvolvidos durante o proxecto. Ten unha velocidade

Especificación	Valor
<i>Modelo</i>	Unitree Go2 EDU
<i>Velocidade máx.</i>	Ata 3.5 m/s
<i>Autonomía</i>	Ata 2 horas
<i>Sensores integrados</i>	Cámara RGB, LiDAR 4D, IMU, sensores de forza
<i>Procesador principal</i>	CPU de 8 núcleos + 16 GB RAM (Versión EDU)
<i>Sistema operativo</i>	Ubuntu + ROS 2
<i>Peso</i>	Aprox. 15 kg

Táboa 2.1: Especificacións principais do robot cuadrúpede Unitree Go2 [1]



Figura 2.1: Robot cuadrúpede Unitree Go2 (versión EDU) con cámara montada.

máxima de ata 3.5 m/s e unha autonomía de aproximadamente dúas horas, o que o fai axeitado para probas prolongadas en campo aberto. Estas características, que poden ser vistas na táboa 2.1, fan do Go2 unha plataforma robusta e versátil para experimentos en locomoción, percepción e interacción co entorno. Ademais, na figura 2.1 móstrase unha imaxe ilustrativa do robot, que permite facerse unha idea máis clara do seu deseño e configuración estrutural.

2.1.2 Cámara RGB

O robot Unitree Go2 inclúe unha cámara RGB integrada que será utilizada como sensor principal para a percepción visual do entorno. Esta cámara permite capturar imaxes en tempo real cunha resolución de ata 1280x720 píxeles e unha frecuencia de 30 imaxes por segundo [1], o que resulta fundamental para a detección de obxectos como a pelota, a portería ou persoas

Especificación	Valor
<i>Tipo</i>	Cámara RGB integrada
<i>Resolución</i>	Ata 1280x720 píxeles
<i>Frecuencia de captura</i>	30 fps (imaxes por segundo)
<i>Campo de visión (Field of View)</i>	Gran angular
<i>Integración</i>	Acceso nativo desde ROS 2

Táboa 2.2: Especificacións da cámara RGB integrada no Unitree Go2 [1]

no campo de xogo.

A cámara RGB será empregada xunto con modelos de visión artificial (como *You Only Look Once*) para identificar e localizar obxectos relevantes no entorno do robot, o que permitirá ao sistema tomar decisións sobre navegación e comportamento. A súa integración nativa no robot garante unha sincronización óptima cos demais sistemas, así como unha transmisión eficiente dos datos ao sistema de procesamento principal. As especificacións deste sensor poden consultarse na táboa 2.2, que resume os principais parámetros técnicos da cámara RGB.

2.1.3 LiDAR 4D

O Unitree Go2 tamén incorpora un sensor LiDAR 4D (modelo L1), que será empregado para a percepción tridimensional do entorno. Este sensor proporciona datos precisos sobre a distancia e disposición de obxectos ao redor do robot usando un escaneo láser en tempo real.

En esencia, un LiDAR (Light Detection and Ranging) funciona emitindo pulsos de luz láser, normalmente no espectro infravermello, en múltiples direccións. Estes pulsos reflíctense nos obxectos do entorno e son recollidos novamente polo sensor. Medindo o tempo que tarda cada pulso en volver ao emisor (tempo de voo ou “time of flight”), o sistema calcula a distancia exacta ao punto de reflexión [14]. Ao repetir este proceso de forma continua mentres o sensor rota ou cambia de ángulo, xérase unha nube de puntos tridimensional que representa con gran precisión a xeometría do entorno.

No caso do LiDAR L1 do Go2, esta capacidade esténdese a un campo de visión horizontal de 360° e vertical de 90° cun alcance de ata 40 metros [2], permitindo unha percepción volumétrica do espazo. As especificacións técnicas deste sensor poden atoparse resumidas na táboa 2.3, que recolle os datos máis relevantes do modelo empregado.

A información obtida do LiDAR será esencial para tarefas de navegación, localización e evitación de obstáculos. Ademais, permitirá combinar a percepción visual da cámara RGB cunha percepción espacial máis robusta e fiable, mellorando a capacidade do robot para moverse de forma autónoma e segura no entorno de xogo.

Especificación	Valor
<i>Modelo</i>	Unitree LiDAR L1
<i>Dimensións</i>	77 × 47 × 44 mm
<i>Peso</i>	200 g
<i>FOV horizontal</i>	360°
<i>FOV vertical</i>	90°
<i>Alcance</i>	Ata 40 metros
<i>Velocidade de escaneo</i>	20 Hz
<i>Datos por segundo</i>	Máis de 600.000 puntos/seg

Táboa 2.3: Especificacións técnicas do **LiDAR L1 4D** incluído no Go2 [2]

Especificación	Valor
<i>Modelo</i>	Intel RealSense D435i
<i>Resolución RGB</i>	Ata 1920x1080 píxeles
<i>Frecuencia de captura RGB</i>	30 fps
<i>Resolución profundidade</i>	Ata 1280x720 píxeles
<i>Frecuencia de captura profundidade</i>	90 fps
<i>Rango de profundidade</i>	de 0.105 ata 10 metros

Táboa 2.4: Especificacións técnicas da cámara Intel RealSense D435i [3]

2.1.4 Cámara RGB-D (Intel RealSense D435i)

O Unitree Go2 permite a integración dunha cámara RGB-D adicional, como a Intel RealSense D435i, que ofrece información tanto de cores como de profundidade. Esta cámara combina imaxes **RGB** con datos de distancia, permitindo ao robot percibir a contorna de forma tridimensional e mellorar a súa navegación e detección de obxectos en escenarios complexos.

A cámara D435i capta imaxes RGB cunha resolución de ata 1920x1080 píxeles a 30 fps, mentres que os datos de profundidade teñen unha resolución máxima de 1280x720 píxeles a 90 fps, cun alcance efectivo de 0.105 a 10 metros [3]. As especificación da cámara poden verse na táboa 2.4.

A información de profundidade proporcionada por esta cámara permite complementar os datos da cámara RGB e do **LiDAR**, especialmente en tarefas de aproximación precisa ao obxecto de pateo e para a localización de obstáculos. A súa integración pode realizarse mediante ROS2, o que facilita a sincronización cos demais sensores do robot e a súa utilización en algoritmos de percepción e control avanzados.

2.2 Software

Nesta sección explícanse as principais ferramentas software utilizadas no desenvolvemento do proxecto.

2.2.1 Docker

Docker é unha plataforma de virtualización lixeira baseada en contedores que permite empacar, distribuír e executar aplicacións de forma illada, reproducíble e consistente, independentemente do sistema operativo anfitrión. A súa inclusión neste proxecto débese a que facilita a reproducibilidade do entorno de desenvolvemento e simulación, reduce os problemas de configuración e asegura que o software funcione da mesma maneira en distintos equipos [15].

A principal vantaxe de Docker respecto a outras solucións como as máquinas virtuais tradicionais é que non require a emulación dun sistema operativo completo. En lugar diso, Docker fai uso do kernel do sistema operativo anfitrión, e só encapsula as bibliotecas, dependencias e configuracións necesarias para a execución da aplicación. Isto fai que os contedores sexan moito máis lixeiros, rápidos de iniciar e sinxelos de distribuír.

2.2.2 Python

Python é unha linguaxe de programación interpretada, de alto nivel e multiparadigma, recoñecida pola súa sinxeleza sintáctica e a súa ampla comunidade de desenvolvedores. A súa versatilidade convértea nunha ferramenta moi utilizada tanto en investigación como en desenvolvemento industrial, especialmente no ámbito da intelixencia artificial, análise de datos e robótica.

No contexto de [ROS 2](#), Python conta cunha interface oficial chamada **rclpy**, que permite crear nodos e interactuar co ecosistema de comunicación do sistema de forma sinxela e efectiva. Esta linguaxe facilita o desenvolvemento rápido de prototipos, a manipulación de estruturas de datos complexas e a interacción con sensores e actuadores.

Durante o desenvolvemento do proxecto, Python foi empregado para a creación de nodos de percepción, control e coordinación do comportamento robótico, así como para a implementación de servidores de acción e scripts de lanzamento.

2.2.3 Visual Studio Code

Visual Studio Code (VS Code) é un editor de código fonte lixeiro e multiplataforma desenvolvido por Microsoft. Ofrece soporte para múltiples linguaxes de programación e conta cun amplo ecosistema de extensións, incluíndo ferramentas específicas para Python, [ROS 2](#) e Git.

No proxecto foi empregado como principal contorno de desenvolvemento, facilitando a edición, depuración e organización do código mediante a súa interface intuitiva e funcionalidades integradas.

2.2.4 Git e GitHub

Git é un sistema de control de versións distribuído amplamente utilizado no desenvolvemento de software. Permite levar un rexistro de todos os cambios realizados no código, facilitar o traballo colaborativo e xestionar distintas versións dun proxecto. GitHub, pola súa parte, é unha plataforma baseada na web que permite aloxar repositorios Git e colaboradores en liña. Durante o desenvolvemento do proxecto, empregouse Git para controlar a evolución do código e GitHub como repositorio central para o almacenamento, seguimento de incidencias e copia de seguridade do traballo.

2.2.5 LaTeX e Overleaf

LaTeX é un sistema de composición de textos orientado á creación de documentos técnicos e científicos de alta calidade tipográfica. É especialmente axeitado para a redacción de memorias, artigos e documentación con fórmulas matemáticas. Overleaf é unha plataforma colaborativa en liña para a edición de documentos LaTeX. Neste proxecto, utilizouse LaTeX a través de Overleaf para redactar a memoria, garantindo un formato profesional e unha xestión clara das referencias bibliográficas, figuras e estrutura do documento.

2.3 ROS 2

[Robot Operating System 2](#) é un conxunto de bibliotecas e ferramentas para o desenvolvemento de software para robótica. Ofrece características como:

- **Comunicación en tempo real** mediante DDS (Data Distribution Service) [16].
- **Compatibilidade multiplataforma**, incluíndo Linux, Windows e macOS [17].
- **Mecanismos de seguridade** como autenticación e cifrado [18].

Estas funcionalidades fan de [ROS 2](#) unha plataforma robusta e escalable para o desenvolvemento de sistemas robóticos complexos [18].

[ROS 2](#) é un framework flexible e modular deseñado para facilitar o desenvolvemento de aplicacións robóticas distribuídas, robustas e escalables [17]. Do mesmo xeito que o seu predecesor, [ROS 1](#), consiste nun conxunto de ferramentas, bibliotecas e convencións que permiten programar, simular e executar sistemas robóticos complexos, pero con importantes melloras

estruturais orientadas á fiabilidade, seguridade e integración en contextos industriais e tempo real [19].

O obxectivo de ROS 2 é ofrecer unha infraestrutura aberta, descentralizada e altamente reutilizable, que facilite a colaboración entre equipos de desenvolvemento de todo o mundo. Isto faise mediante a compartición de paquetes ou *stacks* que engloban funcionalidades diversas como percepción, navegación, planificación de movementos, control, visión por computador, simulación e aprendizaxe automática [20].

2.3.1 Evolución respecto a ROS 1

ROS 2 naceu como resposta a varias limitacións de ROS 1, especialmente en ámbitos como:

- A compatibilidade con sistemas en tempo real.
- A comunicación entre procesos en distintos sistemas operativos (Linux, Windows, macOS).
- A seguridade e autenticación de mensaxes.
- A robustez en redes distribuídas con latencias ou perdas.

Para iso, ROS 2 está construído sobre DDS (Data Distribution Service), un middleware de comunicacións estandarizado, que permite implementar sistemas asincrónicos, fiables e escalables con requisitos industriais [16, 19].

2.3.2 Arquitectura de ROS 2

ROS 2 presenta unha arquitectura modular e distribuída baseada nun grafo de nodos que se comunican entre si mediante un sistema flexible de mensaxes. Esta arquitectura está deseñada para garantir escalabilidade, robustez e compatibilidade con sistemas de tempo real, sendo especialmente axeitada para aplicacións robóticas complexas e distribuídas. A súa infraestrutura facilita o desenvolvemento de sistemas en que múltiples procesos cooperan para realizar tarefas en paralelo, mantendo unha alta capacidade de integración e interoperabilidade [17, 18].

2.3.3 Nodos

Os nodos son as unidades básicas de computación en ROS 2. Cada nodo executa unha función concreta, como a lectura dun sensor, o control de motores ou o procesamento de datos de percepción. Esta separación funcional favorece a modularidade, facilitando a reutilización e o mantemento do código. Os nodos comunícanse entre si a través de tópicos, servizos ou accións, sen necesidade de coñecer a implementación interna doutros nodos, o que permite unha arquitectura desacoplada.

2.3.4 Executor e rclcpp/rclpy

A xestión da execución dos nodos está a cargo dun compoñente denominado executor. Este encárgase de planificar e despachar as operacións segundo eventos asincrónicos, como a recepción de mensaxes ou a execución de temporizadores. ROS 2 proporciona *bindings* para distintas linguaxes de programación, sendo C++ (rclcpp) e Python (rclpy) as máis utilizadas. Isto permite escoller a linguaxe máis axeitada segundo as necesidades de rendemento ou facilidade de desenvolvemento.

2.3.5 Tópicos

Os tópicos son os canles principais de comunicación en ROS 2, baseados no paradigma de publicación e subscrición. Un nodo pode publicar mensaxes nun topic determinado, mentres outros nodos se subscriben para recibilas. Esta comunicación é asincrónica e unidireccional, axeitada para transmitir datos como posicións, imaxes ou lecturas de sensores de forma continua. Os tópicos permiten múltiples publicadores e subscritores, facendo posible a implementación de sistemas complexos e altamente conectados.

2.3.6 Mensaxes

As mensaxes son estruturas de datos que encapsulan a información transmitida entre nodos. Están definidas en ficheiros con extensión .msg e poden incluír tipos primitivos (inteiros, floats, booleanos), así como estruturas compostas (poses, nube de puntos, imaxes). A definición clara das mensaxes garante a interoperabilidade entre distintos nodos e facilita a validación e depuración do sistema.

2.3.7 Servizos

Os servizos permiten unha forma de comunicación síncrona entre dous nodos, baseada no modelo petición-resposta. A diferenza dos tópicos, que son unidireccionais e asincrónicos, os servizos establecen un diálogo directo onde un nodo cliente realiza unha petición e agarda pola resposta do nodo servidor. Esta forma de comunicación é útil para operacións puntuais como solicitar datos específicos ou executar comandos únicos, como reiniciar un sensor ou mover un brazo a unha posición concreta.

2.3.8 Accións

As accións amplían o modelo de servizos para soportar tarefas de longa duración. Unha acción permite iniciar unha tarefa (por exemplo, navegar ata unha posición), seguir o seu

progreso, recibir actualizacións periódicas, e incluso cancelala antes de que remate. Esta funcionalidade é esencial en contextos robóticos nos que a resposta debe adaptarse a cambios no entorno ou interrupcións inesperadas. As accións estrutúranse mediante tres tipos de mensaxes: goal (obxectivo), feedback (progreso) e result (resultado).

2.3.9 Launch files

Para facilitar o despregue e configuración de sistemas complexos, ROS 2 utiliza scripts de lanzamento, chamados *launch files*. Estes ficheiros están escritos en Python e permiten iniciar múltiples nodos, establecer parámetros, definir dependencias e configurar o entorno de execución de maneira estruturada. Os launch files resultan especialmente útiles para probar sistemas en simulación ou para despregar configuracións específicas en robots reais.

2.4 CycloneDDS

CycloneDDS é unha implementación open source do estándar DDS (Data Distribution Service) utilizada por defecto en ROS 2 como middleware de comunicación [21, 22]. DDS é un protocolo de comunicación orientado a datos que permite a transmisión eficiente e escalable de información entre procesos distribuídos, sen necesidade de establecer conexións directas entre eles [23, 24]. Isto permite que os nodos de ROS 2 poidan publicar e subscribirse a tópicos, así como utilizar servizos e accións, de forma transparente, aínda que se atopen en distintos dispositivos ou redes [22, 23].

CycloneDDS destaca pola súa baixa latencia, alto rendemento, soporte para comunicación en tempo real e boa interoperabilidade en contornos heteroxéneos [22, 24]. Ademais, ofrece mecanismos de descubrimento automático de nodos na rede, o que facilita a configuración e escalabilidade de sistemas complexos sen intervención manual [21].

No contexto deste proxecto, CycloneDDS xoga un papel esencial como infraestrutura subxacente para o intercambio de mensaxes entre os distintos nodos ROS 2 [22, 23]. Grazas a el, procesos que corren no ordenador de desenvolvemento e no robot Unitree Go2 poden comunicarse de forma eficiente, garantindo que as deteccións, comandos de movemento e sinais de control se transmitan correctamente [24]. A correcta configuración de CycloneDDS é fundamental para establecer a conectividade entre dispositivos que están en redes diferentes ou teñen restricións de acceso, como ocorre co Go2 [21].

2.5 WebRTC

Web Real-Time Communication é un conxunto de protocolos e APIs deseñados para permitir comunicacións en tempo real (voz, vídeo e datos) entre navegadores e dispositivos sen

necesidade de servidores intermedios [25]. Está optimizado para conexións peer-to-peer, utilizando técnicas como NAT traversal, ICE, STUN e TURN para establecer comunicacións seguras mesmo en contornos de rede complexos [26].

No ámbito da robótica, [WebRTC](#) resulta útil para establecer conexións en tempo real con robots que operan en redes externas ou remotas, facilitando a transmisión de datos de control, vídeo ou estado cunha latencia reducida [27]. A súa arquitectura descentralizada e capacidades de cifrado fan que sexa adecuada para escenarios que requiren supervisión ou interacción directa [25, 26].

No caso do robot Unitree Go2, [WebRTC](#) ofrece unha vía viable para establecer conexións en tempo real co robot desde un sistema remoto [27]. Isto permite tanto a transmisión de vídeo como a comunicación de datos, facilitando tarefas como o control e a monitorización remota de maneira eficiente e segura [25, 26].

2.6 Simuladores

A simulación é unha ferramenta esencial no desenvolvemento de sistemas robóticos, pois permite probar algoritmos e comportamentos sen necesidade de empregar o robot físico. No caso do Unitree Go2, existen distintas opcións que permiten replicar o seu funcionamento nun contorno virtual compatible con [ROS 2](#). Nesta sección veremos os posibles simuladores dispoñibles para tal fin, destacando as súas principais características.

2.6.1 Gazebo Classic

Gazebo Classic é a ferramenta principal de simulación empregada neste proxecto. Trátase dun simulador [Tres dimensións](#) de dinámica física de código aberto que permite construír, probar e validar sistemas robóticos en contornos virtuais complexos, tanto de interior como de exterior, cun elevado grao de realismo [28]. Esta ferramenta leva anos sendo a estándar na comunidade [ROS](#), especialmente en combinación con [ROS 1](#) e, posteriormente, con [ROS 2](#) a través de plugins específicos como `gazebo_ros_pkgs`.

Gazebo Classic permite modelar robots empregando o formato URDF (Unified Robot Description Format) ou SDF (Simulation Description Format), nos cales se definen as súas articulacións (*joints*), partes físicas (*links*), colisións, sensores e propiedades inerciais. Grazas ao seu motor físico integrado (por defecto ODE), é posible simular a dinámica do movemento dos robots, interaccións coa contorna e contactos entre corpos con friccións e forzas realistas [29].

Unha das características máis relevantes de Gazebo Classic é o seu soporte nativo para sensores virtuais como cámaras [RGB](#), cámaras de profundidade, sensores [LiDAR](#), GPS e

IMUs. Estes sensores permiten que os algoritmos de percepción se poidan probar en contornos realistas antes de realizar a súa implementación no robot físico [30].

Existen dúas formas principais de traballar con Gazebo Classic:

- A través do seu **editor de modelos**, accesible desde a interface gráfica, que permite construír escenarios virtuais e robots de forma visual.
- Modificando directamente os ficheiros URDF ou SDF, o que proporciona maior control e flexibilidade na configuración da simulación.

A integración con **ROS 2** faise mediante o uso de plugins específicos que permiten a comunicación entre nodos **ROS** e entidades dentro do mundo simulado. Isto posibilita lanzar algoritmos de navegación, visión ou control en tempo real, exactamente igual que se estivesen a funcionar sobre un robot físico.

En definitiva, Gazebo Classic permite obter un modelo virtual do robot (neste caso, o Unitree Go2) cuxa resposta no mundo simulado é unha aproximación precisa ao seu comportamento no mundo real, facilitando o desenvolvemento, proba e validación de algoritmos antes do seu despregue físico.

2.6.2 Isaac Gym/ Isaac Sim

Isaac Sim é un simulador robótico de nova xeración desenvolvido por NVIDIA, baseado no motor gráfico Omniverse. A súa principal característica distintiva é o uso de renderizado físico realista (PhysX) e a súa orientación a aplicacións de aprendizaxe automática, robótica avanzada e simulación masiva en GPU [31].

Ainda que non existe un modelo oficial do Unitree Go2 incluído por defecto, a plataforma permite importar modelos URDF personalizados e configurar sensores simulados, o que fai viable a simulación deste robot tras un proceso de adaptación [32]. A comunidade de desenvolvedores e investigadores comezou a utilizar Isaac Sim para estudar locomoción cuadrúpeda en robots Unitree, debido á súa capacidade para executar miles de simulacións en paralelo, o que resulta extremadamente útil para métodos baseados en deep reinforcement learning [33].

Isaac Sim destaca tamén pola súa compatibilidade con **ROS 2**, co que se poden probar algoritmos desenvolvidos no entorno real, especialmente aqueles que requiren simulacións de alta fidelidade e sincronización con visión por computador ou planificación de movementos [31, 33].

2.7 YOLO

No presente traballo empregouse o modelo **YOLO**, desenvolvido por Ultralytics, como base para a detección en tempo real de obxectos no entorno do robot. **YOLO** é unha familia

de modelos de visión artificial baseada na detección unificada de obxectos nunha soa pasada (*single-shot detection*), o que lle permite unha gran eficiencia e rapidez na predición das clases e localización dos obxectos nunha imaxe [34].

A principal vantaxe da arquitectura **YOLO** respecto a outros modelos clásicos como R-CNN ou SSD reside na súa capacidade para procesar imaxes completas nun só paso de inferencia, o que o converte nun modelo altamente optimizado para aplicacións en tempo real sobre plataformas con recursos limitados, como robots móbiles ou drones [35, 36, 37].

Algunhas características que destacan de **YOLO** son as seguintes:

- **Arquitectura baseada en PyTorch e exportable a ONNX, TensorRT e OpenVINO**, facilitando a súa integración en distintos dispositivos, desde ordenadores de alto rendemento ata sistemas embebidos.
- **Cabeceiras de predición redeseñadas** con mellor equilibrio entre velocidade e precisión.
- **Soporte nativo para segmentación, clasificación e detección de poses**, o que o converte nun modelo versátil para múltiples tarefas.
- **Entrenamento desde cero e transfer learning** con ferramentas integradas que permiten adestrar modelos personalizados en moi pouco tempo, empregando datasets propios adaptados ao dominio específico do proxecto.

Neste proxecto tiveronse en conta varias versións de **YOLO** buscando un bo equilibrio entre precisión e velocidade de inferencia, fundamental para a integración en tempo real co sistema **ROS 2** e co simulador Gazebo. A detección realízase sobre imaxes **RGB** procedentes dunha cámara simulada, e as **bounding box** xeradas son utilizados posteriormente para filtrar os puntos dunha **nube de puntos LiDAR** proxectada na imaxe [34].

A integración con **ROS 2** levouse a cabo mediante un nodo específico que publica as deteccións como mensaxes personalizadas, o que permite a súa subscripción directa por parte doutros nodos encargados do seguimento, navegación ou localización do obxecto detectado [36].

A robustez, velocidade e sinxeleza de uso de **YOLO** fan que sexa unha das opcións máis recomendadas na actualidade para tarefas de percepción en robótica móbil, visión por computador e sistemas intelixentes embebidos [38].

Metodoloxía e planificación do proxecto

NESTE capítulo descríbese a metodoloxía empregada ao longo do desenvolvemento do proxecto, así como a planificación das distintas fases que o compoñen. O obxectivo é proporcionar unha visión clara e estruturada do proceso seguido, dende a fase inicial de formación ata a validación final do sistema completo e rematando coa documentación. Esta organización permite comprender como se abordaron os distintos retos técnicos e como se estruturou o traballo para acadar os obxectivos propostos.

3.1 Metodoloxía de desenvolvemento

Dado o carácter exploratorio do proxecto e a continua necesidade de verificar o correcto funcionamento de cada compoñente, adoptouse unha metodoloxía de desenvolvemento de tipo áxil. Este carácter exploratorio ven dado polo feito de que se trataba da primeira vez que se traballaba cun conxunto de ferramentas e tecnoloxías como ROS 2, Gazebo Classic ou Docker. Este enfoque resulta especialmente axeitado para proxectos que se realizan por primeira vez, nos que os requisitos poden evolucionar ao longo do tempo e a aprendizaxe progresiva é unha parte integral do proceso [39, 40, 41].

A metodoloxía áxil facilita a adaptación continua fronte aos descubrimentos feitos durante a implementación, permite iterar rapidamente sobre funcionalidades concretas e favorece a entrega incremental de resultados parciais validados. Estas características fan que sexa unha opción moi útil en contornos como a robótica, onde os erros poden ter causas múltiples (hardware, software, simulación, percepción, etc.) e a capacidade de resposta rápida é crítica [42, 43].

Para organizar o traballo adoptouse unha versión simplificada da metodoloxía SCRUM [44, 45], estruturando o proxecto en fases curtas cun backlog de tarefas priorizado. As tarefas

foron elixidas segundo a súa dependencia técnica e o seu impacto funcional. En cada fase buscou acadar un obxectivo concreto e validable (por exemplo, a detección do balón ou a navegación cara a el), permitindo detectar erros de forma temperá e realizar axustes iterativos.

Algúns dos principios de SCRUM integrados no proxecto foron:

- A división do traballo en entregas incrementais e iterativas.
- A flexibilidade para adaptar o plan en función dos resultados acadados.
- A posibilidade de experimentar e aprender sobre a marcha sen comprometer a estabilidade do sistema.

Ademais, a simulación continua en Gazebo Classic permitiu probar sen risco as funcionalidades antes de levalas ao robot real, actuando como un ciclo de validación rápida que encaixa perfectamente coa filosofía áxil. A integración continua entre percepción, navegación e control fíxose posible grazas a esta metodoloxía, que favorece a entrega progresiva e a integración frecuente de compoñentes.

De cara á fase final do proxecto, realizáronse unha serie de adaptacións prácticas para optimizar o tempo dispoñible. Dado que as probas co robot real requiren acceso ao laboratorio do [Centro de Investigación en Tecnoloxías da Información e as Comunicaciones](#), o cal non está dispoñible durante as fins de semana, aproveitouse ese tempo para avanzar na documentación técnica e na redacción da memoria. Esta distribución permitiu manter a produtividade de forma sostida e asegurou un equilibrio eficiente entre o traballo práctico e o reflexivo.

3.2 Requisitos

Co obxectivo de definir claramente as expectativas e condicións que debe cumprir o sistema, nesta sección establécense os requisitos funcionais e non funcionais identificados.

3.2.1 Requisitos funcionais

Os requisitos funcionais refírense ás funcionalidades que o sistema debe ofrecer:

RF-1 O robot debe detectar un balón e un obxectivo mediante visión artificial.

RF-2 O sistema debe calcular a posición tridimensional dos obxectos combinando cámara e sensor [LiDAR](#).

RF-3 O robot debe desprazarse cara a unha posición de aproximación que lle permita aliñarse co balón e o obxectivo.

RF-4 O robot debe executar un comportamento de golpeo sobre o balón.

3.2.2 Requisitos non funcionais

Os requisitos non funcionais definen as características de calidade que debe cumprir o sistema:

RNF-1 **Robustez:** o sistema debe ser capaz de xestionar ruído nos sensores ou obxectos inesperados sen fallar.

RNF-2 **Escalabilidade:** a arquitectura debe permitir integrar novos sensores ou algoritmos de detección no futuro.

RNF-3 **Facilidade de uso:** o despregue do sistema debe realizarse mediante contedores que simplifiquen a reproducibilidade.

RNF-4 **Eficiencia:** o procesamento das deteccións debe realizarse en tempo real.

3.3 Actores

O único actor que interactúa directamente co sistema é o **usuario final**. Este é o encargado de iniciar os nodos desenvolvidos en [ROS 2](#), configurar os parámetros necesarios para a execución, como por exemplo a selección dos obxectos de interese que actuarán como obxecto de pateo e como obxecto destino, e supervisar o comportamento global do robot. Ademais, observa en tempo real os resultados xerados polo sistema, como as deteccións visuais, a integración cos datos do [LiDAR](#), as posicións estimadas ou os movementos executados polo robot. Tamén pode deter ou reiniciar o sistema cando sexa preciso, asegurando así o correcto desenvolvemento das probas e da validación final.

3.4 Fases do proxecto

Neste apartado descríbense as diferentes fases que conforman o desenvolvemento do proxecto, desde a formación inicial ata a documentación final. Cada fase representa un conxunto de tarefas e obxectivos concretos que foron abordados, proporcionando unha visión clara do proceso seguido para acadar os obxectivos propostos.

3.4.1 Fase 0: Formación

Esta fase representa o preludio do proxecto. Para desenvolver novo software en [ROS 2](#), utilizar ferramentas de simulación e controlar o robot Unitree Go2, é imprescindible adquirir unha base sólida sobre as tecnoloxías e metodoloxías que se van empregar ao longo do proxecto.

Tarefas:

- Investigar o funcionamento do sistema operativo [ROS 2](#).
- Aprender os conceptos básicos de programación en Python e C++ para o desenvolvemento de nodos.
- Familiarizarse co simulador Gazebo Classic e as súas funcionalidades.
- Estudar a arquitectura e API do robot Unitree Go2.
- Revisar o uso de contedores Docker para garantir un entorno reproducible.
- Investigar a integración de modelos de visión por computador.

3.4.2 Fase 1: Preparación do entorno

Nesta fase construíuse e configurouse un entorno de desenvolvemento que permitise traballar de forma eficiente tanto en simulación como sobre o robot real. O obxectivo foi garantir que todas as ferramentas necesarias funcionasen de forma integrada e estable.

Tarefas:

- Crear un contedor Docker personalizado baseado en [ROS 2 Humble](#) co simulador Gazebo Classic.
- Configurar a pila de navegación Nav2 e o SLAM Toolbox para mapeo e localización.
- Integrar soporte para sensores, como por exemplo o [LiDAR](#), no contedor.
- Configurar a comunicación [WebRTC](#) para conectar co robot Unitree Go2.
- Probar e validar a execución dos nodos básicos dentro do contedor e en simulación.

3.4.3 Fase 2: Desenvolvemento do módulo de percepción

Esta fase representou un dos piares fundamentais do proxecto, xa que tiña como obxectivo dotar ao robot dunha capacidade de percepción robusta e integrada. Para que o robot sexa quen de interactuar co entorno de forma autónoma é necesario dotalo dunha visión clara da súa contorna e da posición precisa do balón e do obxectivo ao que patealo, que é o obxecto clave da tarefa.

Tarefas:

- Integrar o modelo [YOLO](#) para detectar o balón e o obxectivo nas imaxes da cámara frontal.
- Probar a precisión e robustez do módulo en simulación e analizar os posibles erros.
- Optimizar o fluxo de datos para garantir un procesamento en tempo real.

3.4.4 Fase 3: Control e navegación

Nesta fase intégrase por primeira vez a información visual do modelo de detección co sensor **LiDAR** do robot para obter unha estimación precisa da posición do balón e do obxectivo (xa sexa unha portería, persoa ou punto de referencia). A combinación destes datos permite calcular unha posición intermedia á que debe desprazarse o robot para situarse estratexicamente no entorno. Este posicionamento ten como finalidade preparar o robot para unha posterior acción de empuxe ou golpeo.

Tarefas:

- Integración dos datos de detección visual co **LiDAR** para estimar a posición tridimensional do balón e do obxectivo.
- Calcular unha posición de aproximación do robot que permita aliñar o balón co obxectivo.
- Execución do desprazamento cara á posición calculada mediante movemento con control de alto nivel.
- Xiro do robot sobre si mesmo para quedar aliñado co balón tras alcanzar a posición de aproximación.
- Validación do comportamento completo no simulador para comprobar a lóxica e a precisión do movemento sen risco físico.
- Validación do comportamento sobre o robot real e realización dos axustes necesarios (parámetros de distancia, velocidade, orientación e tolerancias) para garantir un funcionamento preciso e robusto.

3.4.5 Fase 4: Comportamento do golpeo coa pata

Grazas á estimación precisa da posición do balón e do obxectivo obtida na fase anterior, nesta fase abórdase a implementación dun comportamento de golpeo. O obxectivo é que o robot golpee o obxecto de pateo conseguindo que este mesmo impacte co obxecto destino.

Tarefas:

- Deseño ou búsqueda dun comportamento de pateo empregando control de baixo nivel do Go2.
- Validación do comportamento en simulación co entorno virtual.
- Validación no robot real e realización dos axustes necesarios para asegurar un funcionamento robusto.

3.4.6 Fase 5: Validación do comportamento completo

Rematado o desenvolvemento dos distintos módulos do sistema, esta fase ten como obxectivo comprobar o funcionamento global do comportamento do robot en condicións reais. A integración final implica transferir todo o sistema desenvolvido ao robot físico e asegurar que a execución completa, dende a percepción ata a interacción co balón, se realiza de forma coherente, robusta e sen fallos.

A validación abordará tanto o rendemento do sistema en simulación como a súa reproducibilidade no robot real, analizando posibles desviacións, erros de execución ou limitacións non observadas en etapas previas. Esta fase é clave para realizar os últimos axustes finos e garantir que o sistema cumpre os requisitos definidos no contexto real de uso.

Tarefas:

- Integración de todos os módulos desenvolvidos nun único sistema funcional.
- Validación do comportamento completo en escenarios reais, reproducindo situacións similares ás definidas no entorno de xogo.
- Axuste fino de parámetros, adaptacións e melloras necesarias para garantir un funcionamento robusto e coherente no robot real.

3.4.7 Fase 6: Documentación

A fase de documentación é fundamental para garantir que todo o traballo realizado ao longo do proxecto quede recollido de forma clara, reproducible e comprensible para terceiros. Non só serve como soporte para futuras modificacións ou ampliacións do sistema, senón tamén como instrumento de avaliación do proceso seguido, das decisións técnicas tomadas e dos resultados acadados.

Durante esta fase recompilouse e estruturouse toda a información técnica, configuráronse anexos con exemplos de execución e preparouse a memoria final do proxecto, detallando cada unha das fases previas, así como os aspectos máis relevantes a nivel de implementación, probas e conclusións.

Tarefas:

- Recopilación de capturas, configuracións, scripts e rexistros xerados durante o desenvolvemento.
- Organización da documentación técnica asociada ao código, co fin de facilitar a súa comprensión e reutilización.
- Redacción da memoria do proxecto, describindo en detalle a metodoloxía, fases, solucións técnicas e resultados obtidos.

3.5 Seguimento do proxecto

Tras a planificación inicial descrita nas fases do proxecto, levouse a cabo un proceso de seguimento para contrastar a evolución real coa liña base. Esta análise permitiu identificar desviacións e dificultades atopadas ao longo do desenvolvemento.

Durante a execución do traballo xurdiron diversos problemas que condicionaron o progreso. En primeiro lugar, a conexión vía WiFi empregando *CycloneDDS* resultou ser máis complexa do esperado. Aínda que na planificación inicial se asumía que esta comunicación distribuída sería factible, na práctica non se conseguiu establecer unha ligazón estable e fiable, o que obrigou a replantexar o método de conexión decidindo finalmente usar unha conexión ethernet a través dun cable.

Por outra banda, o hardware empregado tamén presentou incidencias. Concretamente, nos robots *Unitree Go2* producíronse dous fallos relevantes: nun dos robots a cámara deixou de funcionar, e no outro produciuse un fallo xeral que imposibilitou o seu uso.

Na Figura 3.1 móstrase unha comparativa entre a planificación inicial (liña base) e o seguimento real do proxecto. Esta representación permite observar as desviacións respecto das tarefas programadas e as adaptacións realizadas debido ás dificultades mencionadas.

3.6 Recursos e custos

Neste apartado detállanse os recursos humanos e materiais empregados para levar a cabo o proxecto, así como unha estimación económica asociada ao seu desenvolvemento. O obxectivo é ofrecer unha visión global do esforzo invertido, tanto a nivel de dedicación persoal como de medios técnicos dispoñibles, e calcular o custo aproximado do proxecto en termos de horas de traballo e equipamento utilizado.

Recursos humanos

O proxecto contou coa participación de dúas persoas principais: o estudante e o tutor académico. O estudante desempeñou os roles de programador, analista e xefe de proxecto, mentres que o tutor actuou como consultor, proporcionando asesoramento e seguimento do desenvolvemento.

A distribución aproximada das horas dedicadas foi a seguinte:

- **Programador:** 220 horas dedicadas ao desenvolvemento do código e implementación de funcionalidades.
- **Analista:** 120 horas destinadas á documentación, análise de requisitos e estudo de [ROS 2](#), *Unitree Go2* e tecnoloxías asociadas.



Figura 3.1: Comparativa entre a planificación inicial e o seguimento do proxecto.

Rol	Horas	Coste €/h	Total €
Programador	220	30	6600
Analista	120	40	4800
Xefe de proxecto	50	50	2.500
Consultor/Tutor	20	50	1000
Total	410	-	14.900

Táboa 3.1: Coste estimado dos recursos humanos utilizados no proxecto.

- **Xefe de proxecto:** 50 horas dedicadas á planificación, coordinación e xestión xeral do proxecto.
- **Consultor:** 20 horas dedicadas a reunións de seguimento e asesoramento.

Establecendo un custo horario de 30 €/h para o programador, 40 €/h para o analista, 50 €/h para o xefe de proxecto e 50 €/h para o tutor, o custo estimado dos recursos humanos é de 16.360 €, esto pode verse desglosado na táboa 3.1.

Recursos materiais

Para a realización do proxecto empregáronse dous recursos materiais principais: un ordenador portátil persoal e o robot Unitree Go2 EDU cos seus accesorios.

O **ordenador portátil** foi a ferramenta principal para o desenvolvemento do software, execución das probas e redacción da documentación. As súas especificacións técnicas son as seguintes:

- Procesador: AMD® Ryzen 7 5800H with Radeon Graphics × 16
- Tarxeta gráfica: Gráficos RENOIR (Renoir, LLVM 15.0.7, DRM 3.57, kernel 6.8.0-65-generic)
- Memoria RAM: 16,0 GiB
- Sistema operativo: Ubuntu 22.04.5 LTS

O **robot Unitree Go2 EDU** foi cedido polo CITIC sen custo directo para o proxecto. O seu valor aproximado de adquisición é de 20.000 €.

Custos estimados

O custo total do proxecto considérase como a suma do custo dos recursos humanos e dos recursos materiais. No caso dos recursos materiais, non teñen un custo directo, dado que o

Concepto	Custo (€)
Recursos humanos	16.360
Recursos materiais	Sen custo directo
Valor total estimado do proxecto	16.360

Táboa 3.2: Resumo dos custos estimados do proxecto.

robot Unitree Go2 foi cedido polo [CITIC](#) e o portátil xa estaba amortizado. Aínda así, para efectos de referencia, o valor estimado do robot é de 20.000 € (Unitree Go2 versión EDU)

Capítulo 4

Deseño

NESTE capítulo descríbese a organización funcional do sistema desenvolvido para a execución da tarefa de patear un obxecto cara a un destino. A arquitectura proposta combina múltiples fontes sensoriais e un control dividido por fases, permitindo que o robot reaccione de forma eficiente ante o entorno inmediato.

4.1 Arquitectura do sistema

A presente sección ten como obxectivo describir a arquitectura global do sistema desenvolvido para a tarefa de patear un balón cara a un obxecto destino. Analízase como se integran os distintos módulos de percepción e control, cal é o fluxo de datos entre eles, e como se organiza o funcionamento por fases dentro dun sistema reactivo. Esta arquitectura pode verse na figura 4.1, representando os principais nodos de percepción e control, así como os tópicos que interconectan os distintos compoñentes do sistema.

Neste contexto, un **sistema reactivo** refírese a aquel cuxo comportamento se basea en responder directamente aos estímulos do entorno inmediato, sen realizar planificación a longo prazo. É dicir, cada acción executada polo robot está determinada polos datos sensoriais actuais (como a posición detectada da pelota ou a distancia frontal ao obxecto), sen construír un modelo global ou anticipar eventos futuros. Esta filosofía de deseño permite unha resposta rápida e robusta ante cambios dinámicos na escena, sendo especialmente útil en contornos non estruturados [46, 47, 48, 49, 50, 51].

A arquitectura estrutúrase arredor dun módulo global de percepción, encargado de proporcionar ao sistema de control a información sensorial necesaria para a toma de decisións. Tal e como se representa na figura 4.1, este bloque de percepción encapsula o conxunto de procesos encargados da análise visual, da integración cos datos **LiDAR**, da estimación de distancias e da localización do robot, entre outros. Aínda que nesta figura se presenta de forma unificada como un único compoñente funcional, na sección 4.3 realizarase un desglose de-

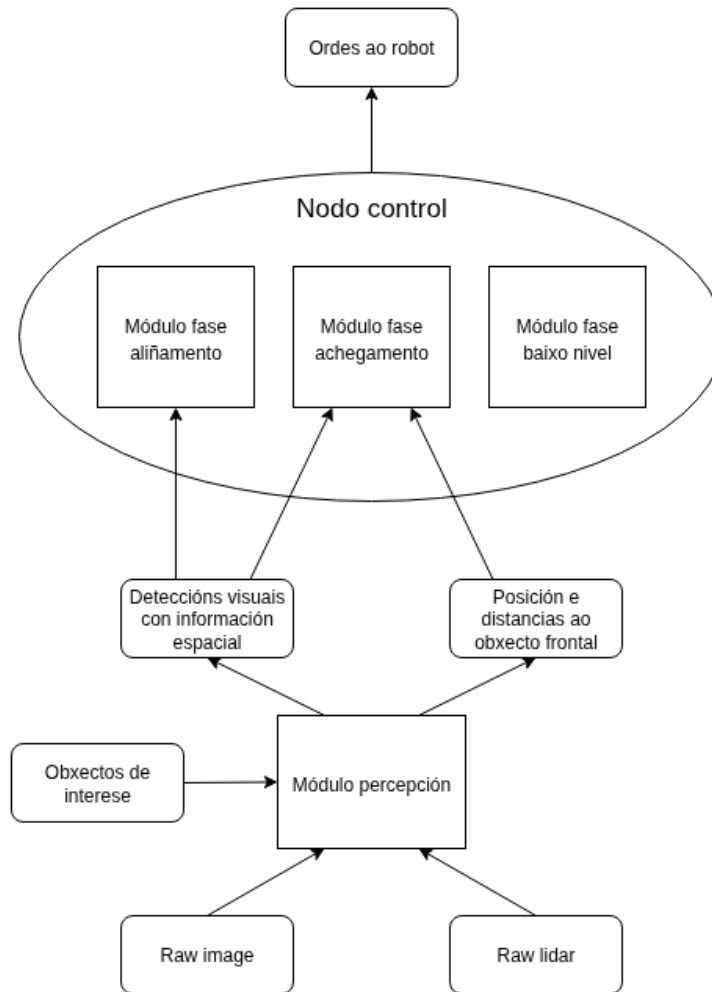


Figura 4.1: Diagrama xeral da arquitectura do sistema, co módulo de percepción, que recibe como entrada a imaxe da cámara, a [nube de puntos LiDAR](#) e os obxectos de interese, e o nodo control, encargado de xerar as ordes para o robot.

tallado dos distintos módulos que o compoñen, describindo as súas funcións específicas e a interacción entre eles.

A diferenza da percepción, o control encárgase nun único nodo centralizado, que recibe os datos de todos os módulos perceptivos e determina as accións que debe executar o robot. Esta decisión de centralizar o control responde á necesidade de coordinar correctamente as distintas fases do comportamento: a aliñación, o achegamento e o baixo nivel. Aínda que ambas fases son independentes desde o punto de vista lóxico e perceptivo, resulta fundamental garantir que non se solapen nin colisionen entre si durante a execución.

Cómpre salientar que a percepción non permanece constante ao longo de todo o proceso, senón que varía significativamente segundo a fase activa. Na fase de aliñamento, o sistema

depende unicamente da cámara RGB para detectar e localizar os obxectos necesarios. Isto permite que o robot se centre no obxecto a patear e se aliñee visualmente co obxectivo final, empregando só a información visual dispoñible.

Porén, unha vez completado este paso, prodúcese unha transición á fase de achegamento, onde a percepción visual deixa de ser efectiva para detectar o balón, debido á perda de visibilidade provocada pola proximidade. Nesta nova fase, o sistema pasa a empregar exclusivamente os datos do LiDAR frontal para localizar a posición da pelota, mentres que a posición do obxectivo se consegue a través da cámara integrada cos datos do LiDAR. Este cambio na fonte de percepción garante unha continuidade robusta na execución do comportamento, adaptando os sensores empregados ás limitacións físicas de cada etapa do proceso.

A representación da arquitectura na figura 4.1 axuda a visualizar esta centralización, así como a forma en que os sensores externos (como o LiDAR ou a cámara RGB) contribúen ao ciclo de percepción e acción. Ao unificar a lóxica de control nun único módulo, evítanse posibles inconsistencias entre decisións tomadas en paralelo e lógrase unha xestión ordenada da transición entre fases. Ademais, isto permite establecer mecanismos de protección que aseguran unha execución máis robusta e previsíbel do comportamento global. Así, o nodo de control actúa como núcleo de decisións reactivas, adaptándose en tempo real ao entorno a partir da información que lle achegan os distintos sensores.

4.2 Modelo de datos

Nesta sección descríbese a estrutura e o propósito dos distintos tipos de datos intercambiados entre os módulos do sistema. O deseño da arquitectura baséase nun enfoque modular, onde os nodos de percepción e control comunícanse mediante tópicos de ROS 2. A maior parte dos datos transmitense utilizando mensaxes estándar proporcionadas por ROS 2 como o *marcador*, imaxes ou *nube de puntos*. Pero tamén se definiron mensaxes personalizadas para dar soporte a funcionalidades específicas da tarefa, como a detección de obxectos, a integración de sensores ou o cálculo de distancias relativas.

4.2.1 Detección visual

As deteccións obtidas mediante o modelo YOLO publícanse inicialmente nun formato que contén unha lista de obxectos detectados, cada un cunha serie de atributos relevantes. Estes inclúen a clase do obxecto (como pelota ou botella), a súa posición na imaxe mediante coordenadas do *bounding box*, a confianza da detección, e información da súa cor representada en formato *Hue, Saturation, Value*. Este conxunto de datos permite tanto a identificación semántica do obxecto como a súa asociación posterior cos puntos da *nube de puntos LiDAR* que se proxectan no mesmo espazo da imaxe.

A maiores, outro tipo de mensaxe estende esta información engadindo a estimación da posición tridimensional de cada obxecto detectado. Esta posición é obtida mediante a integración da información visual co **LiDAR**, e representa a localización do obxecto detectado no espazo tridimensional, dentro dun **frame** global (como o **Odometry**). Isto é crucial para que o sistema poida tomar decisións baseadas na posición real dos obxectos no mundo, e non só na súa localización **2D** na imaxe.

4.2.2 Estimación de distancias frontais

Para permitir un control fino durante a fase de achegamento, definiuse unha mensaxe que resume as distancias medidas cara ao obxecto fronte ao robot. Esta mensaxe contén tres compoñentes: a distancia ao longo do eixo frontal (adiante/atrás), a distancia lateral (esquerda/dereita), e a distancia total calculada como a distancia euclidiana entre o robot e o obxecto. Estes valores son obtidos a partir do filtrado e análise dos puntos **LiDAR**, e permiten ao robot realizar correccións durante os movementos finais antes de executar a acción de pateo.

4.3 Módulo percepción

O sistema de percepción estrutúrase como un conxunto de nodos interconectados, especializados en extraer, transformar e integrar información relevante do entorno a partir de sensores do Go2, como o son a cámara **RGB** e o sensor **LiDAR**. Estes módulos comunícanse entre si a través de tópicos intermedios, cuxos tipos de mensaxe se detallan na sección 4.2.

A figura 4.2 ilustra a arquitectura xeral do sistema perceptivo, mostrando os cinco nodos implicados e os fluxos de información que se establecen entre eles: YOLO*, decaemento **LiDAR**, transformación **frame**, filtrado frontal e integración YOLO* con **LiDAR**. Esta organización modular permite unha separación clara entre as distintas etapas do procesado sensorial.

4.3.1 Nodo YOLO*

Este nodo é responsable de aplicar un modelo de detección de obxectos sobre as imaxes captadas pola cámara. Para cada imaxe recibida, xéranse deteccións con información sobre a clase do obxecto, a súa posición na imaxe e a cor estimada. A saída deste nodo proporciona a base visual sobre a que se constrúe a comprensión do entorno.

4.3.2 Nodo decaemento **LiDAR**

O nodo de decaemento procesa os datos brutos do sensor **LiDAR** para conservar só aqueles puntos situados dentro de zonas de interese, descartando o ruído e o chan. Ademais, mantense unha **nube de puntos** acumulada mediante un mecanismo de decaemento temporal, que

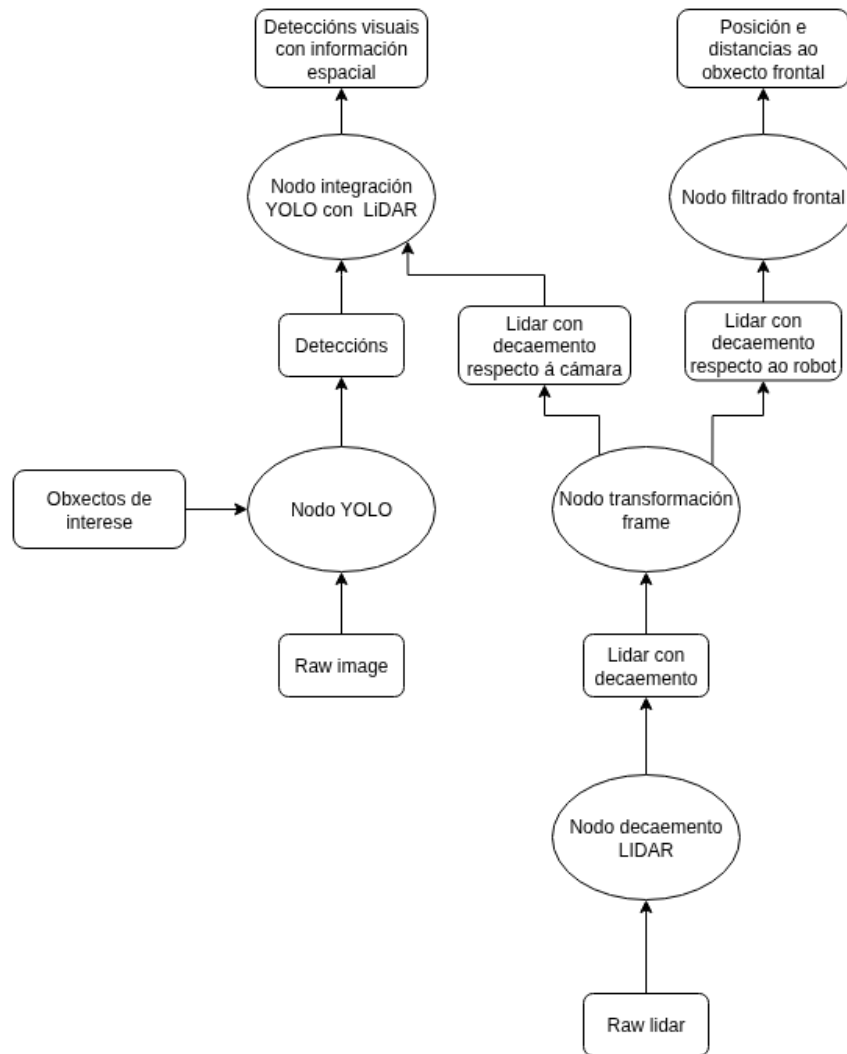


Figura 4.2: Estructura interna do módulo de percepción e o seu fluxo de procesado. Recibe como entradas a imaxe da cámara, a *nube de puntos LiDAR* e a especificación dos obxectos de interese. O sistema de percepción encárgase de construír unha interpretación tridimensional do entorno a partir da información visual e *LiDAR* dispoñible. Mediante a combinación de datos sensoriais e a súa integración espacial, obtéñense representacións que permiten localizar e identificar obxectos de interese con precisión. Como saída, o sistema publica dúas mensaxes principais: as deteccións visuais enriquecidas con información espacial e a posición e distancias relativas ao obxecto frontal detectado.

conserva os puntos relevantes dos últimos segundos, permitindo unha percepción máis robusta e continua. O resultado é unha **nube de puntos** limpa, consistente e restrinxida á zona útil para a percepción activa.

4.3.3 **Nodo transformación frame**

Unha vez filtrados, os puntos do **LiDAR** son transformados aos sistemas de referencia necesarios para as seguintes etapas do sistema perceptivo, concretamente ao **frame** da cámara e ao do robot. Estas transformacións son imprescindibles para, por unha parte, permitir a proxección dos puntos no plano da imaxe e a súa fusión cos datos de detección visual, e por outra, facilitar a súa interpretación espacial dende a perspectiva do propio robot.

4.3.4 **Nodo filtrado frontal**

Este nodo céntrase na análise dos puntos do **LiDAR** situados fronte ao robot co propósito de estimar a posición e as distancias ao obxecto seleccionado para ser pateado. Para iso, realízase unha filtraxe espacial sobre a rexión frontal e calcúlase a distancia media aos puntos relevantes, así como a súa localización media no espazo. A información xerada resulta esencial para os módulos de control encargados da aproximación e execución do golpeo, proporcionando unha medida compacta e directa da posición do obxectivo principal.

4.3.5 **Nodo integración YOLO* con LiDAR**

Neste nodo combínase a información visual procedente da cámara coas medicións tridimensionais do **LiDAR**. Para isto, emprégase a calibración interna da cámara para proxectar os puntos do **LiDAR** no plano da imaxe, determinando cales deles coinciden coas deteccións visuais. Con esta información, calcúlase a posición **3D** de cada obxecto detectado. A posición final exprésase nun **frame** global, garantindo coherencia co resto do sistema. Este proceso permite enriquecer as deteccións visuais con información espacial precisa, proporcionando ao sistema de control datos fiables sobre a localización dos obxectos de interese no espazo tridimensional.

4.4 **Nodo control**

O nodo de control constitúe o núcleo decisorio do comportamento reactivo do robot, orquestrando as accións que deben executarse a partir da información procedente dos distintos módulos perceptivos. A súa responsabilidade principal é xestionar, de forma ordenada e robusta, as tres fases principais do comportamento: o **aliñamento**, o **achegamento** e o **baixo**

nivel. Cada unha destas fases presenta diferentes obxectivos e condicións sensoriais, polo que o nodo adapta tanto a lóxica de control como os criterios de execución segundo o contexto.

Cada fase estrutúrase en varias subfases que permiten modular con maior precisión o comportamento do robot. A pesar de que só se empregan tres parámetros de entrada principais en cada fase (como a posición relativa dos obxectos ou as distancias frontais), a súa interpretación combínase cun conxunto de umbrais que permiten segmentar o comportamento en diferentes subfases. Deste xeito, a presenza de múltiples subfases non implica necesariamente un maior número de variables de entrada, senón unha configuración máis refinada da lóxica de decisión.

Por outra parte, a prioridade das subfases pode variar segundo as características específicas do robot. Por exemplo, nun segundo robot cun comportamento físico distinto, poden ser necesarios axustes na orde das subfases. Isto débese a que certos movementos, como o desprazamento lateral, poden implicar pequenas variacións na traxectoria real, como avanzar ou retroceder lixeiramente, en función da configuración mecánica do sistema.

Ademais, cómpre destacar que os umbrais utilizados para decidir a transición entre subfases non se definen de forma arbitraria, senón que se obteñen a partir dunha análise das variables perceptivas. Estas variables, como o desprazamento horizontal dos obxectos na imaxe ou a distancia frontal medida polo **LiDAR**, son monitorizadas en tempo real e comparadas con valores de referencia baseados no comportamento desexado. A definición destes umbrais pode optimizarse segundo un equilibrio entre precisión e eficiencia: establecer umbrais moi estritos pode mellorar a precisión final do aliñamento ou achegamento, pero tamén aumentar o tempo de execución da tarefa. Pola contra, umbrais máis laxos permiten respostas máis rápidas, a costa dunha menor precisión no resultado.

Un aspecto destacable do comportamento é a flexibilidade introducida ao permitir que unha persoa usuaria especifique previamente tanto a clase (nome) como a cor do obxecto de pateo e do obxecto destino. Esta configuración facilita unha maior interacción e control sobre o comportamento do robot, permitindo adaptar a tarefa a distintos escenarios sen necesidade de modificar a lóxica interna do sistema.

Nos seguintes apartados descríbese o deseño funcional asociado a cada fase dentro deste nodo, detallando a súa estrutura interna, a lóxica de transición entre estados, os criterios perceptivos empregados e os mecanismos deseñados para garantir un comportamento estable e adaptado ás condicións do entorno.

4.4.1 Fase aliñamento

Durante esta primeira fase, o obxectivo principal é posicionar o robot de forma adecuada respecto ao obxecto que se pretende patear e ao obxectivo destino. Para iso, o comportamento divídese en **tres subtarefas**, cuxa execución está guiada por unha orde de prioridade que

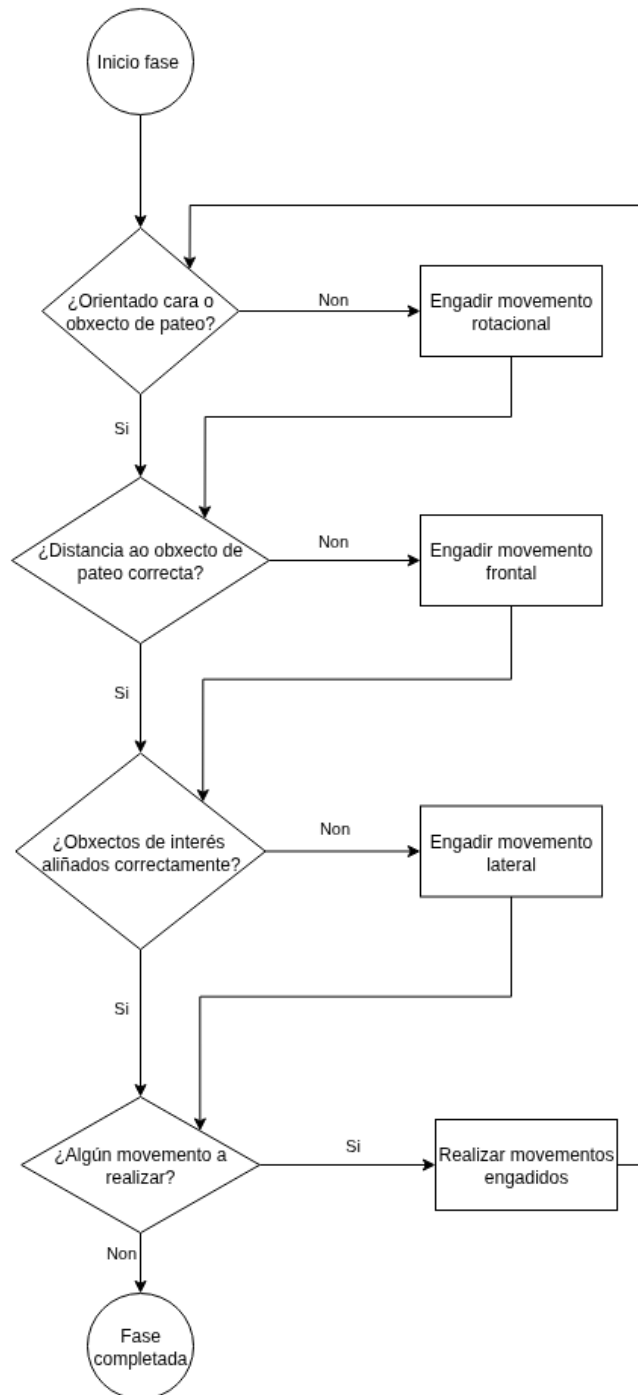
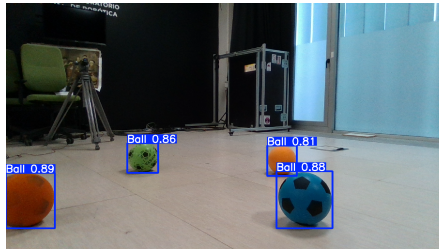
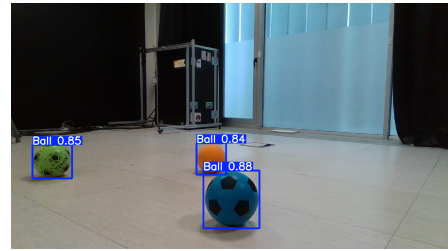


Figura 4.3: Diagrama de flujo do proceso de aliñamento, indicando as condicións de activación e transición entre subtarefas.



(a) Obxecto de pateo desprazado (balón azul) á dereita respecto ao centro da imaxe, requirindo axuste de orientación.



(b) Obxecto de pateo (balón azul) centrado na imaxe tras completar o aliñamento rotacional.

Figura 4.4: Exemplo de aliñamento rotacional: á esquerda, desviación apreciable respecto ao centro; á dereita, centrado correcto do obxecto.

determina cal debe realizarse segundo a necesidade detectada nas percepcións visuais. A figura 4.3 mostra de forma esquemática o fluxo de decisións e transicións entre estas subtarefas. Esta orde garante unha execución coherente e progresiva da aliñación, onde o orden en que se presentan as subtarefas a continuación reflicte tamén a súa prioridade relativa, dende a de maior ata a de menor importancia.

Para xestionar correctamente esta tarefa, o nodo de control define un conxunto de variables clave e aplica un mecanismo xeral de umbrais dobres para cada tipo de movemento visual. Este sistema consiste nun umbral estrito, que determina cando un movemento se considera finalizado, e un umbral máis laxo, que serve para reactivar o movemento no caso de que o obxecto cambie significativamente de posición tras terse aliñado previamente. Esta estratexia evita oscilacións continuas provocadas por pequenas variacións nas deteccións, xa sexa por ruído, movementos internos do sistema ou o pequeno retardo que se produce ao recibir as imaxes da cámara, e asegura un comportamento máis robusto e estable ao longo do tempo.

AL.1 Aliñamento rotacional co obxecto de pateo: o robot xira sobre si mesmo para que o obxecto de pateo apareza centrado na imaxe. Para iso, mídese a desviación horizontal entre o centro da imaxe e o centro da *bounding box* do obxecto. Esta desviación calcúlase como a diferenza en píxeles ou porcentaxe relativa entre o centro horizontal da imaxe e o punto medio entre os bordos esquerdo e dereito do *bounding box* do obxecto. Cando esta desviación é menor a un umbral estrito considérase que o aliñamento está completado, evidenciado na subfigura 4.4b onde o obxecto aparece centrado. Pola contra, se supera un umbral laxo, o axuste reactivase, como ocorre na subfigura 4.4a, onde a desviación respecto ao centro da imaxe aínda é apreciable. Esta acción ten a máxima prioridade, xa que unha orientación correcta é requisito previo para o resto de axustes.

AL.2 Control da distancia ao obxecto de pateo: o nodo estima a distancia frontal ao obxecto de pateo baseándose unicamente na información visual. Para isto, calcúlase



(a) Obxecto de pateo (balón azul) moi lonxe, vese enteiro.

(b) Distancia axeitada, visualízase unha parte do obxecto de pateo (balón azul).

Figura 4.5: Exemplos de avaliación visual da distancia ao obxecto de pateo.

o *ratio* da **bounding box** do obxecto: se este aparece moi ancho en relación á súa altura, interprétase que só se está vendo unha parte do mesmo, o que indica que o robot está suficientemente preto. Se o *ratio* chega a ser demasiado alto, considérase que o robot está demasiado preto do obxecto e debe retroceder.

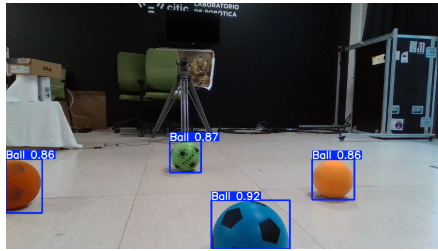
Comparando este *ratio* cun rango óptimo definido por dous umbrais (estrito e laxo), determínase si avanzar, retroceder ou considerar a acción completada. Na figura 4.5 obsérvase un exemplo de obxecto de pateo moi lonxe (subfigura 4.5a), no que se visualiza enteiro, e outro cunha distancia axeitada (subfigura 4.5b), onde só se ve parcialmente.

AL.3 Aliñamento lateral co obxecto destino: unha vez aliñado co obxecto de pateo e situado a unha distancia adecuada, o robot realiza desprazamentos laterais para que o obxecto destino quede aliñado en liña recta co obxecto de pateo. Para avaliar isto, calcúlase a diferenza entre as posicións horizontais (no plano da imaxe) do centro da **bounding box** de pateo e do centro da **bounding box** destino.

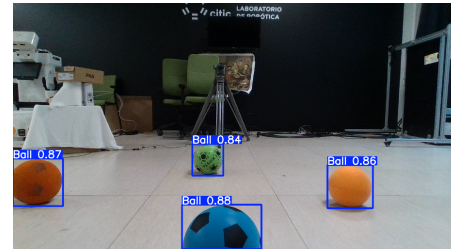
Esta diferenza indícase como o offset entre ambos e compárase cos seus respectivos umbrais para decidir se é necesario activar a corrección, no caso de que o aliñamento sexa incorrecto como se aprecia na subfigura 4.6a, ou finalizar o axuste lateral, no caso de que ambos obxectos estén correctamente aliñados como na subfigura 4.6b.

A prioridade entre accións estaba claramente definida, impedindo que dúas subtarefas se executasen ao mesmo tempo ou que interfirían entre sí. O sistema detectaba cando se alcanzaba o obxectivo de cada subtarefa e transitaba de forma controlada á seguinte, asegurando un comportamento fluído e fiábel baseado nunha execución estrita por prioridades.

Con todo, durante o desenvolvemento e as probas observouse que esta estratexia, aínda que rigorosa, podía prolongar excesivamente o tempo necesario para completar a fase de aliñamento, especialmente en escenarios nos que varios axustes deberían realizarse simultaneamente. Por iso, para optimizar a resposta do robot e acurtar o tempo de execución, o



(a) Obxecto destino (balón verde) desprazado á esquerda respecto ao obxecto de pateo (balón azul), require axuste lateral.



(b) Obxecto de pateo (balón azul) e obxecto destino (balón verde) correctamente aliñados en liña recta.

Figura 4.6: Exemplo de aliñamento lateral: á esquerda, desprazamento horizontal apreciable entre os obxectos; á dereita, aliñamento correcto sen necesidade de axuste.

control evolucionou cara a unha arquitectura máis flexible, na que os movementos poden realizarse en conxunto cando sexa necesario.

Deste modo, cada subtarefa activa o seu movemento segundo as condicións de percepción correspondentes, e cando varias delas requiren axuste ao mesmo tempo, estes execútanse simultaneamente. Esta modificación permite un comportamento máis áxil e adaptativo, acelerando o proceso sen perder precisión nin estabilidade.

Para xestionar correctamente as condicións de activación e desactivación de cada subtarefa, empréganse diversos umbrais baseados en medidas da imaxe, como a posición e tamaño relativo da detección do obxecto a patear. Ademais, debido ao retardo existente na recepción das imaxes da cámara, establécese unha histérese nos umbrais: os valores que activan unha acción son máis estritos ca os que a manteñen, evitando así oscilacións ou cambios prematuros por flutuacións visuais puntuais.

4.4.2 Fase achegamento

Unha vez finalizada a fase de aliñamento, o sistema transita á fase de achegamento, onde os obxectivos perceptivos e de control cambian significativamente. Debido á proximidade do obxecto a patear, a cámara deixa de ser útil para a súa localización, polo que a súa posición determínase agora a partir da información do sensor **LiDAR**. Así mesmo, a posición do obxectivo non se mantén a través da mesma percepción visual directa empregada na fase anterior, senón que se estima mediante un sistema combinado que integra a detección visual inicial coa información espacial proporcionada polo **LiDAR**, permitindo así manter unha estimación robusta da súa localización no espazo tridimensional durante o achegamento.

Nesta fase, os movementos continúan sendo os mesmos que na fase anterior (rotación, avance, retroceso e desprazamento lateral), mais a súa execución está suxeita a novas restricións temporais derivadas de limitacións observadas na estabilidade do robot. En concreto,

detectouse que ao encadear diferentes tipos de movemento, por exemplo pasar dun avance a unha rotación sen ningún intervalo intermedio, o robot experimenta oscilacións ou comportamentos mecánicos imprecisos que dificultan o control preciso da súa posición e orientación. Para mitigar este efecto, establécese un tempo de espera entre cambios de tipo de movemento, permitindo que o sistema se estabilice antes de iniciar a seguinte acción.

Así mesmo, observouse que a realización consecutiva de desprazamentos laterais, especialmente cando se alternan en direccións opostas, provoca desprazamentos involuntarios que afectan negativamente á capacidade do robot para manter unha traxectoria controlada. Para paliar este problema, introdúcese un retardo adicional entre movementos laterais, o que contribúe a manter unha execución máis precisa e segura.

Outra precaución implementada nesta fase é a espera controlada entre movementos de avance cando o robot está moi preto do balón. A curta distancia esixe unha aproximación fina e precisa, polo que se evita encadear movementos de forma continua sen deixar tempo para que se estabilice a posición e orientación.

Do mesmo xeito que se pensou nun principio na fase anterior, esta etapa estrutúrase en subtarefas, executadas segundo unha orde de prioridade predeterminada, dende a de maior ata a de menor prioridade. Esta xerarquía garante unha aproximación progresiva e segura ao obxecto de pateo. Non obstante, a prioridade relativa entre as subfases pode variar lixeiramente segundo o robot utilizado, debido a pequenas diferenzas no seu comportamento dinámico. Esta flexibilidade permite adaptar a execución ás características específicas de cada plataforma sen modificar os criterios básicos de decisión.

A pesar de que nesta fase se controlan unicamente tres variables: o ángulo entre o corpo do robot e a liña que conecta os dous obxectos de interese, a distancia frontal ao obxecto de pateo e a distancia lateral ao mesmo, o comportamento estrutúrase en **seis subtarefas** diferenciadas. Esta separación responde a razóns prácticas de robustez e control: dividir o comportamento en pasos intermedios permite maior estabilidade, modularidade e precisión durante a aproximación.

Ao igual que na fase anterior, nesta etapa tamén se empregan umbrais para determinar cando unha acción concreta pode considerarse completada. Porén, neste caso non se utilizan pares de umbrais para controlar a entrada e saída dunha subfase, senón unicamente umbrais únicos que marcan directamente o cumprimento da condición establecida. Estes valores permiten unha execución clara e directa das transicións.

O esquema xeral de decisións e transicións desta fase móstrase na figura 4.7, que resume visualmente a lóxica seguida polas seis subtarefas. A continuación explicaremos cada unha:

AC.1 Retroceso por proximidade excesiva: se coa información do sensor **LiDAR** se detecta que o robot está demasiado preto do obxecto de pateo, execútase un movemento de retroceso inmediato. Esta acción ten a máxima prioridade, xa que evita colisións non

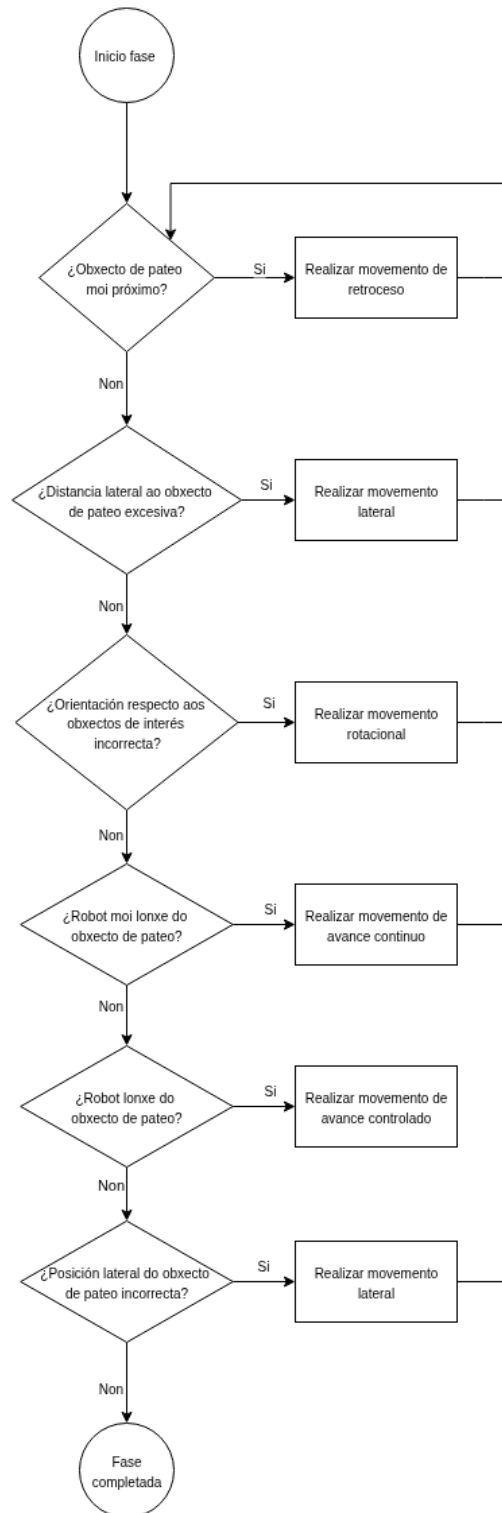
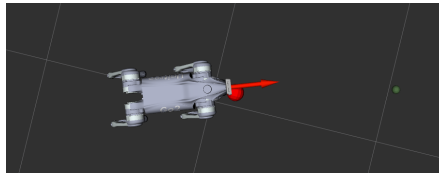
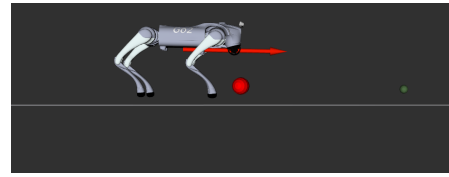


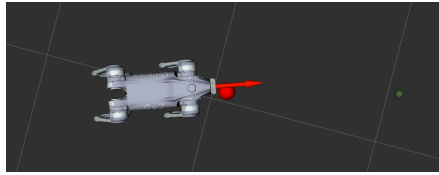
Figura 4.7: Diagrama de fluxo que mostra as decisións e transicións entre subtarefas na fase de achegamento.



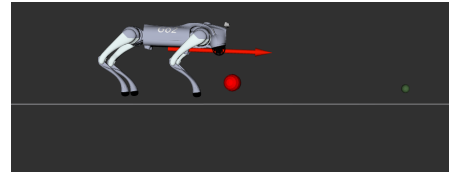
(a) Vista superior de proximidade excesiva.



(b) Vista lateral que confirma a proximidade excesiva.



(c) Vista superior tras o retroceso, con distancia segura.



(d) Vista lateral que mostra a separación adecuada.

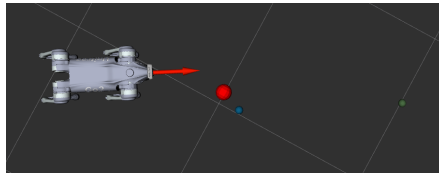
Figura 4.8: Exemplo de retroceso por proximidade excesiva ao obxecto de pateo (esfera vermella): na fila superior obsérvase un estado inicial cunha distancia inferior ao umbral; na fila inferior pode verse unha posición segura tras executar o retroceso.

desexadas e garante espazo suficiente para correccións posteriores.

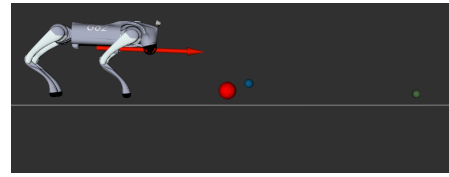
A detección realízase calculando a media dos puntos **LiDAR** situados na zona frontal do robot que caen dentro da área estimada do obxecto de pateo, e obtendo a súa distancia respecto ao robot. Se esta distancia media é inferior a un umbral mínimo establecido, actívase o retroceso tal e como se mostra nas subfiguras 4.8a e 4.8b, onde o robot está excesivamente preto do obxecto. Unha vez executado o retroceso, o robot alcanza unha posición segura, representada nas subfiguras 4.8c e 4.8d, con maior separación respecto ao obxecto de pateo.

AC.2 Corrección lateral por desviación excesiva: esta subfase ten como obxectivo corri-xir desprazamentos laterais excesivos do robot respecto ao obxecto de pateo, que poidan afectar a precisión e estabilidade da aproximación. Para iso, mídese a desviación lateral entre a posición estimada do obxecto de pateo e o eixo central do robot, utilizando os datos do sensor **LiDAR**. Se esta desviación supera o umbral superior establecido, actívase un desprazamento lateral correctivo para achegar o robot á posición desexada.

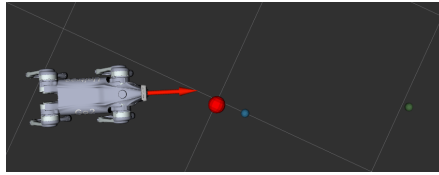
Este proceso está ilustrado nas subfiguras 4.9a e 4.9b, onde se aprecia unha separación lateral considerable entre o robot e o obxecto de pateo, mentres que as subfiguras 4.9c e 4.9d mostran a situación tras realizar a corrección, coa desviación lateral reducida a un nivel aceptable. A acción considérase completada cando a desviación cae por debaixo do umbral estrito, garantindo así unha posición máis adecuada para as fases posteriores de aproximación.



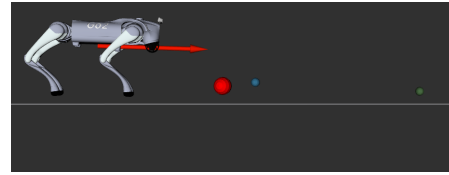
(a) Vista superior dun desplazamento lateral excesivo.



(b) Vista lateral dun desplazamento lateral excesivo.



(c) Vista superior tras o movement lateral correctivo, con distancia lateral mellorada.



(d) Vista lateral tras o movement lateral correctivo, con distancia lateral mellorada.

Figura 4.9: Exemplo de corrección lateral por desviación excesiva respecto ao obxecto de pateo (esfera vermella): na fila superior obsérvase un estado inicial cunha distancia lateral excesiva; na fila inferior pode verse unha posición corrixida tras executar o movemento lateral.

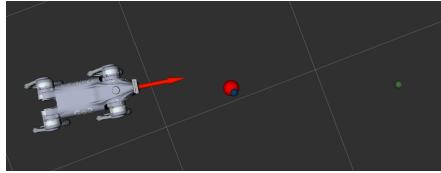
AC.3 Corrección de orientación: o robot axusta a súa rotación para aliñar a súa orientación coa recta que une o obxecto de pateo co obxecto destino. A variable usada nesta subfase é o ángulo formado entre a orientación actual do robot e a dirección estimada desde a posición do obxecto de pateo ata o obxectivo destino, estimado combinando a detección inicial da cámara coa posición relativa actualizada a través do [LiDAR](#).

A acción considérase completada cando este ángulo está dentro dun intervalo de tolerancia definido por umbrais de histérese, como se ilustra nas subfiguras [4.10c](#) e [4.10d](#), onde a orientación do robot coincide coa liña imaxinaria que une ambos obxectos. Pola contra, se o ángulo supera o limiar, como no caso representado nas subfiguras [4.10a](#) e [4.10b](#), a subfase continúa activa ata acadar o aliñamento desexado.

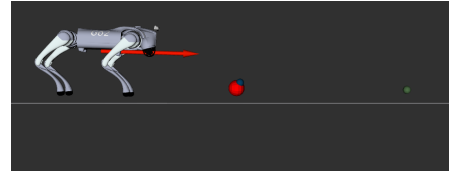
AC.4 Avance a longa distancia: se o robot aínda se atopa lonxe do obxecto de pateo, como ocorre nas subfiguras [4.11a](#) e [4.11b](#), avanza en liña recta para reducir significativamente a distancia a un rango máis controlado, como ocorrería nas subfiguras [4.11c](#) e [4.11d](#). A distancia actual ao obxecto de pateo estímase tomando os puntos [LiDAR](#) dentro do cono central fronte ao robot e estimando a súa posición media.

Se esta distancia supera un certo umbral superior, actívase o avance; cando entra dentro do rango definido polo umbral, a subfase considérase superada.

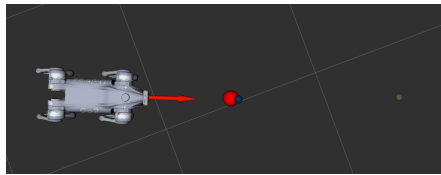
AC.5 Avance a curta distancia: o robot avanza de forma controlada ata unha distancia óptima para realizar a acción final, esta posición pode verse nas subfiguras [4.12c](#) e [4.12d](#).



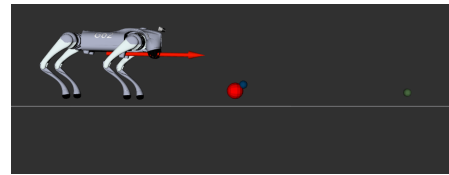
(a) Vista superior coa orientación do robot desviada.



(b) Vista lateral coa orientación do robot desviada.

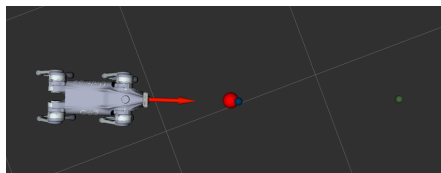


(c) Vista superior tras o axuste, coa orientación do robot aliñada.

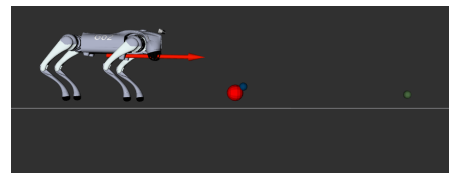


(d) Vista lateral tras o axuste, coa orientación do robot aliñada.

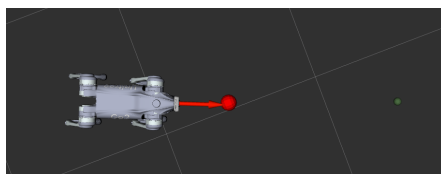
Figura 4.10: Exemplo de corrección de orientación respecto á liña entre os obxectos de pateo (esfera vermella) e destino (esfera verde): na fila superior, desviación inicial; na fila inferior, orientación correcta alcanzada tras o axuste.



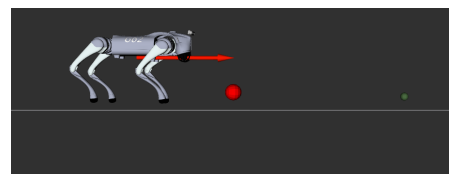
(a) Vista superior do robot lonxe do obxecto de pateo, necesitando avance.



(b) Vista lateral da distancia excesiva ao obxecto de pateo.

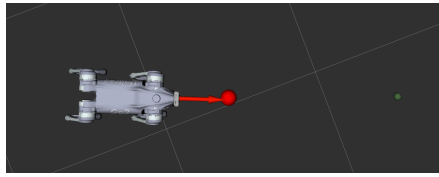


(c) Vista superior tras o avance, o robot alcanza unha distancia adecuada.

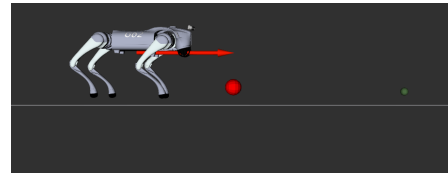


(d) Vista lateral confirmando que a distancia é adecuada.

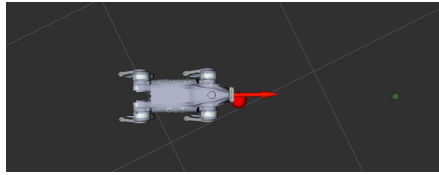
Figura 4.11: Exemplo de avance a longa distancia: na fila superior, o robot encóntrase lonxe do obxecto de pateo (esfera vermella) e é necesario un movemento; na fila inferior, o robot alcanzou a posición desexada tras o avance.



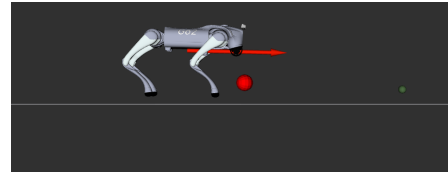
(a) Vista superior mostrando que o robot non alcanzou a distancia óptima.



(b) Vista lateral evidenciando que a distancia frontal aínda é insuficiente.



(c) Vista superior mostrando que o robot alcanzou a distancia óptima tras realizar o movemento de avance.



(d) Vista lateral confirmando que o robot está á distancia correcta tras realizar o movemento de avance.

Figura 4.12: Exemplo de avance a curta distancia: na fila superior, robot lonxe da posición desexada; na fila inferior, o robot está a distancia óptima ao obxecto de pateo (esfera vermella).

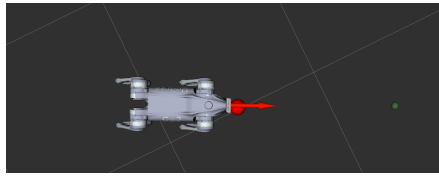
Para decidir cando avanzar, calcúlase a distancia ao obxecto de pateo a partir da media dos puntos **LiDAR** localizados na zona frontal do robot que se atopan dentro da área correspondente ao obxecto. Esta estimación proporciona unha medida fiable e precisa da proximidade real, á que se lle aplica un umbral único que define cando se considera alcanzada a posición adecuada ou pola contra indicar que se debe realizar o avance debido a que a distancia non é correcta, como ocorre nas subfiguras 4.12a e 4.12b.

AC.6 Aliñamento lateral co obxecto destino: unha vez aliñado co obxecto de pateo e situado a unha distancia adecuada, o robot realiza desprazamentos laterais para colocar o obxecto de pateo nunha posición idónea respecto ao seu propio eixo central, como sucede nas subfiguras 4.13c e 4.13d.

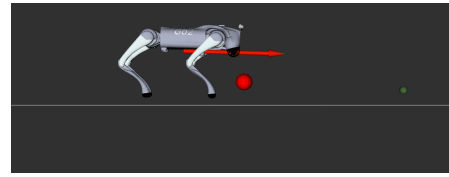
Para iso, calcúlase a diferenza lateral entre a posición do obxecto de pateo, estimada a partir dos datos do **LiDAR**, e o centro do robot. Se esta diferenza cae fóra dun intervalo tolerable, como se observa nas subfiguras 4.13a e 4.13b, actívase o desprazamento lateral. Cando entra dentro do intervalo estrito, considérase completada.

A prioridade de cada subtarefa foi establecida para garantir unha aproximación segura e eficiente ao obxecto de pateo. A prioridade máxima concédese ao retroceso, xa que en situacións de proximidade excesiva resulta fundamental afastarse para evitar un posible contacto co obxecto. Esta acción realízase sempre en primeiro lugar, como medida preventiva para asegurar que calquera outro movemento posterior non provoque unha colisión innecesaria.

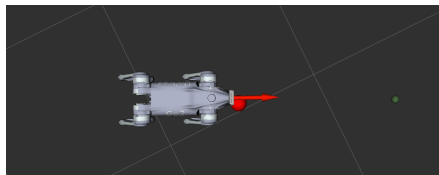
En canto ao aliñamento lateral, a súa posición dentro da secuencia pode variar depen-



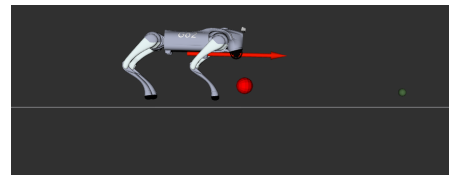
(a) Vista superior mostrando que o obxecto non está correctamente posicionado respecto a posición ideal para o pateo.



(b) Vista lateral na que o obxecto non está correctamente posicionado respecto a posición ideal para o pateo.



(c) Vista superior mostrando que o obxecto está na posición óptima para o pateo despois da corrección lateral.



(d) Vista lateral confirmando que o robot completou correctamente o desprazamento lateral.

Figura 4.13: Exemplo de aliñamento lateral: na fila superior, obxecto de pateo (esfera vermella) nunha posición errónea; fila inferior, obxecto correctamente posicionado respecto ao robot.

dendo do comportamento real do robot durante este tipo de desprazamento. Algúns robots, ao executar movementos laterais, tenden a avanzar lixeiramente, mentres que outros poden retroceder de forma involuntaria. Este posible cambio na prioridade, en función do efecto que produce o movemento no robot, xa foi mencionado anteriormente na sección 4.4. En concreto, se o aliñamento lateral induce un leve avance, é preferible realizalo antes de achegarse demasiado ao obxecto, mentres que se provoca un retroceso, resulta máis axeitado realizalo despois da fase de aproximación próxima. Deste modo, mantense a precisión na colocación final e evítase comprometer a traxectoria por efectos non desexados.

4.4.3 Fase baixo nivel

Unha vez completadas as fases de aliñamento e achegamento, a acción final do sistema consiste en executar o pateo do obxecto desexado. Esta tarefa lévase a cabo empregando un comportamento de baixo nivel xa implementado previamente polo fabricante, o cal permite xerar as ordes de control necesarias directamente sobre os actuadores do robot.

Cómpre destacar que esta solución constitúe unha adaptación, xa que non se dispón dun coñecemento profundo sobre o funcionamento interno dos niveis máis baixos de control do robot. Debido a isto, resulta fundamental actuar con cautela, pois unha execución incorrecta ou descoidada dos movementos podería provocar danos nos motores ou noutras partes sensibles da plataforma robótica. A utilización dun comportamento xa probado permite minimizar estes riscos e facilita a integración segura desta última etapa dentro do sistema xeral.

Detalles de implementación

NESTE capítulo descríbese a implementación dos distintos nodos que forman parte do sistema. Preséntanse os módulos desenvolvidos, a súa funcionalidade e integración, detallando o necesario para comprender o seu funcionamento conxunto.

5.1 Nodo YOLO

Este nodo encárgase de executar o modelo de detección **YOLO** sobre as imaxes recibidas en **ROS 2**, co obxectivo de obter deteccións de obxectos en tempo real. Ademais, para cada obxecto detectado extráese a súa cor no espazo **HSV**, co fin de facilitar a súa identificación e posterior filtrado.

5.1.1 Aplicación do modelo YOLO*

O modelo cárgase ao iniciar o nodo utilizando a librería oficial de *ultralytics* para **YOLO** seguindo as instrucións do código 5.1. Cando se recibe unha mensaxe de imaxe (do tipo *sensor_msgs/msg/Image*) procedente da cámara frontal, esta debe ser convertida a un formato interpretable polo modelo **YOLO**, que traballa con imaxes no formato *numpy.ndarray* e en espazo de cor BGR. Para iso, emprégase a clase *CvBridge*, que permite transformar imaxes entre os formatos de **ROS 2** e OpenCV (código 5.2).

Este proceso transforma a mensaxe *sensor_msgs/msg/Image* nunha imaxe de OpenCV (*cv_image*), que é directamente compatible co modelo de inferencia, devolvendo unha lista

```
1 from ultralytics import YOLO
2
3 self.model = YOLO(model_path) # Carga el modelo preentrenado que se
4                               # encuentra en la ruta model_path
```

Código 5.1: Instrucións para cargar o modelo **YOLO** de *ultralytics*


```

1 from cv_bridge import CvBridge
2
3 self.bridge = CvBridge()
4
5 cv_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding="bgr8")

```

Código 5.2: Traducción de ROS2 a OpenCV

```

1 results = self.model(cv_image)
2
3 detections = results[0].boxes.data.cpu().numpy()

```

Código 5.3: Execución da inferencia do modelo YOLO sobre unha imaxe de OpenCV

de resultados, onde cada entrada representa unha predición sobre a imaxe, o resultado contén todas as deteccións realizadas (ver código 5.3).

5.1.2 Extracción da cor HSV

Unha vez obtidas as deteccións do modelo **YOLO**, procédese a identificar a cor dominante de cada obxecto detectado empregando o espazo de cor **HSV** (Ton, Saturación, Valor), que resulta máis robusto fronte a cambios de iluminación que o espazo **RGB**. Para cada **bounding box**, extráese a rexión correspondente da imaxe e convértese ao espazo **HSV** usando OpenCV como pode verse no código 5.4.

Posteriormente, sepáranse os tres canais **HSV** e calcúlase a mediana de cada un, o que permite obter unha medida robusta da cor dominante, como se pode ver no código 5.5.

5.2 Nodo de filtrado con decaemento no frame ODOM

O obxectivo deste nodo é aplicar un filtrado espacial e temporal sobre a **nube de puntos** procedente do **LiDAR**. Esta estratexia permite manter unha representación densa e acumulada dos puntos visibles recentemente, eliminando aqueles que desaparecen tras un tempo determinado. Este enfoque resulta especialmente útil para estabilizar deteccións en contornas dinámicas.

```

1 roi = img[ymin:ymax, xmin:xmax]
2 roi_hsv = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)

```

Código 5.4: Extracción da rexión de interese e conversión a HSV mediante OpenCV

```

1 h_channel, s_channel, v_channel = cv2.split(roi_hsv)
2 h_median = int(np.median(h_channel))
3 s_median = int(np.median(s_channel))
4 v_median = int(np.median(v_channel))

```

Código 5.5: Cálculo da mediana dos canais HSV para estimar a cor do obxecto

```

1 self.roi_min = np.array([0.0, -2.0, 0.03])
2 self.roi_max = np.array([5.0, 2.0, 0.3])

```

Código 5.6: Definición dos límites da rexión de interese

5.2.1 Definición da Área de Interese e Filtrado Espacial

Este nodo define unha **área de interese (ROI)** no sistema de referencia *base_link*, que representa o volume frontal do robot onde se espera atopar obxectos relevantes (ver código 5.6). Para aplicar correctamente o filtro sobre a *nube de puntos* (normalmente no *frame* do LiDAR), realízase a transformación deste volume ROI desde *base_link* ao *frame* da *nube de puntos* mediante TF2. Isto fáise transformando cada unha das esquinas do cubo definido polo ROI.

Unha vez transformado o ROI ao *frame* da *nube de puntos*, o nodo filtra todos os puntos contidos dentro deste volume espacial, conservando só aqueles que se atopan dentro dos límites definidos.

5.2.2 Decaemento Temporal

Unha vez filtrados espacialmente os puntos relevantes, o nodo almacénaos nunha estrutura tipo *deque*, asociando a cada punto un timestamp que indica o instante no que foi recibido, como se pode ver no código 5.7.

De forma continua, o nodo elimina todos os puntos cuxo tempo de vida supere un limiar definido como **tempo de decaemento**, que neste caso se fixa en 1,5 segundos, o cal se pode ver no código 5.8. Isto garante que a *nube de puntos* publicada corresponda unicamente a observacións recentes, evitando depender dunha soa *nube de puntos* instantánea. A *nube de puntos* resultante publícase no tópico */lidar/filtered_with_decay_odom*.

```

1 current_time = msg.header.stamp.sec + msg.header.stamp.nanosec
2               * 1e-9
3 self.recent_points.append([x, y, z], current_time)

```

Código 5.7: Asociación temporal aos datos da nube de puntos

```

1 while self.recent_points and current_time -
    self.recent_points[0][1] > self.decay_time_sec:
2     self.recent_points.popleft()

```

Código 5.8: Eliminación dos puntos antigos en función do tempo de validez

```

1 from tf2_sensor_msgs.tf2_sensor_msgs import do_transform_cloud
2
3 transformed_cloud = do_transform_cloud(self.latest_cloud_msg,
4                                       transform)

```

Código 5.9: Transformación dunha nube de puntos ao marco de referencia desexado mediante

5.3 Nodo de transformación frame

O nodo encárgase de transformar a *nube de puntos* orixinal, publicada no marco *ODOM*, a dous marcos distintos: *front_camera* e *base_link*. Para realizar esta operación utilízase a función *do_transform_cloud*, que aplica a transformación xeométrica correspondente obtida a través de *TF2*. As *nube de puntos* resultantes publícanse en tópicos separados para o seu uso posterior por outros nodos do sistema (ver código 5.9).

5.4 Nodo de filtrado frontal

Este nodo recibe unha *nube de puntos* no marco de coordenadas *base_link* e aplica un filtrado espacial para detectar obxectos situados xusto fronte ao robot. A zona de interese está definida por límites fixos en *x* e *y*, pero introduce un concepto adicional: o filtrado dinámico en altura, que axusta o rango válido de valores *z* segundo a distancia ao robot.

5.4.1 Área de interese frontal

A primeira parte do filtrado consiste en quedar só cos puntos situados fronte ao robot dentro dun área estreita, como se realiza no código 5.10. O resultado é unha selección de puntos nunha rexión rectangular fronte ao robot.

```

1 mask_xy = (
2     (points[:, 0] >= self.min_x)
3     & (points[:, 0] <= self.max_x)
4     & (points[:, 1] >= self.min_y)
5     & (points[:, 1] <= self.max_y)
6 )
7 points = points[mask_xy]

```

Código 5.10: Filtrado espacial frontal sobre a nube de puntos

```

1 def compute_min_z(self, x):
2     if x <= self.near_x:
3         return self.min_z_near
4     elif x >= self.far_x:
5         return self.min_z_far
6     else:
7         ratio = (x - self.near_x) / (self.far_x - self.near_x)
8         return self.min_z_near + ratio * (self.min_z_far
9             - self.min_z_near)

```

Código 5.11: Interpolación lineal para determinar a altura mínima aceptable segundo a distancia ao robot

```

1 mask_dynamic_z = []
2 for pt in points:
3     x, y, z = pt
4     min_z = self.compute_min_z(x)
5     mask_dynamic_z.append(min_z <= z <= self.max_z)
6
7 mask_dynamic_z = np.array(mask_dynamic_z, dtype=bool)
8 filtered_points = points[mask_dynamic_z]

```

Código 5.12: Aplicación do filtro de altura dinámica punto a punto na nube

5.4.2 Altura dinámica segundo a distancia

Este filtrado en altura non utiliza un rango estático de valores Z, senón que axusta dinamicamente o limiar superior da altura permitida en función da distancia frontal ao robot. Isto débese a unha observación importante: os puntos do chan capturados polo **LiDAR** non manteñen unha altura constante no eixo Z cando se afastan do robot. Pola contra, a súa altura relativa (en Z) decrece lixeiramente de forma progresiva. O cálculo faise cunha interpolación lineal definida que pode ser vista no código 5.11. E a aplicación deste criterio para cada punto da **nube de puntos** pode verse no código 5.12.

5.4.3 Selección dos puntos máis representativos

Unha vez aplicados os filtros espaciais, calcúlase a distancia euclídea ao robot e extráese a mediana para eliminar valores erróneos, como se realiza no código 5.13. A continuación,

```

1 distances = np.linalg.norm(filtered_points, axis=1)
2 median = np.median(distances)
3 threshold = 0.2
4 inlier_mask = np.abs(distances - median) < threshold
5 inlier_points = filtered_points[inlier_mask]

```

Código 5.13: Eliminación de puntos afastados mediante filtrado por mediana da distancia

```

1 x_avg = np.mean(inlier_points[:, 0])
2 y_avg = np.mean(inlier_points[:, 1])
3 z_avg = np.mean(inlier_points[:, 2])

```

Código 5.14: Cálculo da posición media dos puntos filtrados como representación do obxecto

```

1 projected_points, _ = cv2.fisheye.projectPoints(points_3d, rvec,
2                                              tvec, self.K, self.D)
3 u = projected_points[:, 0]
4 v = projected_points[:, 1]

```

Código 5.15: Proxección dos puntos 3D sobre o plano da imaxe cun modelo de cámara *fisheye*

calcúlase a posición media (ou punto central) de todos os puntos, co obxectivo de obter unha representación equilibrada da detección no espazo (ver código 5.14).

Este punto utilízase para representar a posición do obxecto no espazo 3D, ademais, publícase a distancia frontal ao obxecto mediante unha mensaxe personalizada *FrontDistance*, que contén os compoñentes *distance_x*, *distance_y* e *distance_total*.

Finalmente, para facilitar a visualización en *ROS Visualization Tool*, publícase un *marcador* asociado á detección no tópico */detections_marker_array*, facendo uso dun *marcador* tipo *SPHERE* centrado na posición detectada.

5.5 Nodo integración cámara con LiDAR

Este nodo encárgase de asociar os obxectos detectados pola cámara frontal (mediante o modelo *YOLO*) cos puntos da *nube de puntos LiDAR* que se atopan visibles dentro do campo de visión da cámara. O obxectivo é estimar a posición 3D de cada obxecto detectado, proxectando os puntos da *nube de puntos* ao plano da imaxe e identificando cales pertencen a cada *bounding box*. Finalmente, publícase a posición estimada de cada obxecto no marco *ODOM*, xunto con *marcador* de visualización para *RViz*.

5.5.1 Proxección dos puntos LiDAR á imaxe da cámara e asociación ás deteccións

Para comprobar que puntos da *nube de puntos* están visibles desde a cámara, realízase a súa proxección empregando o modelo de cámara tipo *fisheye*, utilizando a matriz intrínseca *K*

```

1 inside_image = (u >= 0) & (u < self.image_width)
2               & (v >= 0) & (v < self.image_height)

```

Código 5.16: Filtrado dos puntos proxectados que caen dentro da imaxe

```

1 in_bbox = (u >= det.left) & (u <= det.right)
2         & (v >= det.top) & (v <= det.bottom)

```

Código 5.17: Identificación dos puntos que se atopan dentro dunha *bounding box* detectada

```

1 median_point = np.median(points_array , axis=0)

```

Código 5.18: Cálculo da mediana dos puntos asociados a unha detección

e os coeficientes de distorsión D , ambos obtidos da mensaxe *CameraInfo*.

Neste caso, non se realiza unha proxección directa convencional mediante multiplicación matricial, xa que a cámara utilizada presenta unha lente *fisheye* que introduce unha forte distorsión nos bordes da imaxe. Este tipo de distorsión non pode ser modelado cunha simple matriz de proxección, por este motivo, emprégase a función *cv2.fisheye.projectPoints* de OpenCV, que permite aplicar unha proxección precisa tendo en conta a distorsión propia deste tipo de ópticas, como se pode visualizar no código 5.15. Despois da proxección, verificase cales dos puntos proxectados caen dentro da imaxe (ver código 5.16).

Unha vez coñecidos os puntos LiDAR que se atopan dentro da imaxe, compróbase cales deles están dentro das *bounding box* das deteccións realizadas por YOLO (ver código 5.17). Os puntos que se atopan dentro da *bounding box* son almacenados e asociados á detección correspondente.

5.5.2 Estimación da posición 3D dos obxectos detectados

Para cada obxecto detectado, procédese a calcular unha estimación da súa posición tridimensional a partir dos puntos LiDAR que lle foron asociados. En primeiro lugar, calcúlase o punto central (mediana) dos puntos, como se realiza no código 5.18. Logo, calcúlase a distancia euclídea de cada punto respecto á mediana, co obxectivo de descartar valores fóra de rango (ver código 5.19). A posición estimada final calcúlase como a media dos puntos considerados válidos, poder verse no código 5.20.

```

1 distances = np.linalg.norm(points_array - median_point , axis=1)
2 filtered_points = points_array[distances <= distance_threshold]

```

Código 5.19: Filtrado dos puntos máis afastados respecto á mediana para obter unha medida máis robusta

```
1 centroid = np.mean(filtered_points , axis=0)
```

Código 5.20: Cálculo do centróide dos puntos filtrados como estimación final da posición 3D

```
1 image_center = self.image_width // 2
2
3 kick_center = self.get_center_x(self.kick_bbox)
4 kick_offset = kick_center - image_center
5
6 def get_center_x(self, det):
7     return (det.left + det.right) // 2
```

Código 5.21: Cálculo do desprazamento horizontal do obxecto de pateo respecto ao centro da imaxe

5.6 Módulo fase aliñamento

5.6.1 Aliñamento rotacional co obxecto de pateo

Nesta primeira subfase, o robot tenta centrar o obxecto de pateo na imaxe, realizando rotacións sobre o seu propio eixe. Para iso, emprégase un control baseado en histérese, que compara o desprazamento horizontal do obxecto detectado respecto ao centro da imaxe con dous umbrais: *center_enter* e *center_exit*.

O centro da imaxe (*image_center*) calcúlase como a metade do ancho da imaxe da cámara, posteriormente, determínase a posición horizontal do centro da *bounding box* do obxecto de pateo (*kick_center*) e calcúlase o desprazamento horizontal (*kick_offset*) como diferenza entre ambos, da maneira que se realiza no código 5.21.

Este valor (*kick_offset*) indica se o obxecto está centrado ou desprazado, e en que dirección. A lóxica de histérese garante estabilidade: só se activa a rotación cando o erro supera o limiar de saída *center_exit*, e só se considera completado o aliñamento cando o erro baixa por debaixo do limiar de entrada *center_enter*. Esta estratexia de dobre limiar (máis estrito para parar e máis laxo para comezar o movemento) foi xa descrita na sección 4.4.1.

O control de velocidade faise proporcional ao erro, multiplicando o *kick_offset* por un pequeno factor de ganancia. Deste xeito, a velocidade de rotación é maior canto máis desprazado estea o obxecto da súa posición desexada no centro da imaxe (ver código 5.22).

```
1 vz = -self.error_speed * kick_offset
```

Código 5.22: Rotación proporcional ao desprazamento horizontal

```

1 ball_width = self.kick_bbox.right - self.kick_bbox.left
2 ball_height = self.kick_bbox.bottom - self.kick_bbox.top
3 ball_ratio = ball_width / float(ball_height + 1e-6)

```

Código 5.23: Cálculo da proporción da bounding box do obxecto de pateo

```

1 vx = self.retreat_speed # Retroceso
2 vx = self.advance_speed # Avance

```

Código 5.24: Desprazamentos frontal segundo a distancia ao obxecto

5.6.2 Control da distancia ao obxecto de pateo

Nesta segunda subfase, o robot determina se está situado a unha distancia adecuada respecto ao obxecto de pateo. Para iso, emprégase unha estratexia baseada na proporción entre o ancho e o alto da **bounding box** do obxecto (*ball_ratio*), así como a posición vertical do bordo inferior da **bounding box** na imaxe (ver código 5.23).

O control baséase na activación e mantemento de dous modos (*retreat_active* e *advance_active*), que garanten unha transición suave entre estados. Os desprazamentos realízanse cunha velocidade constante, como se poder ver no código 5.24.

Se o obxecto está demasiado preto (**bounding box** moi ancha e con proporción elevada), o robot retrocede. Se está lonxe (**bounding box** igual de ancha que alta ou borde inferior alto na imaxe) avanza. Estes movementos están regulados tamén por dous pares de umbrais de histérese: *too_close_enter/too_close_exit* para o retroceso, e *too_far_enter/too_far_exit* para o avance.

5.6.3 Aliñamento lateral co obxecto destino

Unha vez que o robot está aliñado co obxecto de pateo e situado a unha distancia correcta, pasa a aliñarse lateralmente co obxecto destino. O obxectivo é que ambos obxectos queden aliñados en liña recta.

Para iso, calcúlase o desprazamento horizontal entre os centros das dúas caixas de detección (*kick_center* e *goal_center*), e obténse o desprazamento lateral, como se pode ver no código 5.25. Este erro contrólase usando un esquema de histérese similar aos anteriores, o cal foi introducido na sección 4.4.1, mediante os umbrais *lateral_enter* e *lateral_exit*.

```

1 kick_center = self.get_center_x(self.kick_bbox)
2 goal_center = self.get_center_x(self.goal_bbox)
3 offset_lateral = goal_center - kick_center

```

Código 5.25: Cálculo do desprazamento lateral entre obxectos de interese


```
1 vy = offset_lateral * self.error_speed
```

Código 5.26: Movement lateral correctivo segundo desprazamento

```
1 self.last_move_time = self.get_clock().now()
```

Código 5.27: Actualización do tempo do último movemento

Se a diferenza entre o centro do obxecto destino e o do obxecto a patear (*offset_lateral*) supera o primeiro limiar definido, o robot interpreta que o obxecto destino non está correctamente aliñado en termos laterais. Neste caso, execútase un movemento lateral cunha velocidade constante cara á dirección necesaria para corrixir ese desprazamento. A dirección calcúlase en función de que lado está o obxecto destino respecto ao obxecto a patear (ver código 5.26).

5.7 Módulo fase achegamento

Durante esta fase, o robot realiza un achegamento preciso cara ao obxecto de pateo en dirección ao obxecto destino, con correccións sucesivas de posición e orientación. A estratexia está dividida en seis subfases ben diferenciadas, activadas segundo distintas condicións espaciais e angulares. Ademais, todas as accións de movemento están reguladas por temporizadores e lóxicas de espera (*cooldown*) para evitar oscilacións rápidas entre accións incompatibles.

5.7.1 Control temporal entre movementos

Antes de cambiar dun tipo de movemento a outro (por exemplo, pasar de rotación a desprazamento lateral), o sistema require que transcorra un tempo mínimo de espera. Isto evítase mediante unha marca de tempo global *self.last_move_time* que se actualiza tras cada movemento, sexa do tipo que sexa (ver código 5.27). Así, antes de executar calquera nova acción de movemento, compróbase que o tempo pasado dende o último movemento sexa maior ao tempo mínimo de espera (*self.switch_cooldown*) e que a acción que se quere realizar é distinta

```
1 now = self.get_clock().now()
2 elapsed_since_switch = (now - self.last_move_time).nanoseconds / 1e9
3 # Exemplo de querer realizar a rotación
4 if self.last_move_type != "rotation"
5     and elapsed_since_switch < self.switch_cooldown:
6     return
```

Código 5.28: Condicións para executar un novo tipo de movemento

```

1 self.last_lateral_move_time = None
2 self.last_forward_move_time = None

```

Código 5.29: Inicialización dos temporizadores de movemento lateral e frontal

```

1 lateral_elapsed = (now-self.last_lateral_move_time).nanoseconds/1e9
2 if lateral_elapsed < self.cooldown:
3     return

```

Código 5.30: Control do tempo de espera para movemento lateral

á última (*self.last_move_type*), como se pode ver no código 5.28.

Adicionalmente, o desprazamento lateral e o avance próximo dispoñen dun mecanismo de espera específico e independente entre execucións consecutivas, para evitar oscilacións. Cada tipo garda a súa propia marca de tempo, como se realiza no código 5.29.

Durante a fase de achegamento, antes de desprazarse lateralmente, revísase este tempo da forma en que se realiza no código 5.30. E para o avance cara ao obxecto ver o código 5.31. Desta maneira, o sistema distingue entre os cambios de estratexia (mediante o *cooldown* xeral) e a execución continua dun mesmo tipo de acción (controlada por *cooldowns* individuais), garantindo tanto estabilidade como resposta eficiente.

5.7.2 Retroceso por proximidade excesiva

A primeira subfase dentro do módulo de achegamento céntrase na xestión de situacións nas que o robot está excesivamente preto do obxecto de pateo. Este control resulta crítico para garantir que as seguintes manobras de aliñamento e aproximación poidan realizarse coa marxe suficiente, evitando posibles colisións ou imprecisións na execución da acción de pateo.

A lóxica implementada nesta subfase verifica constantemente a distancia ao obxecto detectado mediante o módulo de percepción. En concreto, contrólanse os valores do campo *distance_x* da mensaxe */front_object_distance*, que representa a distancia ao obxecto ao longo do eixo frontal do robot. Se esta distancia é inferior a un limiar definido (*self.too_close_threshold*), considérase que o robot se atopa nunha situación de proximidade excesiva.

Neste caso, a acción correctiva consiste en realizar un desprazamento cara atrás cunha velocidade constante predefinida, como se realiza no código 5.32.

```

1 forward_elapsed = (now-self.last_lateral_move_time).nanoseconds/1e9
2 if forward_elapsed < self.cooldown:
3     return

```

Código 5.31: Control do tempo de espera para movemento frontal

```
1 self.move(self.retreat_speed, 0.0, 0.0)
```

Código 5.32: Desprazamento cara atrás por proximidade excesiva

```
1 lateral_direction = -1.0 if lateral_distance < 0 else 1.0
2 lateral_speed = lateral_direction * self.far_lateral_speed
3 self.move(0.0, lateral_speed, 0.0)
```

Código 5.33: Corrección lateral por desviación excesiva (movemento lateral longo)

5.7.3 Corrección lateral por desviación excesiva

A segunda subfase actívase cando a desviación lateral do obxecto respecto do eixo central do robot é moi grande. O propósito é recentrar a posición lateral antes de continuar co resto do achegamento, evitando que un erro excesivo se propague ás seguintes manobras.

A medida de referencia é *distance_y* (lateral) procedente do tópico de percepción, que se garda en *lateral_distance*. Cando *lateral_distance* supera o limiar *self.too_far_lateral_threshold*, détéctase unha desviación excesiva e prepárase unha corrección de maior amplitude. Para evitar cambios demasiado frecuentes, a execución está condicionada por un período de espera (*self.switch_cooldown*) e polo último tipo de movemento executado, tal e como se define no control temporal (véxase 5.7.1).

A acción correctiva consiste nun desprazamento lateral a velocidade fixa, cuxo signo depende do lado ao que se atopa o obxecto, isto pode observarse no código 5.33. O movemento mantense ata que a desviación cae por debaixo do limiar *self.too_far_lateral_threshold*.

5.7.4 Corrección de orientación

A terceira subfase céntrase na corrección da orientación do robot cara ao obxectivo destino. A orientación desexada calcúlase a partir das posicións actuais do obxecto a patear e do obxecto destino, ambas proporcionadas polo módulo de percepción. A diferenza entre estas dúas posicións permite obter o ángulo que conecta ambos puntos no plano horizontal. Posteriormente, este ángulo desexado compárase coa orientación actual do robot. A diferenza entre ambos valores constitúe o erro angular que se debe corrixir (ver código 5.34).

Unha vez calculado este erro, o robot decide se debe realizar unha rotación correctiva. Se o erro supera un limiar (*self.angle_threshold*), entón execútase un xiro proporcional ao erro, utilizando un factor de ganancia, como se pode visualizar no código 5.35.

5.7.5 Avance a longa distancia

A lóxica desta cuarta subfase analiza o valor de *distance_x*, que representa a distancia frontal ao obxecto detectado, calculado polo módulo de percepción. Se esta distancia supera

```

1 from transformations import euler_from_quaternion
2
3 ball = self.front_object_position
4 goal = self.goal_marker_position
5 try:
6     quat = self.robot_pose.pose.orientation
7     current_yaw, _, _ = euler_from_quaternion([quat.x, quat.y,
8                                                quat.z, quat.w])
9 except Exception as e:
10     self.get_logger().warn(f"No se pudo obtener la orientación del
11                             robot: {e}")
12     return
13 dx = goal.x - ball.x
14 dy = goal.y - ball.y
15 desired_angle = math.atan2(dy, dx)
16 angle_error = self.normalize_angle(desired_angle - current_yaw)
17
18 def normalize_angle(self, angle):
19     return math.atan2(math.sin(angle), math.cos(angle))

```

Código 5.34: Cálculo do erro angular a partir da orientación do robot e as posicións dos obxectos de interese

```

1 rotation_speed = self.rotation_error_speed * angle_error
2 self.move(0.0, 0.0, rotation_speed)

```

Código 5.35: Rotación proporcional ao erro angular detectado

```

1 front_speed = self.advance_speed
2 self.move(front_speed, 0.0, 0.0)

```

Código 5.36: Avance proporcional á distancia frontal ao obxecto

```
1 front_distance = self.front_distances.distance_x
```

Código 5.37: Lectura da distancia frontal ao obxecto

```
1 front_speed = self.close_advance_error_speed * front_distance
2 self.move(front_speed, 0.0, 0.0)
```

Código 5.38: Avance controlado segundo distancia curta ao obxecto

un limiar mínimo (*self.close_threshold*), interprétase que o robot aínda está lonxe do obxectivo, polo que é necesario avanzar.

En caso de cumprirse as condicións de espera (ver sección 5.7.1) e lonxanía, o robot realiza un desprazamento cara adiante proporcional á distancia ao obxecto. A velocidade calcúlase mediante unha ganancia constante aplicada á distancia, como se pode ver no código 5.36.

5.7.6 Avance a curta distancia

Unha vez o robot se atopa lateralmente na posición axeitada, é necesario realizar un movemento de aproximación frontal para acadar unha distancia óptima respecto ao obxecto. Esta quinta subfase ten como obxectivo executar avances curtos e controlados cando a distancia ao obxecto aínda supera un determinado limiar.

A medición da distancia ao obxecto realízase a través do campo *distance_x* recibido do módulo de percepción (ver código 5.37). Se esta distancia supera o limiar definido mínimo en *front_distance_threshold*, e se cumpriron os tempos de espera mencionados na sección 5.7.1, procédese ao avance. A velocidade de avance calcúlase de maneira proporcional á distancia restante, cun factor de escala, como se realiza no código 5.38. Isto permite un avance progresivo e estable, especialmente importante cando o obxecto se atopa xa a pouca distancia.

5.7.7 Aliñamento lateral co obxecto destino

O obxectivo desta sexta e última subfase é que o obxecto a patear se atope a unha distancia lateral concreta respecto ao centro do robot. Esta distancia ideal está configurada mediante o parámetro *self.lateral_distance_position*, que define a separación desexada no momento previo ao pateo. Para acadar este obxectivo, calcúlase un erro lateral a partir das medidas da distancia frontal recibidas do módulo de percepción. En concreto, extráese o valor *distance_y*, que

```
1 lateral_distance = self.front_distances.distance_y
2 lateral_error = lateral_distance - self.lateral_distance_position
```

Código 5.39: Cálculo do erro lateral segundo a posición do obxecto

```

1 lateral_speed = self.lateral_error_speed * lateral_error
2 self.move(0.0, lateral_speed, 0.0)

```

Código 5.40: Desprazamento lateral proporcional ao erro detectado

```

1 def execute_kick(self):
2     self.send_webrtc_kick()
3
4 def send_webrtc_kick(self):
5     msg = WebRtcReq()
6     msg.api_id = 1016
7     msg.parameter = ""
8     msg.topic = "rt/api/sport/request"
9     self.publisher.publish(msg)

```

Código 5.41: Acción para realizar o pateo do obxecto de interés

representa a posición lateral do obxecto máis próximo (ver código 5.39).

Se o valor absoluto do erro lateral supera o limiar definido, *self.lateral_distance_threshold*, considérase necesaria unha corrección. No caso de cumprir as condicións de espera explicadas na sección 5.7.1 execútase o movemento lateral cunha velocidade proporcional ao erro detectado da maneira en que se realiza no código 5.40.

5.8 Módulo fase baixo nivel

A implementación realízase mediante o envío dunha mensaxe de tipo *WebRtcReq* a través dun tópic específico, que activa o comportamento usado para o pateo no sistema interno do robot. Este comportamento forma parte da API nativa da plataforma, que encapsula internamente o control dos motores e a coordinación do movemento.

A seguinte función, que pode verse no código 5.41, define a execución do pateo. O campo *api_id* establece a acción concreta a executar, neste caso o comportamento de pateo, mentres que *topic* indica o canal interno no que debe publicarse a orde.

NESTE capítulo da memoria descríbese o conxunto de experimentos realizados para avaliar o funcionamento do sistema desenvolvido. O obxectivo principal destas probas é verificar a integración dos módulos de percepción e control no robot Unitree Go2, comprobando a súa capacidade para detectar os obxectos de interese e executar o comportamento desexado en diferentes condicións.

6.1 Limitacións do simulador

Nun inicio estaba previsto realizar as primeiras probas nun simulador, coa idea de reducir riscos e acelerar o desenvolvemento. Porén, axiña se comprobou que o simulador non ofrecía garantías reais de que os resultados puidesen extrapolarse ao robot físico. Isto débese a varias limitacións clave:

- **Física non realista:** os modelos de movemento e colisión non replicaban con fidelidade o comportamento dinámico do robot, o que introducía desviacións significativas.
- **Sensores inventados ou aproximados:** a cámara tivo que ser engadida manualmente, polo que a súa configuración non coincidía coa dispoñible no robot real. O mesmo sucedía co [LiDAR](#), que non era equivalente ao sensor físico.
- **Desaxuste coa realidade:** incluso axustando parámetros, as condicións virtuais non eran representativas das probas reais. O feito de que unha funcionalidade funcionase no simulador non era sinónimo de éxito no robot.

Por estes motivos, decidiuse prescindir do simulador e realizar directamente as probas no robot real. Aínda que isto supuxo un maior investimento de tempo, garantiu que os resultados fosen consistentes e robustos, ofrecendo unha validación máis fiable da metodoloxía desenvolvida.

6.2 Conexión vía WiFi con CycloneDDS

Outro aspecto explorado foi o uso de CycloneDDS para establecer comunicacións co robot a través da WiFi. O funcionamento por defecto de CycloneDDS baséase en dous pasos: primeiro un descubrimento *multicast*, e unha vez atopados os participantes establécese a comunicación por *unicast*. Isto provoca un problema cando os dispositivos se atopan en redes distintas, xa que o descubrimento *multicast* non chega a completarse correctamente.

A solución pasa por forzar a comunicación a través de *unicast* especificando manualmente as direccións IP dos participantes, o que require configurar CycloneDDS mediante un ficheiro .xml. O inconveniente principal foi que esta configuración debe aplicarse en todas as máquinas participantes, incluíndo a máquina interna do Go2, á que non se pode acceder directamente.

Para tentar solventar esta restrición, buscouse unha estratexia alternativa: realizar inicialmente o descubrimento a través dunha conexión ethernet directa, co obxectivo de establecer a conexión *unicast*. Unha vez establecida, desconectábase o cable ethernet e, mediante a Jetson (con conexión WiFi), redirixíase á dirección IP ethernet da máquina interna do Go2 para manter a comunicación a través da rede sen fíos. Se ben esta aproximación funcionou en ocasións illadas, non foi posible replicar o proceso de maneira consistente nin fiable, polo que finalmente descartouse como solución viable.

6.3 Problemas cos robots

Durante o desenvolvemento do proxecto, xurdiron diversos problemas relacionados tanto co hardware dos robots empregados como coas condicións de execución das probas. Estes inconvenientes afectaron directamente ao rendemento e á fiabilidade das deteccións visuais, así como á capacidade de control preciso dos movementos. Ademais, provocaron atrasos significativos no calendario previsto, xa que foi necesario investir tempo adicional na identificación das causas, na procura de solucións e na adaptación da metodoloxía de traballo.

Neste apartado descríbense os principais problemas detectados, as súas causas máis probables, e as solucións ou medidas adoptadas para mitígalos.

6.3.1 Problemas coa cámara do Go2A-B

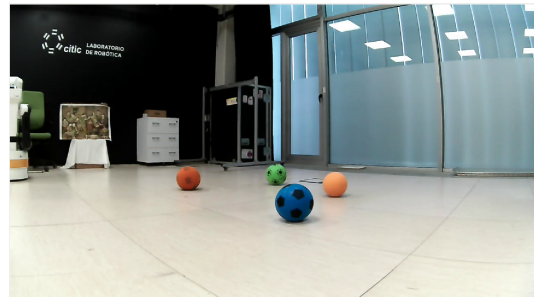
A cámara integrada nos robots Go2 presentou varios problemas que dificultaron as tarefas de percepción. Un deles foi a variación automática do brillo e/ou saturación da imaxe, en función do fondo visual. Este axuste automático provocaba que, en determinadas condicións, a cámara aplicase un brillo excesivo. Este fenómeno fíxose especialmente notable cando se pechaban as cortinas da sala de probas: ao ser paredes e cortinas de cor negra, o sistema da

cámara incrementaba o brillo para compensar, facendo que a imaxe resultase demasiado clara e reducindo o contraste dos obxectos. O problema agravábase ao acender todas as luces da sala, xa que o reflexo da luz no chan intensificaba aínda máis o exceso de brillo, dificultando a detección fiable dos obxectos de interese. Este problema pode visualizarse na subfigura 6.1a.

Nun primeiro momento mitigouse o problema mantendo as cortinas abertas e a iluminación artificial maioritariamente apagada, conseguindo un mellor resultado como o que se pode observar na subfigura 6.1b. Aínda así, o problema persistía parcialmente, especialmente cando o robot se orientaba en certas direccións.



(a) Problemas da cámara no robot Go2: exceso de brillo e imaxe borrosa.



(b) Primeira solución: cortinas abertas e iluminación reducida.

Figura 6.1: Problemas de percepción asociados á cámara integrada dos robots Go2 e primeira solución adoptada.

Adicionalmente, no robot Go2B detectouse outro problema específico: a cámara ofrecía unha imaxe borrosa, na que só se enfocaban correctamente os obxectos situados moi preto da lente. Isto facía que a percepción visual fose extremadamente complicada, especialmente se coincidía co problema de brillo excesivo descrito anteriormente. En casos extremos, mesmo para unha persoa resultaba difícil localizar algúns obxectos na escena.

A solución final foi cambiar ao outro robot dispoñible, o Go2A, cuxa cámara se atopaba en bo estado e non presentaba este defecto de enfoque.

6.3.2 Problemas co Go2A

Tras un período de uso normal, o robot Go2A presentou unha avaría grave: nunha xornada de probas, ao intentar acendelo, non completou a secuencia de arranque. Para descartar problemas na batería, probouse cunha segunda batería, pero o fallo persistiu. Ademais, detectouse un cheiro a queimado procedente do compartimento da batería. A pesar de realizar unha inspección visual e probar distintas conexións, non se puido determinar a causa exacta da avaría, polo que se decidiu contactar co servizo técnico correspondente.

Como medida de continxencia, retomouse o uso do Go2B, ao que se lle instalou unha cámara Red, Green, Blue adquirida recentemente. Grazas á modularidade e robustez do siste-

ma, a substitución da cámara integrada pola cámara montada externamente foi sinxela. Este cambio non só resolveu a indisponibilidade do Go2A, senón que tamén eliminou o problema de autoaxuste de brillo/saturación mencionado na subsección anterior (6.3.1), xa que a nova cámara permitía un control manual máis preciso dos parámetros da imaxe.

6.3.3 Movementsos “suaves” e controlados

Durante as probas iniciais co Go2, non estaba claro o nivel de control que se podía exercer sobre a velocidade e suavidade dos movementos. Tras varios ensaios, descubriuse que o robot ten unha velocidade mínima detectable: se se lle envía unha orde cunha velocidade inferior a este limiar, non se moverá en absoluto. Ademais, non existe un modo de control que permita executar movementos lentos e graduais de forma continua. Isto impide realizar desprazamentos laterais ou rotacionais precisos e controlados, xa que calquera movemento efectivo comeza cunha velocidade mínima predefinida polo sistema.

Adicionalmente observouse que, ao executar un movemento nunha dirección e, a continuación, outro idéntico pero en sentido contrario, o robot non sempre regresa á posición inicial do primeiro desprazamento. Este comportamento provoca pequenos erros que, acumulados, dificultan a reproducibilidade dos axustes de posición.

En conxunto, estas limitacións fan que posicionarse no punto exacto para realizar o pateo resulte máis complexo do que sería se fose posible executar movementos máis lentos, precisos e controlados.

6.4 Probas específicas

Nesta sección descríbense con maior detalle as probas centradas en aspectos concretos do sistema. O seu propósito é avaliar o rendemento de compoñentes individuais, como a detección visual baseada en [YOLO](#), a integración cos datos do [LiDAR](#) ou a execución do comportamento de aproximación e golpeo.

6.4.1 Probas co módulo YOLO*

Co obxectivo de avaliar o rendemento do sistema de percepción baseado en [YOLO](#), adestráronse diferentes versións do modelo (YOLOv5, YOLOv8 e YOLOv11), cada unha nas súas variantes *nano* e *small*, mantendo idénticas condicións de adestramento. En todos os casos, o modelo foi configurado para a detección de dúas clases: pelota e botella.

A comparación realizouse empregando un mesmo vídeo de proba, garantindo que todas as versións procesasen exactamente as mesmas secuencias de imaxes. A partir destas execucións, obtivéronse métricas clave como:

Modelo	Precisión	Recall	Tempo medio (s)	Desviación estándar (s)	Tempo mínimo (s)	Tempo máximo (s)
YOLOv5-nano	0.990	0.975	0.048	0.005	0.041	0.084
YOLOv5-small	0.983	0.973	0.116	0.012	0.100	0.235
YOLOv8-nano	0.994	0.982	0.054	0.005	0.048	0.094
YOLOv8-small	0.962	0.975	0.138	0.013	0.119	0.210
YOLOv11-nano	0.995	0.980	0.068	0.005	0.057	0.121
YOLOv11-small	1.000	1.000	0.147	0.013	0.127	0.220

Táboa 6.1: Comparativa entre as distintas versións de **YOLO** empregadas no sistema de percepción. Resultados para 739 imaxes.

- **Precisión de detección:** número de deteccións correctas sobre o total de obxectos presentes.
- **Taxa de falsas deteccións:** porcentaxe de deteccións sen obxecto real.
- **Tempo medio de inferencia por imaxe:** medida do custo computacional de cada modelo.

Os resultados preséntanse na táboa 6.1, onde se comparan de forma directa os valores obtidos por cada modelo e variante, permitindo analizar o impacto da versión e do tamaño no rendemento global.

A análise dos resultados presentados na Táboa 6.1 mostra que todos os modelos acadaron valores moi elevados de *precisión* e *recall*, sempre por riba de 0.950, o que evidencia a súa robustez na detección tanto de pelotas como de botellas. Isto indica que, independentemente da versión ou do tamaño do modelo, o rendemento en termos de calidade de detección é moi alto e consistente.

Non obstante, a principal diferenza observada entre as variantes *nano* e *small* atópase no tempo medio de inferencia. Mentres que os modelos *nano* ofrecen tempos reducidos, as versións *small* requiren máis do dobre de tempo en comparación. Este resultado reflicte como a elección do modelo implica priorizar entre dúas dimensións distintas a rapidez de execución e o nivel de detalle na detección, sendo os *nano* máis axeitados para escenarios en tempo real, mentres que os *small* poderían ser preferibles cando a prioridade sexa maximizar a precisión absoluta, aínda á costa de maior custo computacional. Dado que neste traballo se busca implementar un comportamento real que require un procesamento o máis próximo posible ao tempo real, resulta máis conveniente empregar os modelos *nano*. Entre estes modelos *nano* seleccionouse o **YOLOv11-nano**, que ofreceu resultados lixeiramente mellores e máis consistentes, polo que foi o modelo empregado no desenvolvemento final.

6.4.2 Probas da percepción LiDAR

O obxectivo principal das probas de percepción LiDAR foi validar o correcto funcionamento do sistema cando se emprega exclusivamente o sensor LiDAR. Neste caso, a tarefa específica consistía en obter a posición do obxecto frontal ao robot. As probas centráronse en escenarios que puidesen xerar confusión ou dificultar a detección, como:

- Presenza de obxectos laterais próximos ao robot.
- O obxecto frontal non se atopaba completamente centrado diante do robot.
- Existencia de obxectos máis grandes ou paredes situadas detrás do obxecto frontal.

Estas condicións permitiron verificar a robustez do algoritmo de percepción LiDAR, asegurando que se puidese determinar correctamente a posición do obxecto frontal sen ser afectado por elementos circundantes. A precisión e consistencia das medicións foron avaliadas en función da concordancia coa posición real observada no entorno experimental.

Presenza de obxectos laterais próximos ao robot.

Neste caso analizouse o comportamento do sistema cando existían elementos próximos situados lateralmente entre o robot e o obxecto de interese. Na Figura 6.2 obsérvase como un obxecto lateral, situado entre o robot e o obxecto de pateo chega a influír na detección do obxecto frontal, facendo que a estimación sexa errónea. Pola contra, na Figura 6.3 móstrase unha situación análoga, mais nesta ocasión o obxecto lateral non afecta ao resultado da detección, identificándose correctamente o obxecto frontal.

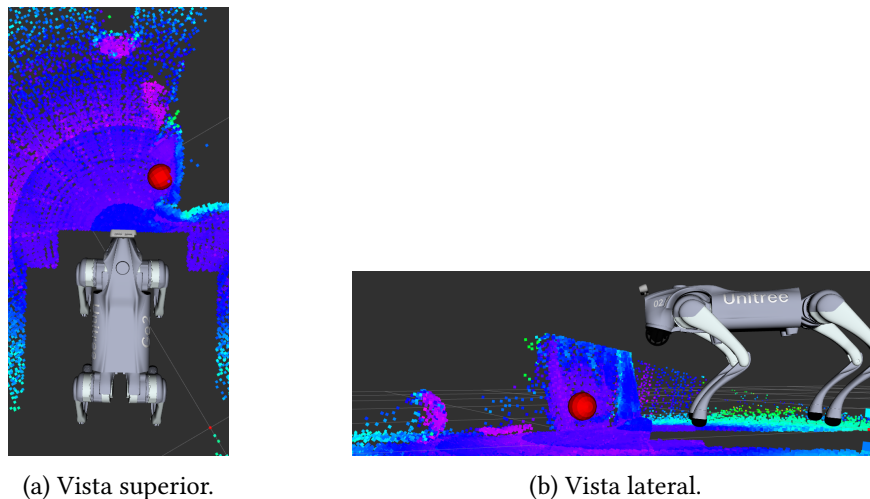


Figura 6.2: Influencia dun obxecto lateral na detección do obxecto frontal.

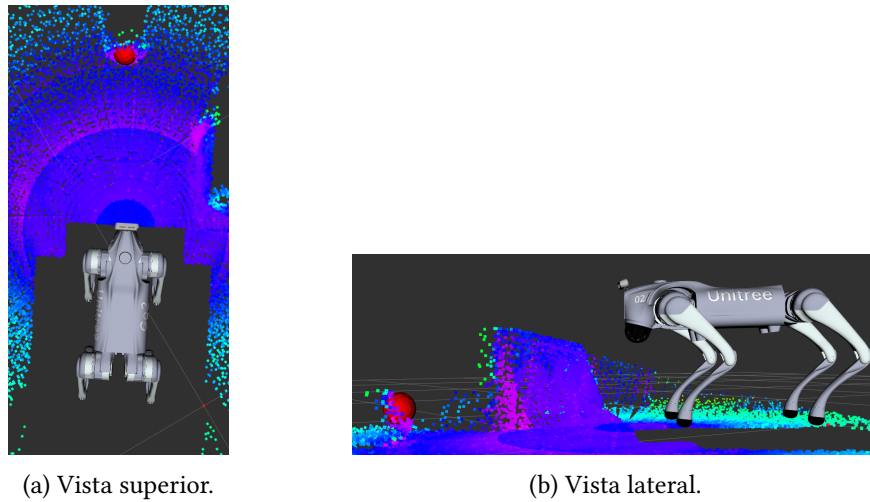


Figura 6.3: Un obxecto lateral non afecta á detección do obxecto frontal.

En conclusión, aínda que en certas condicións a presenza de obxectos laterais poida alterar a detección, este efecto non é crítico no comportamento global. O motivo é que, cando un obxecto lateral chega a impedir a identificación correcta do obxecto frontal, tamén imposibilita o avance físico do robot, polo que o comportamento do sistema vese igualmente afectado.

O obxecto frontal non se atopaba completamente centrado diante do robot.

Outra das probas consistiu en avaliar a robustez do sistema cando o obxecto frontal non se atopaba perfectamente aliñado co eixo central do robot. Na figura 6.4 pódese comprobar como, a pesar de que o obxecto non se sitúa exactamente no centro, o sistema é capaz de identificalo correctamente como obxecto frontal, mantendo así a coherencia da detección.

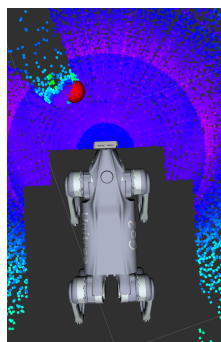


Figura 6.4: Detección correcta do obxecto frontal malia atoparse desprazado lateralmente.

Existencia de obxectos máis grandes ou paredes detrás do obxecto frontal.

Finalmente, avalíouse o efecto da presenza doutros obxectos situados detrás do obxecto de interese. Na figura 6.5 obsérvase como un obxecto de gran tamaño colocado xusto detrás do obxecto de pateo provoca un erro na detección, facendo que o sistema non identifique correctamente a posición do obxecto frontal. Sen embargo, como se mostra na Figura 6.6, cando o obxecto de maior tamaño se encontra a unha distancia lixeiramente maior, este xa non interfere no proceso de detección, permitindo localizar correctamente o obxecto de interese.

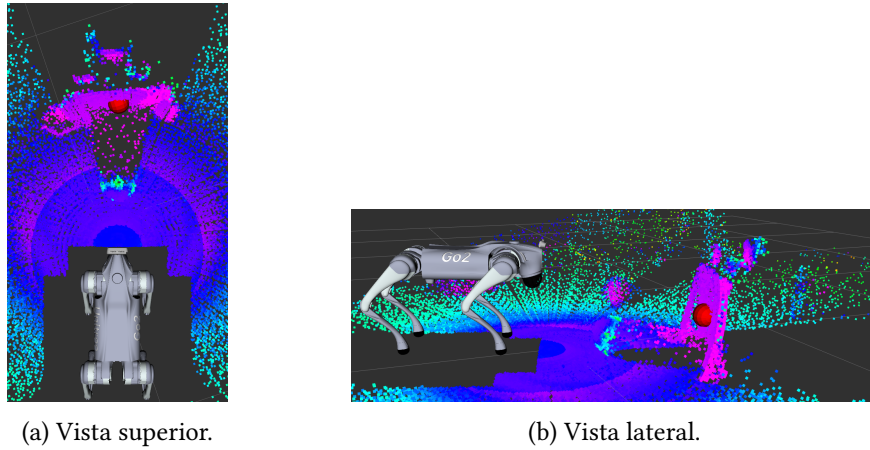


Figura 6.5: Obxecto grande situado detrás do obxecto de interese inflúe na detección frontal.

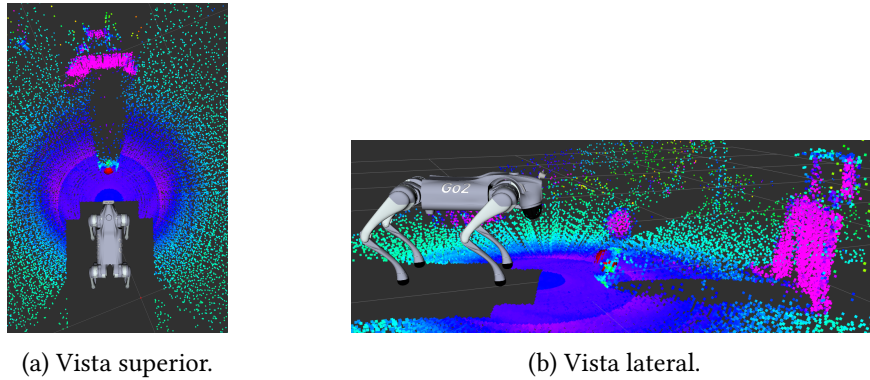


Figura 6.6: Separación suficiente entre o obxecto grande e o obxecto de interese evita interferencias na detección.

Finalmente, como se observa na figura 6.7, incluso cando existe un obxecto pequeno — que podería corresponderse cun obxecto destino— situado moi próximo ao obxecto de pateo, este non chega a afectar á detección frontal, o que evidencia a capacidade do sistema para distinguir entre obxectos de diferentes tamaños e relevancia.

En síntese, estas probas confirmaron que o sistema de percepción **LiDAR** presenta unha

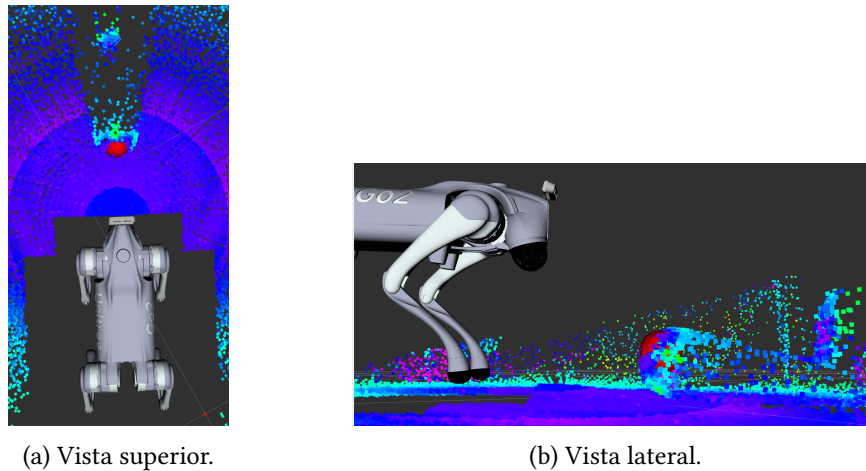


Figura 6.7: Obxecto pequeno preto do obxecto de interese que non interfere na detección.

elevada robustez fronte a diferentes condicións do entorno, asegurando unha detección fiable do obxecto frontal en situacións complexas.

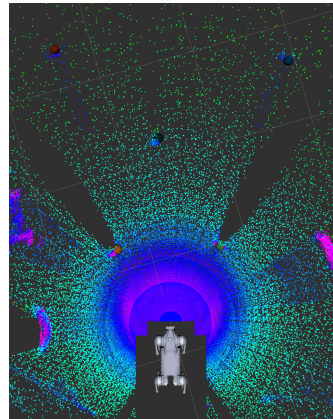
6.4.3 Probas da integración LiDAR e cámara-YOLO*

As probas de integración entre LiDAR e o sistema de visión baseado en cámara con detección YOLO* tiveron como obxectivo avaliar a calidade do posicionamento 3D dos obxectos detectados, combinando información visual e de profundidade. En particular, buscouse que:

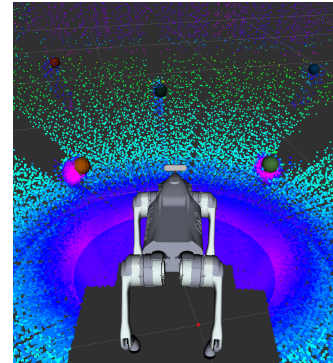
- A posición 3D dos obxectos fose correcta independentemente da súa localización na imaxe captada pola cámara.
- A presenza de múltiples obxectos ou variacións na súa orientación non afectase á precisión do cálculo da posición.
- O sistema combinase de maneira coherente a información do LiDAR coas deteccións de YOLO*, garantindo que a estimación 3D reflectise a posición real do obxecto.

Na figura 6.8 pódese observar como o sistema cumpre cos obxectivos plantexados na proba: aparecen múltiples obxectos situados en diferentes posicións do espazo e, tanto na vista superior como na frontal, as súas posicións 3D son correctamente estimadas. Isto confirma que a localización dos obxectos non depende da súa posición na imaxe, que a presenza de varios obxectos simultáneos non degrada o cálculo da precisión e que a fusión LiDAR-cámara proporciona unha estimación coherente e robusta da realidade.

Estas probas permitiron verificar que a fusión de sensores proporcionaba unha representación consistente e fiable do espazo ao redor do robot, mellorando a precisión na localización de obxectos e garantindo que o sistema fose robusto fronte a variacións na escena visual.



(a) Vista superior coas estimacións 3D dos obxectos.



(b) Vista frontal coas estimacións 3D dos obxectos.

Figura 6.8: Resultados da integración LiDAR–cámara YOLO*, onde se observa que múltiples obxectos en distintas posicións son correctamente localizados en 3D.

6.4.4 Probas da fase aliñamento

A fase de aliñamento ten como obxectivo principal situar o robot nunha posición óptima que lle permita afrontar a seguinte fase de achegamento nas mellores condicións posibles, garantindo que a súa orientación e posicionamento sexan axeitados para executar os movementos posteriores con precisión. Este obxectivo de acadar unha posición ideal é prioritario e condiciona todas as decisións que se toman durante esta etapa.

De maneira secundaria, tamén se persegue optimizar o tempo total que dura a fase, reducindo o período empregado en realizar os axustes necesarios, mais sempre sen comprometer a calidade da posición final acadada. É dicir, a rapidez nunca debe antepoñerse á corrección do aliñamento, xa que unha posición inadecuada podería penalizar o rendemento global do comportamento ao pasar á seguinte fase.

Para acadar estes obxectivos, realizáronse múltiples probas prácticas nas que se introducíron cambios significativos no código, entre os cales destacan:

- Modificación dos umbrais empregados para determinar cando executar os movementos correspondentes a cada subfase, co fin de mellorar a sensibilidade e evitar axustes innecesarios.
- Axuste da velocidade máxima permitida durante a fase, equilibrando a precisión e a rapidez dos desprazamentos.
- Adaptación dinámica das velocidades de movemento en función do erro calculado en cada momento, permitindo que os desprazamentos sexan máis suaves e controlados cando o robot xa está preto da posición ideal.

Variable	Config. 1	Config. 2	Config. 3
Aliñamento rotacional co obxecto de pateo			
center_enter	50	100	75
center_exit	100	150	125
Control da distancia ao obxecto de pateo			
too_close_exit	2.2	2.5	2.5
too_close_enter	1.5	1.8	1.8
too_far_exit	1.0	1.0	1.0
too_far_enter	1.3	1.2	1.2
retreat_speed	-0.12	-0.12	-0.12
advance_speed	0.12	0.12	0.12
Aliñamento lateral co obxecto destino			
lateral_enter	80	80	60
lateral_exit	100	120	100
Outras variables xerais			
min_speed	0.12	0.12	0.12
max_speed	0.20	0.20	0.30
error_speed	0.001	0.001	0.0005

Táboa 6.2: Valores das variables empregadas nas distintas configuracións das probas na fase de aliñamento, agrupadas por subfase.

Estas modificacións no código deron lugar á definición de distintas configuracións experimentais, nas que se asignaron distintos valores ás variables que controlan o comportamento do robot nesta fase. Cada unha destas variables foi previamente descrita no capítulo 5, nas seccións correspondentes onde se explica a súa función e influencia no proceso (véxase a sección 5.6). Na táboa 6.2 recóllense as principais variables xunto cos valores empregados nas diferentes configuracións avaliadas, co obxectivo de analizar o impacto de cada axuste no rendemento da fase de aliñamento.

Adicionalmente, para cada unha destas configuracións analizáronse distintas medidas de rendemento co fin de determinar cal delas resultaba máis axeitada para os obxectivos propostos. Os criterios de avaliación e os resultados obtidos para cada configuración recóllense na táboa 6.3, o que permite establecer unha comparación directa entre os valores das variables empregados e o desempeño final alcanzado.

Tras os resultados obtidos na táboa 6.3 pode observarse unha mellora de tempo para cada configuración, isto é debido a que a primeira configuración empregaba uns valores de velocidade máxima reducida e uns umbrais de entrada e saída máis estritos. Isto provocaba que o

Configurac.	Tempo medio (s)	Desviación estándar (s)	Tempo mínimo (s)	Tempo máximo (s)	Número intentos
Config. 1	56.50	10.20	43.80	74.90	8
Config. 2	40.00	6.80	31.50	55.30	10
Config. 3	28.50	4.60	22.40	37.90	10
Config. 3 + execución paralela	14.70	4.05	8.90	22.70	54

Táboa 6.3: Resultados comparativos das configuracións probadas na fase de aliñamento.

robot realizase numerosos movementos correctivos, xa que superaba con facilidade os límites definidos e debía corrixirse de maneira repetida. Como consecuencia, o tempo medio de execución foi maior.

Na segunda configuración introducíronse melloras ao axustar os valores de entrada e saída dos diferentes umbrais. Estes cambios permitiron reducir o número de correccións innecesarias e aumentar a fluidez do comportamento, conseguindo unha redución significativa no tempo medio.

Para a terceira configuración axustáronse mellor os umbrais e incrementouse lixeiramente a velocidade máxima. Estes cambios permitiron realizar os movementos de maneira máis rápida mentres se seguen evitando as correccións de maneira repetida.

Finalmente, xunto coa terceira configuración engadiuse a posibilidade de executar movementos de forma simultánea. Grazas a isto, o robot puido avanzar, corrixir lateralmente e aliñarse ao mesmo tempo, evitando pausas intermedias e acadando así unha redución moi substancial do tempo de execución (case un cuarto) sen comprometer a porcentaxe de éxito.

6.4.5 Probas da fase achegamento

A fase de achegamento ten como obxectivo principal situar o robot nunha posición idónea para a fase final de pateo. Unha colocación precisa é fundamental para garantir que, ao executar o movemento final de pateo (baixo nivel), o obxecto de pateo golpee o obxecto destino, cumprindo así co obxectivo global da secuencia. A validación máis directa desta posición consiste precisamente en realizar o pateo e comprobar que o impacto se produce tal e como se pretendía.

A pesar de que a precisión posicional é a prioridade nesta fase, tamén se procura, como obxectivo secundario, optimizar o tempo total que esta require. Non obstante, esta optimización resulta máis complexa que na fase de aliñamento, xa que o achegamento implica movementos máis delicados e medidos, nos que unha execución excesivamente rápida pode comprometer a precisión final. Reducir en exceso o tempo pode derivar nunha colocación incorrecta, aumentando a probabilidade de que o obxecto de pateo non acade o destino.

- **Tempos de retardo entre cambios de tipo de movemento:** introducidos para manter a estabilidade do robot. Unha redución destes tempos pode diminuír a duración da fase, pero tamén pode afectar á estabilidade global.
- **Valores dos umbrais de decisión:** uns umbrais máis laxos permiten que o robot cumpra os criterios con maior rapidez, reducindo o tempo total, pero sacrificando precisión na colocación. Pola contra, uns umbrais máis estritos melloran a colocación e incrementan a probabilidade de éxito no pateo, aínda que alonguen a fase.
- **Equilibrio entre tempo e precisión:** esta etapa require atopar un punto intermedio que manteña un tempo de execución razoable sen comprometer a posición final necesaria para o éxito do pateo.

Co obxectivo de axustar o comportamento do robot durante a fase de achegamento, definiuse un conxunto de configuracións experimentais baseadas en diferentes combinacións de parámetros. Estas variables foron descritas previamente no capítulo 5 (véxase a sección 5.7). Na táboa 6.4 preséntanse os valores empregados en cada configuración, o que permite analizar como cada elección inflúe na estratexia de aproximación.

Unha vez establecidas as configuracións, realizáronse diferentes experimentos orientados a avaliar o seu rendemento práctico. Para iso definíronse métricas específicas que permitiron comparar de forma obxectiva o comportamento do robot baixo cada conxunto de parámetros. Os resultados destas probas recóllense na táboa 6.5, facilitando así a identificación das configuracións máis eficientes e axeitadas aos obxectivos do sistema.

Como se pode observar na táboa 6.4, os principais cambios entre as distintas configuracións céntranse en permitir uns umbrais máis laxos, especialmente nas variables relacionadas coa posición lateral (*lateral_distance_position* e *lateral_distance_threshold*). Esta decisión responde á dificultade do robot para acadar unha posición exacta debido a que os movementos non podían ser totalmente controlados. Así, ampliar a marxe de tolerancia na posición lateral permitiu compensar estas limitacións e, en consecuencia, reducir o tempo necesario para completar a tarefa, como se demostra na táboa 6.5. Polo tanto, a adaptación das configuracións non buscou unicamente optimizar o tempo, senón principalmente facilitar que o robot acadase unha posición válida desde a que continuar coa seguinte fase do comportamento, asegurando así un funcionamento máis robusto do sistema.

6.5 Probas finais

Co obxectivo de validar o comportamento completo desenvolvido, realizáronse múltiples probas nas que se executou o sistema de principio a fin, avaliando tanto a súa taxa de éxito como o tempo total de execución. O propósito principal desta fase foi comprobar que o

Variable	Config. 1	Config. 2	Config. 3
Retroceso por proximidade excesiva			
too_close_threshold	0.34	0.34	0.34
retreat_speed	-0.12	-0.12	-0.12
Corrección lateral por desviación excesiva			
too_far_lateral_threshold	0.10	0.10	0.12
far_lateral_speed	0.12	0.12	0.16
Corrección de orientación			
angle_threshold	0.05	0.05	0.07
rotation_error_speed	0.6	0.6	0.6
Avance a longa distancia			
far_threshold	0.70	0.60	0.60
Avance a curta distancia			
close_threshold	0.38	0.37	0.37
close_advance_error_speed	0.4	0.4	0.4
Aliñamento lateral co obxecto destino			
lateral_distance_position	-0.028	-0.033	-0.037
lateral_distance_threshold	0.009	0.012	0.017
lateral_error_speed	0.4	0.4	0.4
Outras variables xerais			
switch_cooldown	2.00	1.75	1.75
min_speed	0.12	0.12	0.12
max_speed	0.20	0.20	0.30

Táboa 6.4: Valores das variables empregadas nas distintas configuracións das probas na fase de acercamento, agrupadas por subfase.

Configuración	Tempo medio (s)	Desviación estándar (s)	Tempo mínimo (s)	Tempo máximo (s)	Porcentaxe éxito (%)	Número intentos
Config. 1	244.27	61.83	168.52	338.41	100	10
Config. 2	95.60	24.73	62.31	143.79	90	10
Config. 3	25.30	12.00	11.50	54.70	90	54

Táboa 6.5: Resultados comparativos das diferentes configuracións probadas na fase de acercamento.

Distancia inicial (m)	Tempo medio (s)	Desviación estándar (s)	Tempo mínimo (s)	Tempo máximo (s)	Precisión (%)	Número intentos
1.20	40.09	13.02	22.10	70.94	90	10
1.60	38.71	12.15	29.55	66.04	90	10
2.00	33.62	10.72	20.55	55.83	40	20

Táboa 6.6: Resultados das probas realizadas a diferentes distancias iniciais.

comportamento era capaz de cumprir a súa función en condicións reais, garantindo que todas as fases previamente deseñadas e optimizadas traballasen de forma coordinada para acadar o obxectivo final.

Ademais, realizáronse probas variando diferentes parámetros físicos e de escenario para verificar a robustez e adaptabilidade do comportamento. En particular, modificouse a distancia inicial entre os obxectos de interese e tamén se cambiaron os propios obxectos de interese utilizados, co fin de demostrar que o sistema mantén un funcionamento correcto baixo condicións diversas.

Como obxectivo secundario, analizouse tamén a eficiencia temporal do comportamento, buscando identificar posibles melloras adicionais que reduzan a duración total da tarefa sen comprometer a taxa de éxito. Entre os aspectos avaliados destacaron:

- A taxa de éxito, definida como a porcentaxe de execucións nas que o obxecto de pateo impactou correctamente no obxecto destino.
- O tempo total de execución do comportamento, dende o inicio da detección ata o remate do pateo.
- A capacidade de adaptación ante cambios na distancia, verificando que o sistema mantén un rendemento satisfactorio.

Para avaliar con maior detalle o rendemento do sistema en condicións diversas, realizáronse múltiples execucións variando a distancia inicial entre os obxectos de interese. O obxectivo destas probas foi comprobar como inflúe este factor tanto no tempo total de execución como na precisión acadada. Os resultados obtidos recóllense na táboa 6.6, onde se mostran para cada distancia considerada o tempo medio investido e a taxa de éxito correspondente.

Se analizamos os resultados obtidos na táboa 6.6, en canto á precisión, o comportamento foi o esperado: ao aumentar a distancia inicial, requírese unha maior exactitude no golpeo, polo que resulta lóxico que a porcentaxe de acerto diminúa. Por outra banda, os tempos de execución presentan un comportamento diferente ao agardado. Pensábase que o achegamento ía tardar máis a maiores distancias, xa que a estimación da posición do obxecto destino depende do LiDAR e, canto máis lonxe se atopa o obxecto, menor é a densidade de puntos

Obxecto destino	Tempo medio (s)	Desviación estándar (s)	Tempo mínimo (s)	Tempo máximo (s)	Precisión (%)	Número intentos
Balón	40.09	13.02	22.10	70.94	90	10
Botella	38.70	7.96	22.54	47.35	60	10

Táboa 6.7: Resultados das probas realizadas con diferentes obxectos de interese (a unha mesma distancia inicial de 1.20 metros).

dispoñibles, o que debería dificultar unha localización estable. Porén, os resultados amosan que o tempo non aumenta de forma proporcional á distancia, senón que mesmo pode reducirse, o que suxire que a obtención da posición mantense suficientemente robusta mesmo en distancias máis longas.

Ademais destas probas baseadas na distancia, realizouse un segundo conxunto de experimentos no que se modificaron os obxectos de interese empregados, mantendo neste caso constante a distancia inicial. O propósito foi analizar a capacidade do sistema para adaptarse a variacións nos obxectos, comprobando se existen diferenzas significativas no tempo de execución ou na taxa de éxito en función do obxecto. Os resultados destas probas recóllense na táboa 6.7, onde se comparan diferentes obxectos baixo as mesmas métricas anteriores.

Analizando os resultados recollidos na táboa 6.7, pódese observar que o tempo medio requerido para completar o comportamento permaneceu relativamente constante a pesar de cambiar o obxecto destino, indicando que o sistema mantén a súa eficiencia na execución independentemente do tipo de obxecto. Por outra parte, a precisión de acerto presenta unha diferenza notable: mentres que para o balón alcanza o 90%, para a botella cae ao 60%. Esta variación é comprensible, xa que a botella ten un ancho menor que o balón, o que reduce o rango de impacto efectivo entre os obxectos de interese e, por tanto, aumenta a dificultade de acertar o golpe. En consecuencia, os resultados evidencian que a precisión depende directamente das dimensións do obxecto destino, sendo máis doada a tarefa de golpeo canto maior sexa o obxecto, debido ao incremento do rango de impacto aceptable.

Conclusións e traballo futuro

NESTE capítulo preséntanse as conclusións xerais do proxecto, destacando os principais logros e aprendizaxes acadados durante o seu desenvolvemento. Tamén se expoñen posibles liñas de mellora e traballo futuro, orientadas a optimizar o sistema e explorar novas aplicacións no ámbito da robótica móbil autónoma.

7.1 Conclusións

Este proxecto permitiu desenvolver un sistema integrado de percepción e control para o robot Unitree Go2, combinando información procedente de sensores [LiDAR](#), cámaras [RGB](#) e algoritmos de visión por computador baseados en [YOLO](#). As principais conclusións son:

- **División en fases:** O comportamento estrutúrase en tres fases principais: aliñamento, achegamento e pateo final (baixo nivel). Esta separación facilitou o desenvolvemento, a depuración e a comprensión do sistema, permitindo traballar de forma modular e incremental.
- **Arquitectura modular e escalable:** O sistema está estruturado mediante nodos ROS2 e módulos propios de percepción e control, permitindo unha integración flexible de novos sensores, algoritmos e comportamentos, así como a reutilización de compoñentes para futuros desenvolvementos.
- **División do control segundo as entradas sensoriais:** O sistema estrutúrase de maneira que en cada fase do comportamento se empregan os sensores máis axeitados. Durante o aliñamento utilízase principalmente a cámara, mentres que no achegamento, cando o obxecto de pateo deixa de ser visible, é o [LiDAR](#) quen garante a súa detección. Esta división mellora a consistencia da execución e facilita a explicación do funcionamento interno do sistema, xa que cada módulo actúa cun criterio claro en función da

dispoñibilidade sensorial. Ademais, a súa implementación modular en ROS 2 asegura a replicabilidade dos resultados en condicións semellantes.

- **Percepción robusta:** A integración do **LiDAR** permitiu detectar correctamente o obxecto frontal ao robot en escenarios con obxectos laterais, obxectos desprazados e obxectos grandes detrás, garantindo a robustez do sistema fronte a interferencias externas.
- **Fusión **LiDAR**–cámara:** A combinación do **LiDAR** coa detección visual baseada en **YOLO** proporcionou estimacións precisas das posicións **3D** dos obxectos, independentemente da súa localización na imaxe e da presenza de múltiples obxectos.

Esta fusión de sensores complementarios resultou esencial, xa que cando o robot se achega ao obxecto de pateo e perde a súa visualización pola cámara, utilízase exclusivamente o **LiDAR** para garantir que a posición do obxecto de interese se manteña correctamente monitorizada. Para o obxecto destino, que permanece visible, pódese empregar a fusión de **LiDAR** e cámara para asegurar unha localización robusta mentras se perda de vista o obxecto de pateo

- **Execución eficiente do comportamento:** As probas experimentais demostraron que o sistema executa o comportamento de forma consistente, mantendo tempos de resposta adecuados, mesmo ante variacións nos obxectos de interese, aínda que a precisión do golpe depende das dimensións dos obxectos de pateo e destino.
- **Validación experimental extensiva en condicións reais de operación:** o sistema foi probado nun gran número de execucións reais, utilizando diferentes distancias, obxectos de pateo e destino, cores e configuracións.

Realizáronse probas específicas para os módulos máis críticos, como a percepción (detección de obxectos e fusión **LiDAR**–cámara) e o control (aliñamento, achegamento e pateo), asegurando que cada nodo funcionase correctamente de forma independente e dentro da arquitectura modular. Isto permitiu identificar problemas con robots reais, como atrasos de comunicación, variacións nas execucións e limitacións de precisión, e confirmar a robustez, consistencia e replicabilidade do sistema.

- **Aprendizaxe e adaptación:** Un aspecto fundamental do traballo foi o proceso de aprendizaxe asociado. Nun inicio non se dispoñía de coñecementos previos nin sobre **ROS 2** nin sobre o propio robot Unitree Go2, polo que foi necesario adquirir competencias nestes ámbitos ao longo do desenvolvemento. Este proceso permitiu comprender en profundidade tanto o funcionamento da arquitectura de **ROS 2** como a integración de sensores e algoritmos de visión, o que representou un crecemento técnico e persoal moi relevante.

Unha reflexión a destacar é que tanto o tempo de execución do comportamento como a precisión acadada poderían terse mellorado significativamente se o robot fose capaz de realizar movementos máis suaves e controlados. A dificultade para xerar desprazamentos finos e estables limitou a capacidade de axustar con exactitude a posición do robot respecto aos obxectos de interese, afectando en ocasións á eficacia do golpe. Este aspecto pon de manifesto a importancia de dispor dun control motor preciso para complementar a robustez do nodo control.

Ademais, os resultados das probas realizadas en condicións reais foron moi significativos. O sistema alcanzou taxas de éxito elevadas, con tempos medios de execución claramente mellorados nas diferentes configuracións avaliadas, chegando a reducir o tempo medio en case catro veces en comparación coas primeiras execucións, mantendo un mínimo impacto na precisión. A eficacia do comportamento revelou depender do tamaño e distancia do obxecto de interese, así como da súa disposición no espazo, demostrando a importancia dunha planificación adaptativa segundo a situación concreta.

Outro aspecto destacable foi a avaliación con distintos obxectos de pateo e destino, empregando tamén cores variadas. En xeral, os fallos na identificación de cores foron mínimos, debido a que as cores empregadas eran bastante diferentes entre elas, o que facilitou unha detección consistente. Con todo, se se quixese distinguir entre cores máis parecidas, como verde escuro (a botella utilizada) e verde claro (o balón de cor verde), sería máis probable que se producisen pequenos erros durante a execución. Este feito confirma a robustez do sistema en escenarios visuais heteroxéneos e a súa capacidade para xeneralizar con obxectos distintos aos empregados nas probas.

O gran número de execucións realizadas permitiu avaliar a reproducibilidade do sistema e identificar problemas con robots reais, como retrasos de comunicación, variacións no movemento efectivo do robot e limitacións na precisión debido á dinámica do hardware. Ademais, durante as probas tamén se puido observar a posibilidade de que algúns compoñentes se estragasen ou sufrisen desgaste, incrementando o custo temporal e o esforzo adicional de validación. Aínda así, estas experiencias proporcionan información esencial para futuras melloras do control, da planificación do comportamento e da robustez xeral do sistema.

Por outra parte, aínda que só se probaron dous tipos de obxectos detectables mediante YOLO con axustes lixeiros de adestramento, o método desenvolvido demostrou ser aplicable a outros obxectos e situacións, mostrando a súa flexibilidade e escalabilidade para aplicacións futuras en robótica autónoma e en escenarios máis complexos.

En conxunto, o proxecto confirmou a viabilidade dunha solución integrada de percepción e control para robots móbiles cuadrúpedes, combinando múltiples sensores, algoritmos de visión en tempo real e planificación reactiva, o que constitúe unha base sólida para aplicacións futuras en robótica autónoma.

7.2 Traballo futuro

En canto ás liñas futuras de desenvolvemento, resulta especialmente interesante explorar o uso de técnicas de *aprendizaxe por reforzo*. A incorporación deste enfoque permitiría que o robot aprendese estratexias máis complexas a partir da súa propia experiencia, adaptándose progresivamente a diferentes escenarios e obxectos de interese. En particular, este tipo de aprendizaxe podería aplicarse de forma selectiva ás fases do comportamento que requiren maior precisión, como os movementos "suaves" e controlados da aproximación ao obxecto de pateo. Desta maneira, complementaríase o enfoque reactivo actual cun comportamento adaptativo capaz de mellorar tanto a eficacia como a suavidade dos movementos.

Outro aspecto a mellorar atópase na comunicación coa plataforma. Actualmente, a conexión que se conseguiu vía wifi mediante *WebRTC* limita o acceso unicamente a determinados tópicos habilitados por Unitree, restrinxindo así a flexibilidade no desenvolvemento e a integración de novos módulos. Por iso, un obxectivo de futuro será configurar correctamente a conexión vía *Wi-Fi* utilizando *CycloneDDS*, o que permitiría acceder a todos os tópicos dispoñibles a través da rede Ethernet. Deste modo, ampliaríanse considerablemente as posibilidades de control, monitorización e intercambio de información co robot, mellorando tanto a escalabilidade como a capacidade de integración do sistema. Como xa se mencionou, a principal restricción é a imposibilidade de acceder ao procesador central do Go2 para facer cambios na súa configuración da rede WiFi.

Apéndices

Acceso ao repositorio do proxecto

NESTE apéndice detállase como acceder ao repositorio público de GitHub do proxecto, onde se atopa o código fonte desenvolvido durante a realización deste traballo.

Para acceder ao código coa implementación dos nodos de ROS 2 e coas funcionalidades descritas neste documento, basta con clonar o seguinte repositorio público de GitHub:

https://github.com/samuelperezfente/RobotPateoPelota_TFG

Adicionalmente, os datos empregados durante o proxecto poden ser descargados desde o seguinte repositorio:

https://github.com/samuelperezfente/RobotPateoPelota_TFG_Datos

Canal YouTube cos vídeos de probas

Co obxectivo de complementar as explicacións presentadas neste documento, creouse un canal de YouTube no cal se recompilan os vídeos das probas realizadas durante o desenvolvemento do proxecto.

O canal atópase dispoñible na seguinte ligazón:

<https://www.youtube.com/@GolpeoBalonUnitreeGo2>

Neste canal pódense visualizar distintas demostracións das fases de aliñamento, achegamento e do comportamento completo. Deste xeito, calquera lector interesado pode comprobar de maneira visual o funcionamento do sistema desenvolvido.

Creación y extracción de vídeos

A lo longo deste proxecto elaboráronse vídeos que foron posteriormente publicados en plataformas como YouTube. Neste apéndice descríbese con detalle o procedemento seguido para crear estes vídeos compostos a partir dos datos recollidos, de xeito que calquera usuario poida replicar o proceso.

C.1 Obtención das imaxes

En total empregáronse tres cámaras diferentes:

- A cámara frontal do propio robot, accesible a través do código dispoñible no repositorio principal.
- Dúas cámaras externas conectadas por USB, xestionadas mediante o paquete *usb_cam* https://github.com/ros-drivers/usb_cam, xa incluído na imaxe de Docker empregada.

Para poder acceder ás cámaras USB dentro do contedor, é preciso conceder permisos aos dispositivos correspondentes. Unha forma sinxela consiste en engadir todos os dispositivos `/dev/video*` ao iniciar o contedor Docker, isto pode ser visto no código C.1.

Unha vez accesibles, os tres tópicos das cámaras se gravaron de forma simultánea mediante o comando: *ros2 bag record*

Deste xeito obtívose un rosbag coas tres fontes de vídeo xa sincronizadas temporalmente.

C.2 Extracción de fotogramas

Os fotogramas foron extraídos empregando o paquete *image_view*, a través do script *extract_frames_rosbag.sh*, o cal pode ser encontrado no repositorio indicado no Apéndice A. Este script espera tres parámetros:

```

1 docker run -it --rm \
2   --name unitree-go2 \
3   --net=host \
4   --env="DISPLAY" \
5   --env="QT_X11_NO_MITSHM=1" \
6   --volume="/tmp/.X11-unix:/tmp/.X11-unix:rw" \
7   $(for dev in /dev/video*; do echo "--device=$dev"; done) \
8   --device=/dev/bus/usb/001/003 \
9   --privileged \
10  unitree-go2-ros

```

Código C.1: Ejecución contenedor Docker

1. **Ruta do ficheiro rosbag:** Nome do arquivo *rosbag* que contén as grabacións das cámaras. Este arquivo debe atoparse nunha carpeta *records* que se encontre co mesmo directorio pai que o script (esto pode ser cambiado nunha variable do script).
2. **Tempo de inicio:** Segundo no que se quere comezar a extracción dos fotogramas.
3. **Tempo de fin:** Segundo no que se quere finalizar a extracción.

Un exemplo práctico de uso sería:

```
./extract_frames_rosbag.sh comportamiento_completo_14_08_2025-11_01 5 70
```

Neste exemplo, *comportamiento_completo_14_08_2025-11_01* é o arquivo rosbag grabado, e os fotogramas se extraerán desde o segundo 5 ata o 70.

section*Montaxe dos vídeos Unha vez dispoñibles os fotogramas, procedeuse á súa montaxe mediante *ffmpeg*. Para crear os vídeos compostos, integráronse as tres imaxes nunha única saída de vídeo:

- A posición e o tamaño de cada imaxe na composición final definíronse mediante variables dentro do script, de forma flexible.
- As resolucións tamén foron configurables a través desas mesmas variables.

Como os tres vídeos proveñen de tópicos gravados de forma sincronizada, mantivéronse aliñados temporalmente. Non obstante, debido a que o número de fotogramas por cámara podía variar, calculouse a taxa de fotogramas (*frames per second*) necesaria para cada unha, garantindo a correcta sincronización.

C.3 Inserción de texto

Ademais das imaxes, os vídeos contiñan información textual sobre o estado do experimento:

- Mensaxes de estado, fases, distancias e erros foron rexistrados durante a execución mediante *loggers*.
- Estes rexistros almacénanse de maneira estándar no tópico */rosout*.
- No script realizouse un filtrado para conservar só os mensaxes relevantes, gardando cada un nun ficheiro cunha marca temporal relativa ao inicio do vídeo.
- Finalmente, empregouse novamente *ffmpeg* para inserir este texto nos vídeos no instante correspondente.

C.4 Resumo do proceso

En resumo, o proceso seguido foi:

1. Gravar simultaneamente as cámaras (robot + USB) nun rosbag.
2. Extraer fotogramas de cada cámara.
3. Montar os vídeos de cada cámara con *ffmpeg*.
4. Compoñer nun único vídeo as diferentes imaxes, configurando posición e resolución.
5. Engadir información textual sincronizada a partir dos logs do tópico */rosout*.

Cómpre destacar que os pasos do 2 ao 5 se realizan automaticamente polo script *extract_frames_rosbag.sh*, polo que o usuario só precisa dispoñer do rosbag gravado (paso 1) e executar dito script cos parámetros adecuados.

Este procedemento permite reproducir os vídeos empregados no proxecto de forma fiel, e pode ser adaptado a outros experimentos con múltiples cámaras e datos adicionais.

Relación de Acrónimos

2D Dúas dimensións. 31

3D Tres dimensións. vi, 15, 33, 51, 69, 70, 78

CITIC Centro de Investigación en Tecnoloxías da Información e as Comunicaciones. 19, 26, 27

CPU Central Processing Unit. 6, 7

FOV Field of View. 8, 9

HSV Hue, Saturation, Value. 30, 46, 47

LiDAR Light Detection and Ranging. vi, vii, 2, 3, 6–9, 15, 17, 19–22, 28–34, 38, 39, 41, 42, 44, 47, 48, 50–52, 61, 64, 66, 68–70, 75, 77, 78

ODOM Odometry. 31, 47, 49, 51

RGB Red, Green, Blue. vii, 3, 6–9, 15, 17, 30, 31, 47, 63, 77

ROS Robot Operating System. 11, 12, 15, 16

ROS 2 Robot Operating System 2. 2, 4, 6–8, 10–18, 20, 21, 24, 30, 46, 78

RViz ROS Visualization Tool. 51

TF2 Transform Frames 2. 48, 49

WebRTC Web Real-Time Communication. 14, 15, 21, 80

YOLO You Only Look Once. vi–viii, 8, 16, 17, 21, 30, 46, 47, 51, 52, 64, 65, 69, 70, 77, 78

Glosario

bounding box Área rectangular que rodea un objeto detectado en una imagen. Proporciona una aproximación de su localización y extensión, utilizada habitualmente en algoritmos de detección y seguimiento.. [17](#), [30](#), [36](#), [37](#), [47](#), [51–54](#)

frame Sistema de referencia utilizado para expresar posiciones y orientaciones de objetos en el espacio. En robótica, cada sensor o parte del robot suele tener un frame asociado (por ejemplo, `odom`, `base_link`, `camera`), y las transformaciones entre ellos permiten relacionar datos en un mismo contexto espacial.. [31](#), [33](#), [47](#), [48](#)

marcador Objeto gráfico utilizado en herramientas de visualización como RViz para mostrar posiciones, trayectorias o resultados de procesos de detección en un espacio tridimensional.. [30](#), [51](#)

nube de puntos Representación geométrica formada por un conjunto de coordenadas tridimensionales obtenidas mediante sensores como LIDAR o cámaras 3D, empleada para modelar el entorno físico.. [8](#), [13](#), [17](#), [29–33](#), [47–51](#)

Bibliografía

- [1] U. Robotics, “Unitree go2 - specifications and features,” 2024. [En línea]. Disponible en: <https://www.unitree.com/go2>
- [2] —, “Lidar l1 - 4d ultra-wide fov lidar sensor,” 2024. [En línea]. Disponible en: <https://www.unitree.com/LiDAR>
- [3] Intel, “Intel® realSense™ depth camera d435i - especificaciones técnicas,” 2020. [En línea]. Disponible en: https://www.codico.com/media/productattach/i/n/intel_d435i_tech_spec_158645_2.pdf
- [4] H. Kitano, M. Asada, Y. Kuniyoshi, I. Noda, and E. Osawa, “Robocup: The robot world cup initiative,” in *Proceedings of the First International Conference on Autonomous Agents*, ser. AGENTS ’97. New York, NY, USA: Association for Computing Machinery, 1997, p. 340–347. [En línea]. Disponible en: <https://doi.org/10.1145/267658.267738>
- [5] H. Kitano, M. Asada, Y. Kuniyoshi, I. Noda, E. Osawai, and H. Matsubara, “Robocup: A challenge problem for ai and robotics,” in *RoboCup-97: Robot Soccer World Cup I*, H. Kitano, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1998, pp. 1–19.
- [6] H. Kitano, M. Asada, Y. Kuniyoshi, I. Noda, E. Osawa, and H. Matsubara, “Robocup: A challenge problem for ai,” *AI Magazine*, vol. 18, no. 1, p. 73, Mar. 1997. [En línea]. Disponible en: <https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/view/1276>
- [7] M. Asada, M. M. Veloso, M. Tambe, I. Noda, H. Kitano, and G. K. Kraetzschmar, “Overview of robocup-98,” *AI Magazine*, vol. 21, no. 1, p. 9, Mar. 2000. [En línea]. Disponible en: <https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/view/1490>
- [8] M. Asada, H. Kitano, I. Noda, and M. Veloso, “Robocup: Today and tomorrow—what we have learned,” *Artificial Intelligence*, vol. 110, no. 2, pp. 193–214, 1999. [En línea]. Disponible en: <https://www.sciencedirect.com/science/article/pii/S0004370299000247>

- [9] B. H. Bennett, D. Nelson, R. Pannier, T. Sullivan, and G. Robinson-Riegler, “Mind: Introduction to cognitive science – a review,” *AI Magazine*, vol. 19, no. 1, p. 133, Mar. 1998. [En línea]. Disponible en: <https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/view/1360>
- [10] Y. Ji, Z. Li, Y. Sun, X. B. Peng, S. Levine, G. Berseth, and K. Sreenath, “Hierarchical reinforcement learning for precise soccer shooting skills using a quadrupedal robot,” in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, pp. 1479–1486.
- [11] Z. Su, Y. Gao, E. Lukas, Y. Li, J. Cai, F. Tulbah, F. Gao, C. Yu, Z. Li, Y. Wu, and K. Sreenath, “Toward real-world cooperative and competitive soccer with quadrupedal robot teams,” 2025. [En línea]. Disponible en: <https://arxiv.org/abs/2505.13834>
- [12] X. Liu, L. Yang, Z. Chen, J. Zhong, and F. Gao, “Motion planning for a legged robot with dynamic characteristics,” *Sensors*, vol. 24, no. 18, 2024. [En línea]. Disponible en: <https://www.mdpi.com/1424-8220/24/18/6070>
- [13] M. R. van de Molengraft, D. D. Hameeteman, W. Kouw, and J. Elfring, “Applying imitation with reinforcement learning for ball manipulation with a quadruped platform,” 2024.
- [14] Y. Li and J. Ibanez-Guzman, “Lidar for autonomous driving: The principles, challenges, and trends for automotive lidar and perception systems,” *IEEE Signal Processing Magazine*, vol. 37, no. 4, pp. 50–61, 2020.
- [15] D. Inc., “Docker documentation,” 2024. [En línea]. Disponible en: <https://docs.docker.com>
- [16] Open Robotics, “Ros 2 dds middleware overview,” 2023. [En línea]. Disponible en: <https://docs.ros.org/en/humble/How-To-Guides/DDS-tuning.html>
- [17] —, “About ros 2,” 2023. [En línea]. Disponible en: <https://docs.ros.org/en/humble/Concepts.html>
- [18] O. Robotics, “Ros 2 documentation,” 2024. [En línea]. Disponible en: <https://docs.ros.org/en/humble/index.html>
- [19] Y. Maruyama, S. Kato, and T. Azumi, “Exploring the performance of ros2,” in *Proceedings of the 13th International Conference on Embedded Software*, ser. EMSOFT ’16. New York, NY, USA: Association for Computing Machinery, 2016. [En línea]. Disponible en: <https://doi.org/10.1145/2968478.2968502>
- [20] Open Robotics, “Ros 2 tutorials and concepts,” 2023. [En línea]. Disponible en: <https://docs.ros.org/en/humble/Tutorials.html>

- [21] G. Pardo-Castellote, “Omg data-distribution service: architectural overview,” in *23rd International Conference on Distributed Computing Systems Workshops, 2003. Proceedings.*, 2003, pp. 200–206.
- [22] V. Bode, C. Trinitis, M. Schulz, D. Buettner, and T. Preklik, “Adopting user-space networking for dds message-oriented middleware,” in *2024 IEEE International Conference on Pervasive Computing and Communications (PerCom)*, 2024, pp. 36–46.
- [23] —, “Dds implementations as real-time middleware – a systematic evaluation,” in *2023 IEEE 29th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, 2023, pp. 186–195.
- [24] —, “Advancing user-space networking for dds message-oriented middleware: Further extensions,” *Pervasive and Mobile Computing*, vol. 107, p. 102013, 2025. [En línea]. Disponible en: <https://www.sciencedirect.com/science/article/pii/S1574119225000021>
- [25] S. Loreto and S. P. Romano, *Real-time communication with WebRTC: peer-to-peer in the browser.* ” O’Reilly Media, Inc.”, 2014.
- [26] B. Sredojev, D. Samardzija, and D. Posarac, “Webrtc technology overview and signaling solution design and implementation,” in *2015 38th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*, 2015, pp. 1006–1009.
- [27] I. H. Chowdhury, “Integrated monitoring and control interface for unitree go2 robot,” 2025.
- [28] N. Koenig and A. Howard, “Design and use paradigms for gazebo, an open-source multi-robot simulator,” in *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566)*, vol. 3, 2004, pp. 2149–2154 vol.3. [En línea]. Disponible en: <https://ieeexplore.ieee.org/abstract/document/1389727>
- [29] O. Robotics, “Gazebo classic documentation,” 2024. [En línea]. Disponible en: <https://classic.gazebosim.org/tutorials>
- [30] —, “Sensors in gazebo classic,” 2023. [En línea]. Disponible en: <https://classic.gazebosim.org/tutorials?cat=sensors>
- [31] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State, “Isaac gym: High performance gpu-based physics simulation for robot learning,” 2021. [En línea]. Disponible en: <https://arxiv.org/abs/2108.10470>

- [32] Z. Shu, "Reinforcement learning for quadruped robot locomotion control," *HKU Theses Online (HKUTO)*, 2024.
- [33] M. Rojas, G. Hermosilla, D. Yunge, and G. Farias, "An easy to use deep reinforcement learning library for ai mobile robots in isaac sim," *Applied Sciences*, vol. 12, no. 17, 2022. [En línea]. Disponible en: <https://www.mdpi.com/2076-3417/12/17/8429>
- [34] T. Diwan, G. Anirudh, and J. V. Tembhurne, "Object detection using yolo: challenges, architectural successors, datasets and applications," *multimedia Tools and Applications*, vol. 82, no. 6, pp. 9243–9275, 2023.
- [35] J. Du, "Understanding of object detection based on cnn family and yolo," *Journal of Physics: Conference Series*, vol. 1004, no. 1, p. 012029, apr 2018. [En línea]. Disponible en: <https://dx.doi.org/10.1088/1742-6596/1004/1/012029>
- [36] C. Liu, Y. Tao, J. Liang, K. Li, and Y. Chen, "Object detection based on yolo network," in *2018 IEEE 4th Information Technology and Mechatronics Engineering Conference (ITOEC)*, 2018, pp. 799–803.
- [37] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.
- [38] G. Jocher and Ultralytics, "Yolov8: New vision ai architectures," 2023. [En línea]. Disponible en: <https://github.com/ultralytics/ultralytics>
- [39] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, D. Hunt, R. Jeffries *et al.*, "Manifesto for agile software development," 2001, disponible en: <https://agilemanifesto.org/>.
- [40] L. Williams, "Agile software development methodologies and practices," in *Advances in Computers*, ser. Advances in Computers, M. V. Zelkowitz, Ed. Elsevier, 2010, vol. 80, pp. 1–44. [En línea]. Disponible en: <https://www.sciencedirect.com/science/article/pii/S0065245810800014>
- [41] A. Koch, "Agile software development," 2004.
- [42] T. Dingsøyr, S. Nerur, V. Balijepally, and N. B. Moe, "A decade of agile methodologies: Towards explaining agile software development," pp. 1213–1221, 2012.
- [43] S. Ilieva, P. Ivanov, and E. Stefanova, "Analyses of an agile methodology implementation," in *Proceedings. 30th Euromicro Conference, 2004.*, 2004, pp. 326–333.

- [44] K. Schwaber and J. Sutherland, “The scrum guide,” 2020, disponible en: <https://scrumguides.org>.
- [45] A. Srivastava, S. Bhardwaj, and S. Saraswat, “Scrum model for agile methodology,” in *2017 International Conference on Computing, Communication and Automation (ICCCA)*, 2017, pp. 864–869.
- [46] R. C. Arkin, “Reactive robotic systems,” *The handbook of brain theory and neural networks*, pp. 793–796, 1995.
- [47] A. Billard, S. Mirrazavi, and N. Figueroa, *Learning for adaptive and reactive robot control: a dynamical systems approach*. Mit Press, 2022.
- [48] A. Lager, G. Spampinato, A. V. Papadopoulos, and T. Nolte, “Towards reactive robot applications in dynamic environments,” in *2019 24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2019, pp. 1603–1606.
- [49] E. Baklouti, N. B. Amor, and M. Jallouli, “Reactive control architecture for mobile robot autonomous navigation,” *Robotics and Autonomous Systems*, vol. 89, pp. 9–14, 2017. [En línea]. Disponible en: <https://www.sciencedirect.com/science/article/pii/S0921889016305413>
- [50] A. Ram and J. C. Santamaria, “Multistrategy learning in reactive control systems for autonomous robotic navigation,” *Informatica*, vol. 17, no. 4, pp. 347–369, 1993.
- [51] G. Franck, C. Vincent, and C. Francois, “A reactive multi-agent system for localization and tracking in mobile robotics,” in *16th IEEE International Conference on Tools with Artificial Intelligence*, 2004, pp. 431–435.